

Calcul matriciel, opérations, inversibilité et représentations des applications linéaires

Charles EDOU NZE

Table des matières

Jalon 9 : Calcul matriciel, opérations, inversibilité et représentations des applications linéaires	4
1. Présentation du concept clé	4
2. Formalisation & Rigueur Académique	5
A. Définitions Formelles	5
B. Théorèmes, Propositions & Lemmes	6
3. Le Noyau Dur : Démonstrations Pas-à-Pas	8
Démonstration du Théorème Pivot : Matrice d'une composée et produit matriciel	8
4. Exercices d'Application & Pratique de Concours	9
Exercice 1 : Application Directe (Inversion de matrice 2x2)	9
Exercice 2 : Niveau Avancé (Noyau et Rang matriciel)	11
5. Ancrage & Application en Intelligence Artificielle	13
6. Liens Sémantiques & Maillage Obsidian	14
Concepts Futurs dépendants : [[Jalon 10 (Changements de base)]], [[Jalon 29 (Éléments propres)]], [[Jalon 43 (Systèmes différentiels linéaires d'ordre 1 et calcul de l'exponentielle de matrice.)]]	14
Exercices d'Application	15
Exercice 1 : Opérations Linéaires Élémentaires sur les Matrices	16
Énoncé	16
Correction Détaillée	16
1. Détermination de la matrice $C = A + B$	16
2. Détermination de la matrice $D = A - B$	17
3. Détermination de la matrice $E = \lambda A$	17
Exercice 2 : Calcul du Produit Matriciel	18
Énoncé	18
Correction Détaillée	18
Exercice 3 : Produit Matriciel et Propriété de Transposition	20
Énoncé	20
Correction Détaillée	20
1. Calcul du produit matriciel $C = AB$	20
2. Détermination des matrices transposées A^T et B^T	21
3. Calcul du produit matriciel $D = B^T A^T$	21
4. Vérification de l'égalité $D = C^T$	21
Exercice 4 : Résolution d'une équation matricielle impliquant la transposition	23
Énoncé	23
Correction Détaillée	23
Exercice 5 : Inversibilité et inverse d'une matrice paramétrée, application à un système	

linéaire	25
Énoncé	25
Correction Détaillée	25
Question 1 : Détermination de l'ensemble des valeurs de x pour lesquelles M_x est inversible. .	25
Question 2 : Calcul de l'inverse M_x^{-1} pour les valeurs où M_x est inversible.	26
Question 3 : Résolution du système linéaire $M_3\mathbf{v} = \mathbf{b}$ pour $x = 3$	26
Exercice 6 : Représentation matricielle d'une application linéaire et changement de base	29
Énoncé	29
Correction Détaillée	29
Question 1 : Détermination de la matrice A de f dans la base canonique $\mathcal{B}$	29
Question 2 : Détermination de la matrice de passage P et justification de son inversibilité . .	30
Question 3 : Calcul de la matrice inverse P^{-1}	30
Question 4 : Détermination de la matrice B de f dans la base $\mathcal{B}'$	31
Exercice 7 : Étude d'une application linéaire sur l'espace des matrices carrées d'ordre 2	33
Énoncé	33
Correction Détaillée	33
Question 1 : Démontrer que Φ_A est une application linéaire.	33
Question 2 : Déterminer la matrice $M_{\mathcal{B}}(\Phi_A)$ représentant l'application linéaire Φ_A dans la base $\mathcal{B}$	34
Question 3 : Déterminer le noyau $\text{Ker}(\Phi_A)$ et l'image $\text{Im}(\Phi_A)$ de Φ_A . En déduire si Φ_A est injective, surjective, ou bijective.	35
Exercice 8 : Polynômes de Matrices, Polynôme Minimal et Inversibilité	37
Énoncé	37
Correction Détaillée	37
1. Détermination du polynôme caractéristique $P_A(X)$	37
2. Vérification du théorème de Cayley-Hamilton	38
3. Détermination du polynôme minimal $\mu_A(X)$	39
4. Expression de A^{-1} en fonction de A	40
5. Expression de A^n en fonction de A	40
Exercice 9 : Analyse de la Nilpotence et de l'Inversibilité dans les Algèbres de Matrices	42
Énoncé	42
Correction Détaillée	42
Exercice 10 : Décomposition de Fitting et Projecteurs Associés pour une Matrice Annulée par un Polynôme Non-Simple	47
Énoncé	47
Correction Détaillée	47
1. Analyse du Polynôme Annulateur et du Spectre	47
2. Décomposition de l'Espace Vectoriel	48
3. Propriétés des Matrices Commutantes	49
Travaux Pratiques	51
TP 1 : Représentation et Opérations Fondamentales sur les Matrices	52
Objectif	52
Implémentation Python pur	52
Explications	56
Représentation des Matrices	56
Validation et Robustesse	56
Complexité Algorithmique	56
Choix de Conception "From Scratch"	57

Précision des Nombres Flottants	57
TP 2 : Multiplication Matricielle et Transposition	58
Objectif	58
Implémentation Python pur	58
Explications	64
Conception de la classe <code>Matrix</code>	64
Opérations implémentées	64
Assertions et validation	64
TP 3 : Opérations Fondamentales sur les Matrices et Transformations Linéaires	66
Objectif	66
Implémentation Python pur	66
Explications	71
Représentation des Matrices	71
Complexité des Opérations	71
Assertions et Robustesse	71
Importance des Transformations Linéaires	72
TP 04 : Implémentation du Pivot de Gauss	73
Objectif	73
Implémentation Python pur	73
Explications	74
TP 05 : Calcul du Déterminant Récursif et Inversion	75
Objectif	75
Implémentation Python pur	75
Explications	76

Jalon 9 : Calcul matriciel, opérations, inversibilité et représentations des applications linéaires

1. Présentation du concept clé

Cette section doit rendre le concept physique, visuel ou métaphorique sans utiliser aucun formalisme mathématique complexe. - **La Genèse et la Nécessité Mathématique** : L'idée de la matrice, bien que formalisée au XIXe siècle par des mathématiciens comme Cayley, trouve ses racines dans la résolution de systèmes d'équations linéaires, une préoccupation millénaire. Dès l'Antiquité, les Chinois utilisaient des méthodes équivalentes à l'élimination de Gauss pour résoudre des problèmes pratiques. Cependant, ces méthodes étaient laborieuses et spécifiques à chaque système. Le besoin d'une notation compacte et d'un cadre algébrique pour manipuler ces systèmes est devenu pressant avec l'avènement de la géométrie analytique et la description des transformations dans l'espace. Avant les matrices, décrire une transformation linéaire (comme une rotation, une mise à l'échelle, une projection) sur des vecteurs nécessitait d'écrire des systèmes d'équations complexes pour chaque composante. Pour des espaces de grande dimension, cela devenait ingérable, obscurcissant la structure sous-jacente de la transformation. Les matrices ont été inventées pour compacter toute cette information en un seul objet rectangulaire. Elles permettent de représenter des transformations complexes de manière concise et de calculer l'effet de transformations successives (composition) par une simple opération algébrique : le produit matriciel. C'est une révolution pour la concision, l'efficacité calculatoire et la compréhension structurelle des phénomènes linéaires. Elles ont transformé la manière dont nous abordons non seulement l'algèbre linéaire, mais aussi la physique, l'ingénierie et, plus récemment, l'intelligence artificielle.

- **La Métaphore des Machines de Transformation** : Imaginez une série de machines dans une usine. Chaque machine prend des matières premières (des nombres, des "quantités") et les transforme en produits finis selon une recette fixe. Une **matrice** est précisément cette "recette" ou ce "plan de transformation" encodé dans un tableau de nombres. Elle décrit comment les entrées sont mélangées, pondérées et transformées pour produire les sorties.
 - Si vous avez une machine qui transforme des pigments bruts en couleurs primaires, et une autre machine qui transforme ces couleurs primaires en teintes spécifiques de peinture, la **multiplication de matrices** est l'équivalent de brancher ces deux machines l'une après l'autre. Le résultat est une seule "super-machine" qui va directement des pigments bruts aux teintes de peinture finales, encapsulant l'effet combiné des deux transformations.
 - L'**inversibilité** d'une matrice, c'est comme savoir si votre machine de transformation est réversible. Pouvez-vous, à partir du produit fini, retrouver exactement les matières premières d'origine, sans aucune perte d'information ? Si la machine est conçue de telle sorte qu'à chaque produit correspond une unique combinaison de matières premières, alors elle est inversible. Si, au contraire, elle écrase, mélange irréversiblement, ou perd des informations (par exemple, si plusieurs combinaisons de matières premières aboutissent au même produit, ou si certains produits ne peuvent pas être obtenus), alors elle n'est pas inversible.
- **Visualisation Géométrique** : Une matrice 2×2 peut être visualisée comme une transformation

géométrique du plan. Elle prend le carré unité (défini par les vecteurs de base canonique $(1, 0)$ et $(0, 1)$) et le déforme en un parallélogramme. Les colonnes de la matrice sont les vecteurs où atterrissent $(1, 0)$ et $(0, 1)$ après la transformation.

- Si la matrice est inversible, le parallélogramme résultant a une aire non nulle : l'espace est étiré, compressé ou pivoté, mais aucune dimension n'est perdue. Vous pouvez "défaire" la transformation pour revenir à l'état initial.
- Si la matrice n'est pas inversible, le parallélogramme est "écrasé" en une ligne ou même un point (son aire est nulle). Cela signifie que la transformation a réduit la dimension de l'espace, rendant impossible de retrouver l'état initial : l'information a été irrémédiablement perdue.

2. Formalisation & Rigueur Académique

Le niveau bascule ici instantanément dans l'exigence pure des mathématiques supérieures.

A. Définitions Formelles

Soient $\mathbb{K}$ un corps commutatif (typiquement $\mathbb{R}$ ou $\mathbb{C}$), et $n, p, q \in \mathbb{N}^*$ des entiers naturels non nuls.

1. **Matrice** ($M \in \mathcal{M}_{n,p}(\mathbb{K})$) : Un tableau rectangulaire de n lignes et p colonnes dont les éléments sont des scalaires du corps $\mathbb{K}$. Une matrice A est notée $A = (a_{i,j})$ où $a_{i,j}$ est le coefficient situé à l'intersection de la i -ème ligne ($1 \leq i \leq n$) et de la j -ème colonne ($1 \leq j \leq p$). L'ensemble de toutes ces matrices est noté $\mathcal{M}_{n,p}(\mathbb{K})$. Si $n = p$, on parle de matrice carrée d'ordre n et on note $\mathcal{M}_n(\mathbb{K})$. *Exemple* : La matrice $A = \begin{pmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{pmatrix}$ est une matrice de $\mathcal{M}_{2,3}(\mathbb{R})$. Ici, $a_{1,1} = 1$, $a_{1,2} = 2$, $a_{2,3} = 6$.

2. **Matrice Nulle** ($0_{n,p}$) : La matrice dont tous les coefficients sont nuls. $0_{n,p} = (0)_{i,j}$ pour tout $1 \leq i \leq n$ et $1 \leq j \leq p$. *Exemple* : $0_{2,3} = \begin{pmatrix} 0 & 0 & 0 \\ 0 & 0 & 0 \end{pmatrix}$.

3. **Matrice Identité** (I_n) : La matrice carrée d'ordre n dont les coefficients diagonaux sont égaux à 1 et tous les autres sont nuls. $I_n = (\delta_{i,j})$ où $\delta_{i,j}$ est le symbole de Kronecker, défini par $\delta_{i,j} = 1$ si $i = j$, et $\delta_{i,j} = 0$ si $i \neq j$. *Exemple* : $I_3 = \begin{pmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{pmatrix}$.

4. **Somme Matricielle** : Soient $A = (a_{i,j}) \in \mathcal{M}_{n,p}(\mathbb{K})$ et $B = (b_{i,j}) \in \mathcal{M}_{n,p}(\mathbb{K})$. Leur somme $C = A + B \in \mathcal{M}_{n,p}(\mathbb{K})$ est définie par l'addition de leurs coefficients correspondants :

$$c_{i,j} = a_{i,j} + b_{i,j} \quad \text{pour tout } 1 \leq i \leq n, 1 \leq j \leq p$$

$$\text{Exemple : } \begin{pmatrix} 1 & 2 \\ 3 & 4 \end{pmatrix} + \begin{pmatrix} 5 & 6 \\ 7 & 8 \end{pmatrix} = \begin{pmatrix} 1+5 & 2+6 \\ 3+7 & 4+8 \end{pmatrix} = \begin{pmatrix} 6 & 8 \\ 10 & 12 \end{pmatrix}.$$

5. **Multiplication par un scalaire** : Soit $A = (a_{i,j}) \in \mathcal{M}_{n,p}(\mathbb{K})$ et $\lambda \in \mathbb{K}$. Le produit $\lambda A \in \mathcal{M}_{n,p}(\mathbb{K})$ est défini par la multiplication de chaque coefficient de A par le scalaire λ :

$$(\lambda A)_{i,j} = \lambda a_{i,j} \quad \text{pour tout } 1 \leq i \leq n, 1 \leq j \leq p$$

$$\text{Exemple : } 3 \begin{pmatrix} 1 & 2 \\ 3 & 4 \end{pmatrix} = \begin{pmatrix} 3 \times 1 & 3 \times 2 \\ 3 \times 3 & 3 \times 4 \end{pmatrix} = \begin{pmatrix} 3 & 6 \\ 9 & 12 \end{pmatrix}.$$

6. **Produit Matriciel** : Soit $A = (a_{i,k}) \in \mathcal{M}_{n,p}(\mathbb{K})$ et $B = (b_{k,j}) \in \mathcal{M}_{p,q}(\mathbb{K})$. Le produit $C = AB \in \mathcal{M}_{n,q}(\mathbb{K})$ est défini par :

$$c_{i,j} = \sum_{k=1}^p a_{i,k} b_{k,j} \quad \text{pour tout } 1 \leq i \leq n, 1 \leq j \leq q$$

Le nombre de colonnes de A (qui est p) doit impérativement être égal au nombre de lignes de B (qui est p). Le coefficient $c_{i,j}$ est obtenu en effectuant le produit scalaire de la i -ème ligne de A par la j -ème colonne de B . *Exemple* : Soient $A = \begin{pmatrix} 1 & 2 \\ 3 & 4 \end{pmatrix}$ et $B = \begin{pmatrix} 5 & 6 \\ 7 & 8 \end{pmatrix}$. $C = AB = \begin{pmatrix} (1 \times 5) + (2 \times 7) & (1 \times 6) + (2 \times 8) \\ (3 \times 5) + (4 \times 7) & (3 \times 6) + (4 \times 8) \end{pmatrix} = \begin{pmatrix} 5 + 14 & 6 + 16 \\ 15 + 28 & 18 + 32 \end{pmatrix} = \begin{pmatrix} 19 & 22 \\ 43 & 50 \end{pmatrix}$.

7. **Matrice Inversible** : Une matrice carrée $A \in \mathcal{M}_n(\mathbb{K})$ est dite inversible (ou régulière) s'il existe une unique matrice $B \in \mathcal{M}_n(\mathbb{K})$ telle que $AB = BA = I_n$. La matrice B , si elle existe, est appelée l'inverse de A , et est notée A^{-1} . *Exemple* : La matrice $A = \begin{pmatrix} 1 & 1 \\ 0 & 1 \end{pmatrix}$ a pour inverse $A^{-1} = \begin{pmatrix} 1 & -1 \\ 0 & 1 \end{pmatrix}$.

Vérification : $AA^{-1} = \begin{pmatrix} 1 & 1 \\ 0 & 1 \end{pmatrix} \begin{pmatrix} 1 & -1 \\ 0 & 1 \end{pmatrix} = \begin{pmatrix} 1 \cdot 1 + 1 \cdot 0 & 1 \cdot (-1) + 1 \cdot 1 \\ 0 \cdot 1 + 1 \cdot 0 & 0 \cdot (-1) + 1 \cdot 1 \end{pmatrix} = \begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix} = I_2$. De même, $A^{-1}A = \begin{pmatrix} 1 & -1 \\ 0 & 1 \end{pmatrix} \begin{pmatrix} 1 & 1 \\ 0 & 1 \end{pmatrix} = \begin{pmatrix} 1 \cdot 1 + (-1) \cdot 0 & 1 \cdot 1 + (-1) \cdot 1 \\ 0 \cdot 1 + 1 \cdot 0 & 0 \cdot 1 + 1 \cdot 1 \end{pmatrix} = \begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix} = I_2$.

8. **Représentation d'une application linéaire** : Soient E et F deux $\mathbb{K}$ -espaces vectoriels de dimensions finies p et n respectivement. Soient $\mathcal{B}_E = (e_1, \dots, e_p)$ une base de E et $\mathcal{B}_F = (f_1, \dots, f_n)$ une base de F . Pour toute application linéaire $f \in \mathcal{L}(E, F)$, la matrice de f relativement aux bases $\mathcal{B}_E$ et $\mathcal{B}_F$, notée $\text{Mat}_{\mathcal{B}_E, \mathcal{B}_F}(f)$, est la matrice $A = (a_{i,j}) \in \mathcal{M}_{n,p}(\mathbb{K})$ dont la j -ème colonne est constituée des coordonnées du vecteur $f(e_j)$ dans la base $\mathcal{B}_F$. Autrement dit, pour chaque $j \in \{1, \dots, p\}$:

$$f(e_j) = \sum_{i=1}^n a_{i,j} f_i$$

Exemple : Soit $f : \mathbb{R}^2 \rightarrow \mathbb{R}^2$ définie par $f(x, y) = (x + y, 2x - y)$. Soit $\mathcal{B} = (e_1, e_2)$ la base canonique de $\mathbb{R}^2$, avec $e_1 = (1, 0)$ et $e_2 = (0, 1)$. $f(e_1) = f(1, 0) = (1 + 0, 2 \cdot 1 - 0) = (1, 2) = 1 \cdot e_1 + 2 \cdot e_2$. $f(e_2) = f(0, 1) = (0 + 1, 2 \cdot 0 - 1) = (1, -1) = 1 \cdot e_1 + (-1) \cdot e_2$. La matrice de f dans la base canonique est donc $\text{Mat}_{\mathcal{B}, \mathcal{B}}(f) = \begin{pmatrix} 1 & 1 \\ 2 & -1 \end{pmatrix}$.

9. **Noyau d'une Matrice** ($\ker A$) : Pour une matrice $A \in \mathcal{M}_{n,p}(\mathbb{K})$, le noyau de A est l'ensemble des vecteurs colonnes $X \in \mathcal{M}_{p,1}(\mathbb{K})$ tels que $AX = 0_{n,1}$. C'est un sous-espace vectoriel de $\mathbb{K}^p$. Il correspond au noyau de l'application linéaire $f_A : \mathbb{K}^p \rightarrow \mathbb{K}^n$ canoniquement associée à A .

$$\ker A = \{X \in \mathbb{K}^p \mid AX = 0\}$$

10. **Image d'une Matrice** ($\text{Im } A$) : Pour une matrice $A \in \mathcal{M}_{n,p}(\mathbb{K})$, l'image de A est l'ensemble des vecteurs colonnes $Y \in \mathcal{M}_{n,1}(\mathbb{K})$ qui peuvent être écrits comme AX pour un certain $X \in \mathcal{M}_{p,1}(\mathbb{K})$. C'est un sous-espace vectoriel de $\mathbb{K}^n$, engendré par les colonnes de A . Il correspond à l'image de l'application linéaire $f_A : \mathbb{K}^p \rightarrow \mathbb{K}^n$ canoniquement associée à A .

$$\text{Im } A = \{AX \mid X \in \mathbb{K}^p\}$$

11. **Rang d'une Matrice** ($\text{rg } A$) : Le rang d'une matrice A est la dimension de son image, c'est-à-dire la dimension du sous-espace vectoriel engendré par ses colonnes. Il est équivalent à la dimension du sous-espace vectoriel engendré par ses lignes.

$$\text{rg } A = \dim(\text{Im } A)$$

B. Théorèmes, Propositions & Lemmes

Proposition (Structure d'espace vectoriel de $\mathcal{M}_{n,p}(\mathbb{K})$) : L'ensemble $\mathcal{M}_{n,p}(\mathbb{K})$ muni de l'addition matricielle et de la multiplication par un scalaire est un $\mathbb{K}$ -espace vectoriel de dimension np . La base canonique de cet espace est formée des matrices $E_{i,j}$ dont le coefficient à la position (i, j) est 1 et tous les autres sont nuls.

Théorème (Propriétés du Produit Matriciel) : Soient $A \in \mathcal{M}_{n,p}(\mathbb{K})$, $B \in \mathcal{M}_{p,q}(\mathbb{K})$, $C \in \mathcal{M}_{q,r}(\mathbb{K})$ et $D \in \mathcal{M}_{p,q}(\mathbb{K})$, $\lambda \in \mathbb{K}$. 1. **Associativité :** $(AB)C = A(BC)$. Le produit matriciel est associatif, ce qui signifie que l'ordre des parenthèses n'affecte pas le résultat pour une chaîne de multiplications. 2. **Distributivité :** $A(B + D) = AB + AD$ (si $A \in \mathcal{M}_{n,p}(\mathbb{K})$, $B, D \in \mathcal{M}_{p,q}(\mathbb{K})$) et $(A + B)C = AC + BC$ (si $A, B \in \mathcal{M}_{n,p}(\mathbb{K})$, $C \in \mathcal{M}_{p,q}(\mathbb{K})$). Le produit matriciel est distributif par rapport à l'addition matricielle. 3. **Homogénéité :** $\lambda(AB) = (\lambda A)B = A(\lambda B)$. La multiplication par un scalaire peut être déplacée dans un produit matriciel. 4. **Élément Neutre :** Pour toute matrice $A \in \mathcal{M}_{n,p}(\mathbb{K})$, $I_n A = A$ et $A I_p = A$. Les matrices identités jouent le rôle d'éléments neutres pour la multiplication matricielle. 5. **Non-commutativité :** En général, $AB \neq BA$. Le produit matriciel n'est pas commutatif. *Contre-exemple :* Soient $A = \begin{pmatrix} 1 & 1 \\ 0 & 1 \end{pmatrix}$ et $B = \begin{pmatrix} 1 & 0 \\ 1 & 1 \end{pmatrix}$. $AB = \begin{pmatrix} 1 & 1 \\ 0 & 1 \end{pmatrix} \begin{pmatrix} 1 & 0 \\ 1 & 1 \end{pmatrix} = \begin{pmatrix} 1 \cdot 1 + 1 \cdot 1 & 1 \cdot 0 + 1 \cdot 1 \\ 0 \cdot 1 + 1 \cdot 1 & 0 \cdot 0 + 1 \cdot 1 \end{pmatrix} = \begin{pmatrix} 2 & 1 \\ 1 & 1 \end{pmatrix}$. $BA = \begin{pmatrix} 1 & 0 \\ 1 & 1 \end{pmatrix} \begin{pmatrix} 1 & 1 \\ 0 & 1 \end{pmatrix} = \begin{pmatrix} 1 \cdot 1 + 0 \cdot 0 & 1 \cdot 1 + 0 \cdot 1 \\ 1 \cdot 1 + 1 \cdot 0 & 1 \cdot 1 + 1 \cdot 1 \end{pmatrix} = \begin{pmatrix} 1 & 1 \\ 1 & 2 \end{pmatrix}$. Puisque $AB \neq BA$, le produit matriciel n'est pas commutatif.

Théorème de l'Isomorphisme entre applications linéaires et matrices : Soient E et F des $\mathbb{K}$ -espaces vectoriels de dimensions p et n respectivement, munis de bases $\mathcal{B}_E$ et $\mathcal{B}_F$. L'application $\Phi : \mathcal{L}(E, F) \rightarrow \mathcal{M}_{n,p}(\mathbb{K})$ définie par $\Phi(f) = \text{Mat}_{\mathcal{B}_E, \mathcal{B}_F}(f)$ est un isomorphisme d'espaces vectoriels. Cela signifie que l'ensemble des applications linéaires entre deux espaces vectoriels de dimension finie est structurellement identique à l'ensemble des matrices de dimensions correspondantes.

Théorème Pivot (Matrice d'une composée et produit matriciel) : Soient E, F, G des $\mathbb{K}$ -espaces vectoriels de dimensions finies q, p, n respectivement. Soient $\mathcal{B}_E, \mathcal{B}_F, \mathcal{B}_G$ des bases respectives de E, F, G . Soient $f \in \mathcal{L}(E, F)$ et $g \in \mathcal{L}(F, G)$. Alors la matrice de l'application linéaire composée $g \circ f \in \mathcal{L}(E, G)$ est le produit des matrices de g et f :

$$\text{Mat}_{\mathcal{B}_E, \mathcal{B}_G}(g \circ f) = \text{Mat}_{\mathcal{B}_F, \mathcal{B}_G}(g) \times \text{Mat}_{\mathcal{B}_E, \mathcal{B}_F}(f)$$

Ce théorème est fondamental car il établit le lien profond entre la composition des applications linéaires et le produit matriciel, justifiant la définition complexe de ce dernier.

Théorème du Rang (ou Théorème de la dimension) : Soit $f \in \mathcal{L}(E, F)$ une application linéaire où E est un espace vectoriel de dimension finie. Alors :

$$\dim E = \dim(\ker f) + \dim(\text{Im } f)$$

Pour une matrice $A \in \mathcal{M}_{n,p}(\mathbb{K})$, cela se traduit par :

$$p = \dim(\ker A) + \text{rg } A$$

où p est le nombre de colonnes de A (la dimension de l'espace de départ $\mathbb{K}^p$). Ce théorème est crucial pour comprendre la perte ou la conservation d'information lors d'une transformation linéaire.

Théorème (Caractérisations de l'inversibilité) : Soit $A \in \mathcal{M}_n(\mathbb{K})$ une matrice carrée d'ordre n . Les propriétés suivantes sont équivalentes : 1. A est inversible. 2. L'application linéaire $f_A : \mathbb{K}^n \rightarrow \mathbb{K}^n$ associée à A (dans la base canonique) est un isomorphisme (c'est-à-dire bijective). 3. $\det(A) \neq 0$. Le déterminant est un scalaire qui capture l'information sur la "déformation" de l'espace par la transformation. 4. $\text{rg}(A) = n$. La matrice a un rang maximal, ce qui signifie que ses colonnes (et ses lignes) sont linéairement indépendantes. 5. $\ker A = \{0_{\mathbb{K}^n}\}$. Le noyau de A est réduit au vecteur nul, ce qui signifie que la transformation est injective (aucun vecteur non nul n'est "écrasé" en zéro). 6. Les colonnes de A forment une base de $\mathbb{K}^n$. 7. Les lignes de A forment une base de $\mathbb{K}^n$. 8. Il existe $B \in \mathcal{M}_n(\mathbb{K})$ telle que $AB = I_n$. (L'existence d'un inverse à droite suffit pour garantir l'inversibilité). 9. Il existe $C \in \mathcal{M}_n(\mathbb{K})$ telle que $CA = I_n$. (L'existence d'un inverse à gauche suffit pour garantir l'inversibilité).

Proposition (Propriétés de l'inverse) : Soient $A, B \in \mathcal{M}_n(\mathbb{K})$ deux matrices inversibles. Alors : 1. L'inverse A^{-1} est unique. 2. A^{-1} est inversible et $(A^{-1})^{-1} = A$. 3. Le produit AB est inversible et $(AB)^{-1} = B^{-1}A^{-1}$. L'ordre est inversé, ce qui est crucial. 4. Pour tout $\lambda \in \mathbb{K}^*$, λA est inversible et $(\lambda A)^{-1} = \frac{1}{\lambda}A^{-1}$. 5. La transposée A^T est inversible et $(A^T)^{-1} = (A^{-1})^T$.

3. Le Noyau Dur : Démonstrations Pas-à-Pas

Rappel : Écris CHAQUE ligne de calcul intermédiaire sans sauter aucune étape.

Démonstration du Théorème Pivot : Matrice d'une composée et produit matriciel

Soient $f : E \rightarrow F$ et $g : F \rightarrow G$ deux applications linéaires. Soient $\mathcal{B}_E = (e_j)_{1 \leq j \leq q}$ une base de E (de dimension q). Soient $\mathcal{B}_F = (f_k)_{1 \leq k \leq p}$ une base de F (de dimension p). Soient $\mathcal{B}_G = (g_i)_{1 \leq i \leq n}$ une base de G (de dimension n).

Montrons que $\text{Mat}_{\mathcal{B}_E, \mathcal{B}_G}(g \circ f) = \text{Mat}_{\mathcal{B}_F, \mathcal{B}_G}(g) \times \text{Mat}_{\mathcal{B}_E, \mathcal{B}_F}(f)$.

1. **Initialisation / Cadre :** Nous allons calculer les coefficients de la matrice de l'application composée $g \circ f$ et montrer qu'ils correspondent à la définition du produit matriciel des matrices de g et f . Soit $A = \text{Mat}_{\mathcal{B}_F, \mathcal{B}_G}(g) = (a_{i,k}) \in \mathcal{M}_{n,p}(\mathbb{K})$. Par définition de la matrice d'une application linéaire, pour tout vecteur de base $f_k \in \mathcal{B}_F$ (où $1 \leq k \leq p$), le vecteur $g(f_k)$ s'écrit dans la base $\mathcal{B}_G$ comme une combinaison linéaire des vecteurs de $\mathcal{B}_G$ dont les coefficients forment la k -ème colonne de A :

$$g(f_k) = \sum_{i=1}^n a_{i,k} g_i \quad (*)$$

Soit $B = \text{Mat}_{\mathcal{B}_E, \mathcal{B}_F}(f) = (b_{k,j}) \in \mathcal{M}_{p,q}(\mathbb{K})$. Par définition de la matrice d'une application linéaire, pour tout vecteur de base $e_j \in \mathcal{B}_E$ (où $1 \leq j \leq q$), le vecteur $f(e_j)$ s'écrit dans la base $\mathcal{B}_F$ comme une combinaison linéaire des vecteurs de $\mathcal{B}_F$ dont les coefficients forment la j -ème colonne de B :

$$f(e_j) = \sum_{k=1}^p b_{k,j} f_k \quad (**)$$

Nous cherchons à déterminer la matrice $C = \text{Mat}_{\mathcal{B}_E, \mathcal{B}_G}(g \circ f) = (c_{i,j}) \in \mathcal{M}_{n,q}(\mathbb{K})$. Par définition, pour tout vecteur de base $e_j \in \mathcal{B}_E$, le vecteur $(g \circ f)(e_j)$ s'écrit dans la base $\mathcal{B}_G$ comme :

$$(g \circ f)(e_j) = \sum_{i=1}^n c_{i,j} g_i$$

Notre objectif est de montrer que le coefficient $c_{i,j}$ est égal à $\sum_{k=1}^p a_{i,k} b_{k,j}$, ce qui est la définition du coefficient (i,j) du produit matriciel AB .

2. **Étape 1 : Calcul de $(g \circ f)(e_j)$ en utilisant la définition de f** Nous commençons par l'expression de l'image du j -ème vecteur de base de E par l'application composée $g \circ f$:

$$(g \circ f)(e_j) = g(f(e_j))$$

En utilisant l'expression de $f(e_j)$ donnée par l'équation (**), nous substituons cette somme dans l'argument de g :

$$(g \circ f)(e_j) = g\left(\sum_{k=1}^p b_{k,j} f_k\right)$$

3. **Étape 2 : Utilisation de la linéarité de g** Puisque g est une application linéaire, elle respecte l'addition vectorielle et la multiplication par un scalaire. Nous pouvons donc "sortir" les scalaires $b_{k,j}$

de l'application g et décomposer la somme :

$$(g \circ f)(e_j) = \sum_{k=1}^p g(b_{k,j} f_k)$$

Par la propriété d'homogénéité de g (multiplication par un scalaire) :

$$(g \circ f)(e_j) = \sum_{k=1}^p b_{k,j} g(f_k)$$

4. **Étape 3 : Utilisation de la définition de g** Maintenant, nous utilisons l'expression de $g(f_k)$ donnée par l'équation (*) et nous la substituons dans la somme :

$$(g \circ f)(e_j) = \sum_{k=1}^p b_{k,j} \left(\sum_{i=1}^n a_{i,k} g_i \right)$$

5. **Étape 4 : Réarrangement des sommes** Nous avons une somme de sommes. Nous pouvons intervertir l'ordre des sommations, car les sommes sont finies et les scalaires commutent :

$$(g \circ f)(e_j) = \sum_{k=1}^p \sum_{i=1}^n (b_{k,j} a_{i,k}) g_i$$

Pour regrouper les termes selon les vecteurs de base g_i , nous réorganisons la somme en plaçant la sommation sur i à l'extérieur :

$$(g \circ f)(e_j) = \sum_{i=1}^n \left(\sum_{k=1}^p a_{i,k} b_{k,j} \right) g_i$$

Nous avons simplement réordonné les termes $b_{k,j} a_{i,k}$ en $a_{i,k} b_{k,j}$ par commutativité de la multiplication dans le corps $\mathbb{K}$.

6. **Conclusion :** Par identification avec la définition de la matrice de $(g \circ f)$, qui est $(g \circ f)(e_j) = \sum_{i=1}^n c_{i,j} g_i$, nous obtenons que le coefficient $c_{i,j}$ de la matrice $\text{Mat}_{\mathcal{B}_E, \mathcal{B}_G}(g \circ f)$ est :

$$c_{i,j} = \sum_{k=1}^p a_{i,k} b_{k,j}$$

Cette expression est précisément la définition du coefficient (i, j) du produit matriciel AB , où $A = (a_{i,k})$ et $B = (b_{k,j})$. Par conséquent, nous avons démontré que $\text{Mat}_{\mathcal{B}_E, \mathcal{B}_G}(g \circ f) = \text{Mat}_{\mathcal{B}_F, \mathcal{B}_G}(g) \times \text{Mat}_{\mathcal{B}_E, \mathcal{B}_F}(f)$.

4. Exercices d'Application & Pratique de Concours

Proposer au moins 2 exercices progressifs corrigés de façon exhaustive, sans aucune ellipse.

Exercice 1 : Application Directe (Inversion de matrice 2x2)

Énoncé : Soit la matrice $A = \begin{pmatrix} 1 & 2 \\ 3 & 4 \end{pmatrix} \in \mathcal{M}_2(\mathbb{R})$. 1. Calculer le déterminant de A . 2. Déterminer si A est inversible. 3. Si oui, calculer A^{-1} en utilisant la méthode du pivot de Gauss (ou méthode de l'élimination de Gauss-Jordan). **Correction Détaillée :** * *Analyse de l'énoncé :* L'exercice demande de vérifier l'inversibilité d'une matrice 2×2 via son déterminant, puis de calculer son inverse en utilisant une méthode systématique d'algèbre linéaire. * *Résolution pas-à-pas :* 1. **Calcul du déterminant de A :** Pour une matrice 2×2

générique, $A = \begin{pmatrix} a & b \\ c & d \end{pmatrix}$, le déterminant est défini par la formule $\det A = ad - bc$. Pour la matrice donnée $A = \begin{pmatrix} 1 & 2 \\ 3 & 4 \end{pmatrix}$, nous identifions $a = 1$, $b = 2$, $c = 3$, $d = 4$. Nous substituons ces valeurs dans la formule du déterminant :

$$\det A = (1 \times 4) - (2 \times 3)$$

Nous effectuons les multiplications :

$$\det A = 4 - 6$$

Nous effectuons la soustraction :

$$\det A = -2$$

2. Détermination de l'inversibilité de A : D'après le Théorème des Caractérisations de l'inversibilité (point 3), une matrice carrée A est inversible si et seulement si son déterminant est non nul. Nous avons calculé $\det A = -2$. Puisque $-2 \neq 0$, la matrice A est inversible. **3. Calcul de A^{-1} par la méthode du pivot de Gauss (Gauss-Jordan) :** La méthode consiste à former une matrice augmentée $(A|I_n)$, où I_n est la matrice identité de même ordre que A . Ensuite, nous appliquons une série d'opérations élémentaires sur les lignes de cette matrice augmentée pour transformer la partie gauche (A) en la matrice identité (I_n). Lorsque la partie gauche est devenue I_n , la partie droite sera l'inverse A^{-1} . Pour notre matrice $A \in \mathcal{M}_2(\mathbb{R})$, la matrice identité est $I_2 = \begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix}$. La matrice augmentée initiale est :

$$\left(\begin{array}{cc|cc} 1 & 2 & 1 & 0 \\ 3 & 4 & 0 & 1 \end{array} \right)$$

* **Étape 1 : Annuler le coefficient sous le pivot de la première colonne.** Le pivot de la première colonne est $a_{1,1} = 1$. Nous voulons annuler le coefficient $a_{2,1} = 3$. Nous effectuons l'opération élémentaire sur les lignes : $L_2 \leftarrow L_2 - 3L_1$. Calcul de la nouvelle ligne L_2 :

$$(L_2)_1 = 3 - 3 \times (L_1)_1 = 3 - 3 \times 1 = 3 - 3 = 0$$

$$(L_2)_2 = 4 - 3 \times (L_1)_2 = 4 - 3 \times 2 = 4 - 6 = -2$$

$$(L_2)_3 = 0 - 3 \times (L_1)_3 = 0 - 3 \times 1 = 0 - 3 = -3$$

$$(L_2)_4 = 1 - 3 \times (L_1)_4 = 1 - 3 \times 0 = 1 - 0 = 1$$

La nouvelle matrice augmentée est :

$$\left(\begin{array}{cc|cc} 1 & 2 & 1 & 0 \\ 0 & -2 & -3 & 1 \end{array} \right)$$

* **Étape 2 : Normaliser le pivot de la deuxième colonne.** Le pivot de la deuxième colonne est $a_{2,2} = -2$. Nous voulons qu'il soit égal à 1. Nous effectuons l'opération élémentaire sur les lignes : $L_2 \leftarrow -\frac{1}{2}L_2$. Calcul de la nouvelle ligne L_2 :

$$(L_2)_1 = -\frac{1}{2} \times 0 = 0$$

$$(L_2)_2 = -\frac{1}{2} \times (-2) = 1$$

$$(L_2)_3 = -\frac{1}{2} \times (-3) = \frac{3}{2}$$

$$(L_2)_4 = -\frac{1}{2} \times 1 = -\frac{1}{2}$$

La nouvelle matrice augmentée est :

$$\left(\begin{array}{cc|cc} 1 & 2 & 1 & 0 \\ 0 & 1 & \frac{3}{2} & -\frac{1}{2} \end{array} \right)$$

* **Étape 3 : Annuler le coefficient au-dessus du pivot de la deuxième colonne.** Le pivot de la deuxième colonne est $a_{2,2} = 1$. Nous voulons annuler le coefficient $a_{1,2} = 2$. Nous effectuons l'opération élémentaire sur les lignes : $L_1 \leftarrow L_1 - 2L_2$. Calcul de la nouvelle ligne L_1 :

$$\begin{aligned}(L_1)_1 &= 1 - 2 \times (L_2)_1 = 1 - 2 \times 0 = 1 - 0 = 1 \\(L_1)_2 &= 2 - 2 \times (L_2)_2 = 2 - 2 \times 1 = 2 - 2 = 0 \\(L_1)_3 &= 1 - 2 \times (L_2)_3 = 1 - 2 \times \frac{3}{2} = 1 - 3 = -2 \\(L_1)_4 &= 0 - 2 \times (L_2)_4 = 0 - 2 \times \left(-\frac{1}{2}\right) = 0 + 1 = 1\end{aligned}$$

La nouvelle matrice augmentée est :

$$\left(\begin{array}{cc|cc} 1 & 0 & -2 & 1 \\ 0 & 1 & \frac{3}{2} & -\frac{1}{2} \end{array} \right)$$

La partie gauche de la matrice augmentée est maintenant la matrice identité I_2 . La partie droite est l'inverse de A . **Conclusion :** L'inverse de A est $A^{-1} = \begin{pmatrix} -2 & 1 \\ \frac{3}{2} & -\frac{1}{2} \end{pmatrix}$. Pour vérifier ce résultat, nous calculons le produit AA^{-1} :

$$\begin{aligned}AA^{-1} &= \begin{pmatrix} 1 & 2 \\ 3 & 4 \end{pmatrix} \begin{pmatrix} -2 & 1 \\ \frac{3}{2} & -\frac{1}{2} \end{pmatrix} \\AA^{-1} &= \begin{pmatrix} (1)(-2) + (2)(\frac{3}{2}) & (1)(1) + (2)(-\frac{1}{2}) \\ (3)(-2) + (4)(\frac{3}{2}) & (3)(1) + (4)(-\frac{1}{2}) \end{pmatrix} \\AA^{-1} &= \begin{pmatrix} -2 + 3 & 1 - 1 \\ -6 + 6 & 3 - 2 \end{pmatrix} \\AA^{-1} &= \begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix}\end{aligned}$$

Le produit est bien la matrice identité I_2 , ce qui confirme l'exactitude de notre calcul de A^{-1} .

Exercice 2 : Niveau Avancé (Noyau et Rang matriciel)

Énoncé : Soit la matrice $M = \begin{pmatrix} 1 & 1 & 1 \\ 1 & 2 & 3 \\ 2 & 3 & 4 \end{pmatrix} \in \mathcal{M}_3(\mathbb{R})$. 1. Déterminer le rang de M . 2. Déterminer une base

du noyau de M . 3. Vérifier le théorème du rang pour cette matrice. **Correction Détaillée :** * *Analyse de l'énoncé :* Cet exercice demande de trouver le rang et le noyau d'une matrice 3×3 . Le rang est la dimension de l'image de la transformation linéaire associée, et le noyau est l'ensemble des vecteurs qui sont transformés en vecteur nul. La méthode la plus efficace pour ces deux tâches est l'échelonnement de la matrice par des opérations élémentaires sur les lignes. * *Résolution pas-à-pas :* 1. **Détermination du rang de M par échelonnement :** Le rang d'une matrice est le nombre de pivots (éléments non nuls en début de ligne) dans sa forme échelonnée. Nous allons appliquer des opérations élémentaires sur les lignes de M pour la transformer en une matrice échelonnée. La matrice initiale est :

$$M = \begin{pmatrix} 1 & 1 & 1 \\ 1 & 2 & 3 \\ 2 & 3 & 4 \end{pmatrix}$$

* **Étape 1 : Annuler les coefficients sous le premier pivot (1ère colonne, 1ère ligne).** Le pivot est $M_{1,1} = 1$. Nous voulons annuler $M_{2,1} = 1$ et $M_{3,1} = 2$. Opération pour L_2 : $L_2 \leftarrow L_2 - L_1$.

$$\begin{aligned}(L_2)_{\text{nouvelle},1} &= M_{2,1} - M_{1,1} = 1 - 1 = 0 \\(L_2)_{\text{nouvelle},2} &= M_{2,2} - M_{1,2} = 2 - 1 = 1\end{aligned}$$

$$(L_2)_{\text{nouvelle},3} = M_{2,3} - M_{1,3} = 3 - 1 = 2$$

Opération pour $L_3 : L_3 \leftarrow L_3 - 2L_1$.

$$(L_3)_{\text{nouvelle},1} = M_{3,1} - 2 \times M_{1,1} = 2 - 2 \times 1 = 2 - 2 = 0$$

$$(L_3)_{\text{nouvelle},2} = M_{3,2} - 2 \times M_{1,2} = 3 - 2 \times 1 = 3 - 2 = 1$$

$$(L_3)_{\text{nouvelle},3} = M_{3,3} - 2 \times M_{1,3} = 4 - 2 \times 1 = 4 - 2 = 2$$

La matrice devient :

$$\begin{pmatrix} 1 & 1 & 1 \\ 0 & 1 & 2 \\ 0 & 1 & 2 \end{pmatrix}$$

* **Étape 2 : Annuler les coefficients sous le deuxième pivot (2ème colonne, 2ème ligne).** Le pivot est $M_{2,2} = 1$. Nous voulons annuler $M_{3,2} = 1$. Opération pour $L_3 : L_3 \leftarrow L_3 - L_2$.

$$(L_3)_{\text{nouvelle},1} = (L_3)_{\text{ancienne},1} - (L_2)_{\text{ancienne},1} = 0 - 0 = 0$$

$$(L_3)_{\text{nouvelle},2} = (L_3)_{\text{ancienne},2} - (L_2)_{\text{ancienne},2} = 1 - 1 = 0$$

$$(L_3)_{\text{nouvelle},3} = (L_3)_{\text{ancienne},3} - (L_2)_{\text{ancienne},3} = 2 - 2 = 0$$

La matrice échelonnée est :

$$M' = \begin{pmatrix} 1 & 1 & 1 \\ 0 & 1 & 2 \\ 0 & 0 & 0 \end{pmatrix}$$

Le nombre de lignes non nulles dans la matrice échelonnée M' est 2 (les deux premières lignes). Les pivots sont 1 (en position (1,1)) et 1 (en position (2,2)). Donc, le rang de M est $\text{rg } M = 2$.

2. **Détermination d'une base du noyau de M :

Le noyau de M , noté $\ker M$, est l'ensemble des vecteurs $X = \begin{pmatrix} x \\ y \\ z \end{pmatrix}$

La résolution du système linéaire $MX=0_{\{3,1\}}$ est équivalente à la résolution du système $M'X=0_{\{3,1\}}$

Le système $M'X=0_{\{3,1\}}$ s'écrit :

$$\begin{cases} 1x + 1y + 1z = 0 & \text{(Eq. 1)} \\ 0x + 1y + 2z = 0 & \text{(Eq. 2)} \\ 0x + 0y + 0z = 0 & \text{(Eq. 3)} \end{cases}$$

L'équation (Eq. 3) est $0=0$, elle n'apporte pas d'information supplémentaire.

À partir de (Eq. 2), nous avons $y + 2z = 0$. Nous pouvons exprimer y en fonction de z :

$$y = -2z$$

Maintenant, substituons cette expression de y dans (Eq. 1) :

$$x + (-2z) + z = 0$$

$$x - z = 0$$

Nous pouvons exprimer x en fonction de z :

$$x = z$$

Les solutions du système sont donc les vecteurs X dont les composantes s'écrivent en fonction d'un

$$X = \begin{pmatrix} x \\ y \\ z \end{pmatrix} = \begin{pmatrix} z \\ -2z \\ z \end{pmatrix}$$

Nous pouvons factoriser le paramètre z :

$$X = z \begin{pmatrix} 1 \\ -2 \\ 1 \end{pmatrix}$$

Le noyau de M est l'ensemble de tous les multiples scalaires du vecteur $\begin{pmatrix} 1 \\ -2 \\ 1 \end{pmatrix}$

Par conséquent, une base du noyau de M est le singleton $\left\{ \begin{pmatrix} 1 \\ -2 \\ 1 \end{pmatrix} \right\}$

3. **Vérification du théorème du rang :

Le théorème du rang stipule que pour une application linéaire $f: E \rightarrow F$ (ou une matrice $A \in \mathcal{M}(F, E)$)

Dans notre cas, M est une matrice 3×3 , donc elle représente une application linéaire de $\mathbb{R}^3$ vers $\mathbb{R}^3$.

Nous avons trouvé :

- Le rang de M est $\text{rg } M = 2$.

- La dimension du noyau de M est $\dim(\ker M) = 1$ (car le noyau est engendré par un seul vecteur).

Vérifions le théorème du rang en substituant ces valeurs :

$$\dim E = \dim(\ker M) + \text{rg } M$$

$$3 = 1 + 2$$

$$3 = 3$$

L'égalité est vérifiée, ce qui confirme l'exactitude de nos calculs pour le rang et le noyau.

5. Ancrage & Application en Intelligence Artificielle

Démontrer la finalité technologique moderne de ce jalon théorique. - **Le Pont Théorique** : En Intelligence Artificielle, et plus particulièrement dans les réseaux de neurones profonds, le calcul matriciel est le cœur battant de toutes les opérations. Chaque couche d'un réseau de neurones effectue une transformation linéaire sur ses entrées, suivie d'une activation non linéaire. Ces transformations linéaires sont précisément représentées par des **matrices de poids**. Les "poids" et les "biais" d'un réseau de neurones sont des coefficients matriciels et vectoriels qui sont ajustés pendant l'entraînement. L'inférence (le processus de faire une prédiction avec un modèle entraîné) n'est rien d'autre qu'une succession de multiplications matricielles et d'additions vectorielles. La composition de ces transformations linéaires entre les couches est directement modélisée par le produit matriciel, comme le démontre le Théorème Pivot.

- **Exemple Concret :**

- **Inférence et Entraînement dans les Réseaux de Neurones** : Considérons une couche dense (ou *fully connected*) d'un réseau de neurones. Si l'entrée est un vecteur $x \in \mathbb{R}^p$ (représentant par exemple les activations de la couche précédente ou les caractéristiques d'entrée) et la sortie est un vecteur $h \in \mathbb{R}^n$ (les activations de la couche actuelle avant l'activation non linéaire), la transformation linéaire est donnée par $h = Wx + b$, où $W \in \mathcal{M}_{n,p}(\mathbb{R})$ est la matrice des poids et $b \in \mathbb{R}^n$ est le vecteur de biais. La multiplication matricielle Wx est l'opération fondamentale. Lors de l'entraînement, les gradients de la fonction de perte par rapport aux poids W et aux biais b sont calculés via la règle de la chaîne (rétropropagation), qui implique également des multiplications matricielles (ou des produits de Jacobi, qui sont des généralisations matricielles).
- **Accélération GPU** : Les unités de traitement graphique (GPU) sont devenues absolument indispensables pour l'entraînement des modèles d'IA. Leur architecture massivement parallèle est spécifiquement optimisée pour effectuer des millions, voire des milliards, de multiplications matricielles en virgule flottante par seconde (FLOPS). Cette capacité à exécuter des opérations matricielles en parallèle est la raison principale de leur efficacité pour les calculs massifs requis par les réseaux de neurones, où des matrices de poids de très grande taille doivent être multipliées par des vecteurs d'entrée ou d'autres matrices de gradients.
- **Optimisation des Grands Modèles de Langage (LLM) avec LoRA** : Les grands modèles de langage (LLM) comme GPT-3, Llama ou Mixtral possèdent des milliards de paramètres, principalement stockés dans d'énormes matrices de poids. Entraîner ou même ajuster (fine-tuner) ces modèles pour des tâches spécifiques est extrêmement coûteux en calcul et en mémoire. Des techniques d'optimisation comme **LoRA (Low-Rank Adaptation)** exploitent directement les concepts de rang et de produit matriciel. Au lieu de modifier directement la matrice de poids $W \in \mathcal{M}_{n,p}(\mathbb{R})$ d'origine (qui contient $n \times p$ paramètres), LoRA propose d'ajouter une petite matrice de "mise à jour" ΔW telle que la nouvelle matrice de poids effective soit $W' = W + \Delta W$. La clé est que ΔW est construite comme le produit de deux matrices de rang faible : $\Delta W = BA$, où $B \in \mathcal{M}_{n,r}(\mathbb{R})$ et $A \in \mathcal{M}_{r,p}(\mathbb{R})$ avec $r \ll \min(n, p)$. Le produit BA est une matrice de rang au plus r . Au lieu d'apprendre $n \times p$ paramètres pour W' , on n'apprend que $n \times r + r \times p$ paramètres pour B et A . Par exemple, si $n = p = 1000$ et $r = 4$, on passe de $1000 \times 1000 = 1\,000\,000$ paramètres à $1000 \times 4 + 4 \times 1000 = 8000$ paramètres. Cela réduit drastiquement le nombre de paramètres à entraîner et la mémoire requise, tout en conservant une grande partie de la performance du modèle. C'est une application directe et sophistiquée des concepts de rang et de produit matriciel pour rendre l'IA à grande échelle plus accessible et efficace.

6. Liens Sémantiques & Maillage Obsidian

- **Concepts Précédents requis :** [[Jalon-7 (Espaces vectoriels abstraits)], [[Jalon-8 (Applications linéaires)]]
-

Concepts Futurs dépendants : [[Jalon 10 (Changements de base)], [[Jalon 29 (Éléments propres)], [[Jalon 43 (Systèmes différentiels linéaires d'ordre 1 et calcul de l'exponentielle de matrice.)]]

Exercices d'Application

Exercice 1 : Opérations Linéaires Élémentaires sur les Matrices

Difficulté : ★○○○○

Énoncé

Soit $\mathbb{K}$ un corps commutatif, que nous identifierons à $\mathbb{R}$ dans le cadre de cet exercice. Nous considérons l'espace vectoriel $\mathcal{M}_{2,2}(\mathbb{K})$ des matrices carrées d'ordre 2 à coefficients dans $\mathbb{K}$.

Soient les matrices A et B définies comme suit :

$$A = \begin{pmatrix} 1 & 2 \\ 3 & 4 \end{pmatrix} \in \mathcal{M}_{2,2}(\mathbb{K})$$

$$B = \begin{pmatrix} 5 & 6 \\ 7 & 8 \end{pmatrix} \in \mathcal{M}_{2,2}(\mathbb{K})$$

Soit également le scalaire $\lambda = 3 \in \mathbb{K}$.

Déterminer les matrices $C, D, E \in \mathcal{M}_{2,2}(\mathbb{K})$ telles que : 1. $C = A + B$ 2. $D = A - B$ 3. $E = \lambda A$

Pour chacune de ces matrices, expliciter l'ensemble de ses coefficients $(M_{i,j})_{1 \leq i,j \leq 2}$.

Correction Détaillée

Nous allons procéder à la détermination de chaque matrice en appliquant les définitions formelles des opérations matricielles.

1. Détermination de la matrice $C = A + B$

Par définition, l'addition de deux matrices $A = (A_{i,j})$ et $B = (B_{i,j})$ de même dimension $n \times p$ est la matrice $C = (C_{i,j})$ de dimension $n \times p$ dont les coefficients sont donnés par $C_{i,j} = A_{i,j} + B_{i,j}$ pour tout $1 \leq i \leq n$ et $1 \leq j \leq p$. Dans notre cas, $n = 2$ et $p = 2$.

Les coefficients de C sont calculés comme suit : * $C_{1,1} = A_{1,1} + B_{1,1} = 1 + 5 = 6$ * $C_{1,2} = A_{1,2} + B_{1,2} = 2 + 6 = 8$
* $C_{2,1} = A_{2,1} + B_{2,1} = 3 + 7 = 10$ * $C_{2,2} = A_{2,2} + B_{2,2} = 4 + 8 = 12$

Ainsi, la matrice C est :

$$C = \begin{pmatrix} 6 & 8 \\ 10 & 12 \end{pmatrix}$$

2. Détermination de la matrice $D = A - B$

Par définition, la soustraction de deux matrices $A = (A_{i,j})$ et $B = (B_{i,j})$ de même dimension $n \times p$ est la matrice $D = (D_{i,j})$ de dimension $n \times p$ dont les coefficients sont donnés par $D_{i,j} = A_{i,j} - B_{i,j}$ pour tout $1 \leq i \leq n$ et $1 \leq j \leq p$. Dans notre cas, $n = 2$ et $p = 2$.

Les coefficients de D sont calculés comme suit : * $D_{1,1} = A_{1,1} - B_{1,1} = 1 - 5 = -4$ * $D_{1,2} = A_{1,2} - B_{1,2} = 2 - 6 = -4$ * $D_{2,1} = A_{2,1} - B_{2,1} = 3 - 7 = -4$ * $D_{2,2} = A_{2,2} - B_{2,2} = 4 - 8 = -4$

Ainsi, la matrice D est :

$$D = \begin{pmatrix} -4 & -4 \\ -4 & -4 \end{pmatrix}$$

3. Détermination de la matrice $E = \lambda A$

Par définition, la multiplication d'une matrice $A = (A_{i,j})$ de dimension $n \times p$ par un scalaire $\lambda \in \mathbb{K}$ est la matrice $E = (E_{i,j})$ de dimension $n \times p$ dont les coefficients sont donnés par $E_{i,j} = \lambda \cdot A_{i,j}$ pour tout $1 \leq i \leq n$ et $1 \leq j \leq p$. Dans notre cas, $n = 2$, $p = 2$, et $\lambda = 3$.

Les coefficients de E sont calculés comme suit : * $E_{1,1} = \lambda \cdot A_{1,1} = 3 \cdot 1 = 3$ * $E_{1,2} = \lambda \cdot A_{1,2} = 3 \cdot 2 = 6$ * $E_{2,1} = \lambda \cdot A_{2,1} = 3 \cdot 3 = 9$ * $E_{2,2} = \lambda \cdot A_{2,2} = 3 \cdot 4 = 12$

Ainsi, la matrice E est :

$$E = \begin{pmatrix} 3 & 6 \\ 9 & 12 \end{pmatrix}$$

Exégèse Conceptuelle et Rigueur Académique : La résolution de ce problème nécessite une compréhension profonde de la structure de $\mathbb{K}$ -espace vectoriel. Il ne suffit pas d'appliquer aveuglément les formules. Soit E un espace vectoriel sur le corps commutatif $\mathbb{K}$. Considérons un endomorphisme $u \in \mathcal{L}(E)$. La matrice représentative $M = \text{Mat}_{\mathcal{B}}(u)$ dans une base $\mathcal{B}$ encode toute l'information géométrique de u . En particulier, le théorème du rang, $\dim(E) = \dim(\ker(u)) + \text{rg}(u)$, nous assure que toute perte de dimension dans l'image est rigoureusement compensée par la dimension du noyau. La démonstration repose sur l'extraction d'une base de $\ker(u)$, complétée en une base de E , dont les images par u forment alors une base de $\text{Im}(u)$. Chaque étape du pivot de Gauss sur M correspond à un changement de base préservant le rang. Cas pathologique : si le corps $\mathbb{K}$ est de caractéristique finie, par exemple $\mathbb{F}_2$, les notions de distance et d'angle s'effondrent, mais les propriétés d'incidence algébrique encodées par la matrice demeurent intactes.

Exercice 2 : Calcul du Produit Matriciel

Difficulté : ★○○○○

Énoncé

Soit $\mathcal{M}_{m,n}(\mathbb{R})$ l'espace vectoriel des matrices à m lignes et n colonnes dont les coefficients sont des nombres réels. Considérons les deux matrices $A \in \mathcal{M}_{2,3}(\mathbb{R})$ et $B \in \mathcal{M}_{3,2}(\mathbb{R})$ définies comme suit :

$$A = \begin{pmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{pmatrix}$$

$$B = \begin{pmatrix} 7 & 8 \\ 9 & 10 \\ 11 & 12 \end{pmatrix}$$

Déterminez la matrice produit $C = AB$. Précisez les dimensions de la matrice résultante C .

Correction Détaillée

Pour calculer le produit de deux matrices $A \in \mathcal{M}_{m,n}(\mathbb{R})$ et $B \in \mathcal{M}_{n,p}(\mathbb{R})$, la matrice résultante $C = AB$ est de dimension $m \times p$, c'est-à-dire $C \in \mathcal{M}_{m,p}(\mathbb{R})$. Chaque élément C_{ij} de la matrice C est obtenu par la somme des produits des éléments de la i -ième ligne de A par les éléments correspondants de la j -ième colonne de B . Formellement, $C_{ij} = \sum_{k=1}^n A_{ik}B_{kj}$.

Dans le cas présent, $A \in \mathcal{M}_{2,3}(\mathbb{R})$ et $B \in \mathcal{M}_{3,2}(\mathbb{R})$. Par conséquent, la matrice produit $C = AB$ sera de dimension 2×2 , c'est-à-dire $C \in \mathcal{M}_{2,2}(\mathbb{R})$.

Nous devons calculer les quatre éléments de la matrice C : C_{11} , C_{12} , C_{21} , et C_{22} .

1. **Calcul de l'élément C_{11} :** L'élément C_{11} est obtenu en multipliant les éléments de la première ligne de A par les éléments de la première colonne de B et en sommant les produits. $C_{11} = A_{11}B_{11} + A_{12}B_{21} + A_{13}B_{31}$
 $C_{11} = (1)(7) + (2)(9) + (3)(11)$ $C_{11} = 7 + 18 + 33$ $C_{11} = 25 + 33$ $C_{11} = 58$
2. **Calcul de l'élément C_{12} :** L'élément C_{12} est obtenu en multipliant les éléments de la première ligne de A par les éléments de la deuxième colonne de B et en sommant les produits. $C_{12} = A_{11}B_{12} + A_{12}B_{22} + A_{13}B_{32}$
 $C_{12} = (1)(8) + (2)(10) + (3)(12)$ $C_{12} = 8 + 20 + 36$ $C_{12} = 28 + 36$ $C_{12} = 64$
3. **Calcul de l'élément C_{21} :** L'élément C_{21} est obtenu en multipliant les éléments de la deuxième ligne de A par les éléments de la première colonne de B et en sommant les produits. $C_{21} = A_{21}B_{11} + A_{22}B_{21} + A_{23}B_{31}$
 $C_{21} = (4)(7) + (5)(9) + (6)(11)$ $C_{21} = 28 + 45 + 66$ $C_{21} = 73 + 66$ $C_{21} = 139$

4. **Calcul de l'élément C_{22} :** L'élément C_{22} est obtenu en multipliant les éléments de la deuxième ligne de A par les éléments de la deuxième colonne de B et en sommant les produits. $C_{22} = A_{21}B_{12} + A_{22}B_{22} + A_{23}B_{32}$ $C_{22} = (4)(8) + (5)(10) + (6)(12)$ $C_{22} = 32 + 50 + 72$ $C_{22} = 82 + 72$ $C_{22} = 154$

En assemblant ces éléments, nous obtenons la matrice C :

$$C = \begin{pmatrix} C_{11} & C_{12} \\ C_{21} & C_{22} \end{pmatrix}$$

$$C = \begin{pmatrix} 58 & 64 \\ 139 & 154 \end{pmatrix}$$

La matrice produit $C = AB$ est donc :

$$C = \begin{pmatrix} 58 & 64 \\ 139 & 154 \end{pmatrix} \in \mathcal{M}_{2,2}(\mathbb{R})$$

Exégèse Conceptuelle et Rigueur Académique : La résolution de ce problème nécessite une compréhension profonde de la structure de $\mathbb{K}$ -espace vectoriel. Il ne suffit pas d'appliquer aveuglément les formules. Soit E un espace vectoriel sur le corps commutatif $\mathbb{K}$. Considérons un endomorphisme $u \in \mathcal{L}(E)$. La matrice représentative $M = \text{Mat}_{\mathcal{B}}(u)$ dans une base $\mathcal{B}$ encode toute l'information géométrique de u . En particulier, le théorème du rang, $\dim(E) = \dim(\ker(u)) + \text{rg}(u)$, nous assure que toute perte de dimension dans l'image est rigoureusement compensée par la dimension du noyau. La démonstration repose sur l'extraction d'une base de $\ker(u)$, complétée en une base de E , dont les images par u forment alors une base de $\text{Im}(u)$. Chaque étape du pivot de Gauss sur M correspond à un changement de base préservant le rang. Cas pathologique : si le corps $\mathbb{K}$ est de caractéristique finie, par exemple $\mathbb{F}_2$, les notions de distance et d'angle s'effondrent, mais les propriétés d'incidence algébrique encodées par la matrice demeurent intactes.

Exercice 3 : Produit Matriciel et Propriété de Transposition

Difficulté : ★★○○○

Énoncé

Soient $\mathbb{R}$ le corps des nombres réels, et $\mathcal{M}_{m,n}(\mathbb{R})$ l'espace vectoriel des matrices à m lignes et n colonnes à coefficients dans $\mathbb{R}$. Considérons les matrices $A \in \mathcal{M}_{2,3}(\mathbb{R})$ et $B \in \mathcal{M}_{3,2}(\mathbb{R})$ définies par :

$$A = \begin{pmatrix} 1 & 2 & 0 \\ -1 & 0 & 3 \end{pmatrix}$$

$$B = \begin{pmatrix} 2 & 1 \\ 0 & -1 \\ 3 & 0 \end{pmatrix}$$

1. Calculer le produit matriciel $C = AB$. Préciser les dimensions de la matrice C . 2. Déterminer les matrices transposées A^T et B^T . Préciser leurs dimensions respectives. 3. Calculer le produit matriciel $D = B^T A^T$. Préciser les dimensions de la matrice D . 4. Vérifier que la matrice D est égale à la matrice transposée de C , c'est-à-dire $D = C^T$.

Correction Détaillée

1. Calcul du produit matriciel $C = AB$

La matrice A est de dimension 2×3 et la matrice B est de dimension 3×2 . Le nombre de colonnes de A (qui est 3) est égal au nombre de lignes de B (qui est 3), donc le produit AB est bien défini. La matrice résultante $C = AB$ sera de dimension 2×2 .

Soit $C = (C_{ij})$ où $C_{ij} = \sum_{k=1}^3 A_{ik} B_{kj}$.

Calcul des éléments de C :
* $C_{11} = A_{11}B_{11} + A_{12}B_{21} + A_{13}B_{31}$ $C_{11} = (1)(2) + (2)(0) + (0)(3)$ $C_{11} = 2 + 0 + 0$
 $C_{11} = 2$

• $C_{12} = A_{11}B_{12} + A_{12}B_{22} + A_{13}B_{32}$ $C_{12} = (1)(1) + (2)(-1) + (0)(0)$ $C_{12} = 1 - 2 + 0$ $C_{12} = -1$

• $C_{21} = A_{21}B_{11} + A_{22}B_{21} + A_{23}B_{31}$ $C_{21} = (-1)(2) + (0)(0) + (3)(3)$ $C_{21} = -2 + 0 + 9$ $C_{21} = 7$

• $C_{22} = A_{21}B_{12} + A_{22}B_{22} + A_{23}B_{32}$ $C_{22} = (-1)(1) + (0)(-1) + (3)(0)$ $C_{22} = -1 + 0 + 0$ $C_{22} = -1$

Ainsi, la matrice C est :

$$C = \begin{pmatrix} 2 & -1 \\ 7 & -1 \end{pmatrix}$$

La matrice C est de dimension 2×2 .

2. Détermination des matrices transposées A^T et B^T

La transposée d'une matrice $M = (M_{ij})$ est la matrice $M^T = (M_{ji})$. Les lignes de M deviennent les colonnes de M^T , et les colonnes de M deviennent les lignes de M^T .

Pour la matrice $A \in \mathcal{M}_{2,3}(\mathbb{R})$:

$$A = \begin{pmatrix} 1 & 2 & 0 \\ -1 & 0 & 3 \end{pmatrix}$$

Sa transposée A^T est de dimension 3×2 :

$$A^T = \begin{pmatrix} 1 & -1 \\ 2 & 0 \\ 0 & 3 \end{pmatrix}$$

Pour la matrice $B \in \mathcal{M}_{3,2}(\mathbb{R})$:

$$B = \begin{pmatrix} 2 & 1 \\ 0 & -1 \\ 3 & 0 \end{pmatrix}$$

Sa transposée B^T est de dimension 2×3 :

$$B^T = \begin{pmatrix} 2 & 0 & 3 \\ 1 & -1 & 0 \end{pmatrix}$$

3. Calcul du produit matriciel $D = B^T A^T$

La matrice B^T est de dimension 2×3 et la matrice A^T est de dimension 3×2 . Le nombre de colonnes de B^T (qui est 3) est égal au nombre de lignes de A^T (qui est 3), donc le produit $B^T A^T$ est bien défini. La matrice résultante $D = B^T A^T$ sera de dimension 2×2 .

Soit $D = (D_{ij})$ où $D_{ij} = \sum_{k=1}^3 (B^T)_{ik} (A^T)_{kj}$.

Calcul des éléments de D : * $D_{11} = (B^T)_{11}(A^T)_{11} + (B^T)_{12}(A^T)_{21} + (B^T)_{13}(A^T)_{31}$ $D_{11} = (2)(1) + (0)(2) + (3)(0)$
 $D_{11} = 2 + 0 + 0$ $D_{11} = 2$

- $D_{12} = (B^T)_{11}(A^T)_{12} + (B^T)_{12}(A^T)_{22} + (B^T)_{13}(A^T)_{32}$ $D_{12} = (2)(-1) + (0)(0) + (3)(3)$ $D_{12} = -2 + 0 + 9$
 $D_{12} = 7$

- $D_{21} = (B^T)_{21}(A^T)_{11} + (B^T)_{22}(A^T)_{21} + (B^T)_{23}(A^T)_{31}$ $D_{21} = (1)(1) + (-1)(2) + (0)(0)$ $D_{21} = 1 - 2 + 0$
 $D_{21} = -1$

- $D_{22} = (B^T)_{21}(A^T)_{12} + (B^T)_{22}(A^T)_{22} + (B^T)_{23}(A^T)_{32}$ $D_{22} = (1)(-1) + (-1)(0) + (0)(3)$ $D_{22} = -1 + 0 + 0$
 $D_{22} = -1$

Ainsi, la matrice D est :

$$D = \begin{pmatrix} 2 & 7 \\ -1 & -1 \end{pmatrix}$$

La matrice D est de dimension 2×2 .

4. Vérification de l'égalité $D = C^T$

Nous avons calculé la matrice C :

$$C = \begin{pmatrix} 2 & -1 \\ 7 & -1 \end{pmatrix}$$

La transposée de C , notée C^T , est obtenue en échangeant ses lignes et ses colonnes :

$$C^T = \begin{pmatrix} 2 & 7 \\ -1 & -1 \end{pmatrix}$$

En comparant la matrice D obtenue à la question 3 et la matrice C^T que nous venons de calculer :

$$D = \begin{pmatrix} 2 & 7 \\ -1 & -1 \end{pmatrix}$$

$$C^T = \begin{pmatrix} 2 & 7 \\ -1 & -1 \end{pmatrix}$$

Nous constatons que $D_{ij} = (C^T)_{ij}$ pour tous $i, j \in \{1, 2\}$.

Par conséquent, nous avons bien vérifié que $D = C^T$, ce qui illustre la propriété générale $(AB)^T = B^T A^T$.

Exégèse Conceptuelle et Rigueur Académique : La résolution de ce problème nécessite une compréhension profonde de la structure de $\mathbb{K}$ -espace vectoriel. Il ne suffit pas d'appliquer aveuglément les formules. Soit E un espace vectoriel sur le corps commutatif $\mathbb{K}$. Considérons un endomorphisme $u \in \mathcal{L}(E)$. La matrice représentative $M = \text{Mat}_{\mathcal{B}}(u)$ dans une base $\mathcal{B}$ encode toute l'information géométrique de u . En particulier, le théorème du rang, $\dim(E) = \dim(\ker(u)) + \text{rg}(u)$, nous assure que toute perte de dimension dans l'image est rigoureusement compensée par la dimension du noyau. La démonstration repose sur l'extraction d'une base de $\ker(u)$, complétée en une base de E , dont les images par u forment alors une base de $\text{Im}(u)$. Chaque étape du pivot de Gauss sur M correspond à un changement de base préservant le rang. Cas pathologique : si le corps $\mathbb{K}$ est de caractéristique finie, par exemple $\mathbb{F}_2$, les notions de distance et d'angle s'effondrent, mais les propriétés d'incidence algébrique encodées par la matrice demeurent intactes.

Exercice 4 : Résolution d'une équation matricielle impliquant la transposition

Difficulté : ★★○○○

Énoncé

Soient A et B deux matrices appartenant à l'espace $\mathcal{M}_{2,2}(\mathbb{R})$ des matrices carrées d'ordre 2 à coefficients réels. On définit ces matrices par :

$$A = \begin{pmatrix} 1 & 2 \\ 3 & 4 \end{pmatrix}$$
$$B = \begin{pmatrix} 5 & 0 \\ -1 & 6 \end{pmatrix}$$

Déterminer l'unique matrice $X \in \mathcal{M}_{2,2}(\mathbb{R})$ satisfaisant l'équation matricielle suivante :

$$2X + A^T = B$$

où A^T désigne la transposée de la matrice A .

Correction Détaillée

Nous sommes invités à déterminer la matrice $X \in \mathcal{M}_{2,2}(\mathbb{R})$ qui est solution de l'équation matricielle $2X + A^T = B$.

Étape 1 : Isolation de la matrice X dans l'équation. L'équation donnée est :

$$2X + A^T = B$$

Pour isoler $2X$, nous soustrayons la matrice A^T des deux côtés de l'équation. Par les propriétés de l'algèbre matricielle, cette opération est bien définie dans $\mathcal{M}_{2,2}(\mathbb{R})$:

$$2X = B - A^T$$

Ensuite, pour obtenir X , nous multiplions les deux côtés de l'équation par le scalaire $\frac{1}{2}$. La multiplication scalaire est également une opération bien définie dans $\mathcal{M}_{2,2}(\mathbb{R})$:

$$X = \frac{1}{2}(B - A^T)$$

Cette expression nous indique la séquence des calculs à effectuer.

Étape 2 : Calcul de la transposée de la matrice A . La matrice A est donnée par :

$$A = \begin{pmatrix} 1 & 2 \\ 3 & 4 \end{pmatrix}$$

La transposée d'une matrice M , notée M^T , est obtenue en échangeant ses lignes et ses colonnes. Formellement, si $M = (m_{ij})$, alors $M^T = (m'_{ij})$ où $m'_{ij} = m_{ji}$. Appliquons cette définition à la matrice A :

$$A^T = \begin{pmatrix} A_{11} & A_{21} \\ A_{12} & A_{22} \end{pmatrix} = \begin{pmatrix} 1 & 3 \\ 2 & 4 \end{pmatrix}$$

Étape 3 : Calcul de la différence matricielle $B - A^T$. Nous avons les matrices B et A^T :

$$B = \begin{pmatrix} 5 & 0 \\ -1 & 6 \end{pmatrix}$$

$$A^T = \begin{pmatrix} 1 & 3 \\ 2 & 4 \end{pmatrix}$$

La soustraction de deux matrices de mêmes dimensions s'effectue élément par élément. Formellement, si $M = (m_{ij})$ et $N = (n_{ij})$, alors $M - N = (m_{ij} - n_{ij})$.

$$B - A^T = \begin{pmatrix} 5 & 0 \\ -1 & 6 \end{pmatrix} - \begin{pmatrix} 1 & 3 \\ 2 & 4 \end{pmatrix}$$

$$B - A^T = \begin{pmatrix} 5-1 & 0-3 \\ -1-2 & 6-4 \end{pmatrix}$$

Effectuons les soustractions arithmétiques pour chaque élément :

$$B - A^T = \begin{pmatrix} 4 & -3 \\ -3 & 2 \end{pmatrix}$$

Étape 4 : Multiplication par le scalaire $\frac{1}{2}$. Nous devons maintenant multiplier la matrice résultante de l'Étape 3 par le scalaire $\frac{1}{2}$ pour obtenir X .

$$X = \frac{1}{2}(B - A^T) = \frac{1}{2} \begin{pmatrix} 4 & -3 \\ -3 & 2 \end{pmatrix}$$

La multiplication d'une matrice par un scalaire s'effectue en multipliant chaque élément de la matrice par ce scalaire. Formellement, si c est un scalaire et $M = (m_{ij})$, alors $cM = (c \cdot m_{ij})$.

$$X = \begin{pmatrix} \frac{1}{2} \times 4 & \frac{1}{2} \times (-3) \\ \frac{1}{2} \times (-3) & \frac{1}{2} \times 2 \end{pmatrix}$$

Effectuons les multiplications arithmétiques pour chaque élément :

$$X = \begin{pmatrix} 2 & -\frac{3}{2} \\ -\frac{3}{2} & 1 \end{pmatrix}$$

Conclusion : La matrice X satisfaisant l'équation $2X + A^T = B$ est :

$$X = \begin{pmatrix} 2 & -\frac{3}{2} \\ -\frac{3}{2} & 1 \end{pmatrix}$$

Exégèse Conceptuelle et Rigueur Académique : La résolution de ce problème nécessite une compréhension profonde de la structure de $\mathbb{K}$ -espace vectoriel. Il ne suffit pas d'appliquer aveuglément les formules. Soit E un espace vectoriel sur le corps commutatif $\mathbb{K}$. Considérons un endomorphisme $u \in \mathcal{L}(E)$. La matrice représentative $M = \text{Mat}_{\mathcal{B}}(u)$ dans une base $\mathcal{B}$ encode toute l'information géométrique de u . En particulier, le théorème du rang, $\dim(E) = \dim(\ker(u)) + \text{rg}(u)$, nous assure que toute perte de dimension dans l'image est rigoureusement compensée par la dimension du noyau. La démonstration repose sur l'extraction d'une base de $\ker(u)$, complétée en une base de E , dont les images par u forment alors une base de $\text{Im}(u)$. Chaque étape du pivot de Gauss sur M correspond à un changement de base préservant le rang. Cas pathologique : si le corps $\mathbb{K}$ est de caractéristique finie, par exemple $\mathbb{F}_2$, les notions de distance et d'angle s'effondrent, mais les propriétés d'incidence algébrique encodées par la matrice demeurent intactes.

Exercice 5 : Inversibilité et inverse d'une matrice paramétrée, application à un système linéaire

Difficulté : ★★☆☆

Énoncé

Soit x un scalaire réel, c'est-à-dire $x \in \mathbb{R}$. Considérons la matrice $M_x \in \mathcal{M}_2(\mathbb{R})$ définie par :

$$M_x = \begin{pmatrix} x & 1 \\ 1 & x \end{pmatrix}$$

1. Déterminer l'ensemble des valeurs de $x \in \mathbb{R}$ pour lesquelles la matrice M_x est inversible.
2. Pour tout x appartenant à cet ensemble, calculer l'inverse M_x^{-1} de la matrice M_x .
3. Soit le vecteur colonne $\mathbf{b} = \begin{pmatrix} 2 \\ 4 \end{pmatrix} \in \mathbb{R}^2$. Pour la valeur spécifique $x = 3$, résoudre le système d'équations linéaires $M_3 \mathbf{v} = \mathbf{b}$, où $\mathbf{v} = \begin{pmatrix} v_1 \\ v_2 \end{pmatrix} \in \mathbb{R}^2$.

Correction Détaillée

Question 1 : Détermination de l'ensemble des valeurs de x pour lesquelles M_x est inversible.

Une matrice carrée est inversible si et seulement si son déterminant est non nul. Nous allons donc calculer le déterminant de la matrice M_x .

Pour une matrice $A = \begin{pmatrix} a & b \\ c & d \end{pmatrix} \in \mathcal{M}_2(\mathbb{R})$, son déterminant est donné par la formule :

$$\det(A) = ad - bc$$

Appliquons cette formule à la matrice $M_x = \begin{pmatrix} x & 1 \\ 1 & x \end{pmatrix}$. Ici, nous avons $a = x$, $b = 1$, $c = 1$, et $d = x$.

$$\det(M_x) = (x)(x) - (1)(1)$$

$$\det(M_x) = x^2 - 1$$

La matrice M_x est inversible si et seulement si son déterminant est non nul :

$$\det(M_x) \neq 0$$

$$x^2 - 1 \neq 0$$

Cette inégalité peut être factorisée en utilisant l'identité remarquable $a^2 - b^2 = (a - b)(a + b)$:

$$(x - 1)(x + 1) \neq 0$$

Pour qu'un produit de deux facteurs soit non nul, il faut que chacun des facteurs soit non nul. Ainsi, nous avons :

$$x - 1 \neq 0 \quad \text{et} \quad x + 1 \neq 0$$

Ce qui implique :

$$x \neq 1 \quad \text{et} \quad x \neq -1$$

Par conséquent, la matrice M_x est inversible pour toutes les valeurs de $x \in \mathbb{R}$ à l'exception de $x = 1$ et $x = -1$. L'ensemble des valeurs de x pour lesquelles M_x est inversible est $\mathbb{R} \setminus \{-1, 1\}$.

Question 2 : Calcul de l'inverse M_x^{-1} pour les valeurs où M_x est inversible.

Pour une matrice $A = \begin{pmatrix} a & b \\ c & d \end{pmatrix} \in \mathcal{M}_2(\mathbb{R})$ inversible (c'est-à-dire $\det(A) = ad - bc \neq 0$), son inverse A^{-1} est donnée par la formule :

$$A^{-1} = \frac{1}{\det(A)} \begin{pmatrix} d & -b \\ -c & a \end{pmatrix}$$

Nous avons déjà calculé $\det(M_x) = x^2 - 1$. Pour la matrice $M_x = \begin{pmatrix} x & 1 \\ 1 & x \end{pmatrix}$, nous avons $a = x$, $b = 1$, $c = 1$, et $d = x$.

En substituant ces valeurs dans la formule de l'inverse, pour $x \in \mathbb{R} \setminus \{-1, 1\}$:

$$M_x^{-1} = \frac{1}{x^2 - 1} \begin{pmatrix} x & -1 \\ -1 & x \end{pmatrix}$$

Nous pouvons également écrire cette matrice en distribuant le facteur scalaire $\frac{1}{x^2 - 1}$ à chaque élément de la matrice :

$$M_x^{-1} = \begin{pmatrix} \frac{x}{x^2 - 1} & \frac{-1}{x^2 - 1} \\ \frac{-1}{x^2 - 1} & \frac{x}{x^2 - 1} \end{pmatrix}$$

Question 3 : Résolution du système linéaire $M_3 \mathbf{v} = \mathbf{b}$ pour $x = 3$.

Pour $x = 3$, la matrice M_x devient M_3 .

$$M_3 = \begin{pmatrix} 3 & 1 \\ 1 & 3 \end{pmatrix}$$

Vérifions d'abord si M_3 est inversible. D'après la question 1, M_x est inversible pour $x \in \mathbb{R} \setminus \{-1, 1\}$. Puisque $3 \notin \{-1, 1\}$, la matrice M_3 est bien inversible. Son déterminant est $\det(M_3) = 3^2 - 1 = 9 - 1 = 8$. Puisque $\det(M_3) = 8 \neq 0$, M_3 est inversible.

Le système d'équations linéaires est $M_3 \mathbf{v} = \mathbf{b}$. Puisque M_3 est inversible, nous pouvons multiplier les deux côtés de l'équation par M_3^{-1} à gauche pour isoler le vecteur $\mathbf{v}$:

$$M_3^{-1}(M_3 \mathbf{v}) = M_3^{-1} \mathbf{b}$$

Par associativité de la multiplication matricielle, nous avons :

$$(M_3^{-1} M_3) \mathbf{v} = M_3^{-1} \mathbf{b}$$

Puisque le produit d'une matrice par son inverse est la matrice identité ($M_3^{-1}M_3 = I_2$, où $I_2 = \begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix}$ est la matrice identité d'ordre 2) :

$$I_2 \mathbf{v} = M_3^{-1} \mathbf{b}$$

La multiplication par la matrice identité ne change pas le vecteur, donc :

$$\mathbf{v} = M_3^{-1} \mathbf{b}$$

Nous utilisons la formule de l'inverse trouvée à la question 2, en substituant $x = 3$:

$$M_3^{-1} = \frac{1}{3^2 - 1} \begin{pmatrix} 3 & -1 \\ -1 & 3 \end{pmatrix}$$

$$M_3^{-1} = \frac{1}{9 - 1} \begin{pmatrix} 3 & -1 \\ -1 & 3 \end{pmatrix}$$

$$M_3^{-1} = \frac{1}{8} \begin{pmatrix} 3 & -1 \\ -1 & 3 \end{pmatrix}$$

Maintenant, nous calculons le produit $M_3^{-1} \mathbf{b}$ avec $\mathbf{b} = \begin{pmatrix} 2 \\ 4 \end{pmatrix}$:

$$\mathbf{v} = \frac{1}{8} \begin{pmatrix} 3 & -1 \\ -1 & 3 \end{pmatrix} \begin{pmatrix} 2 \\ 4 \end{pmatrix}$$

Effectuons la multiplication matrice-vecteur. Le résultat est un vecteur colonne dont les éléments sont obtenus par le produit scalaire des lignes de la matrice (sans le facteur $\frac{1}{8}$ pour l'instant) avec le vecteur $\mathbf{b}$.

Le premier élément du vecteur résultant est :

$$(3)(2) + (-1)(4) = 6 - 4 = 2$$

Le second élément du vecteur résultant est :

$$(-1)(2) + (3)(4) = -2 + 12 = 10$$

Donc, le produit matriciel donne :

$$\mathbf{v} = \frac{1}{8} \begin{pmatrix} 2 \\ 10 \end{pmatrix}$$

Enfin, nous distribuons le facteur scalaire $\frac{1}{8}$ aux éléments du vecteur :

$$\mathbf{v} = \begin{pmatrix} \frac{2}{8} \\ \frac{10}{8} \end{pmatrix}$$

En simplifiant les fractions :

$$\mathbf{v} = \begin{pmatrix} \frac{1}{4} \\ \frac{5}{4} \end{pmatrix}$$

La solution du système linéaire $M_3 \mathbf{v} = \mathbf{b}$ est donc $\mathbf{v} = \begin{pmatrix} 1/4 \\ 5/4 \end{pmatrix}$.

Exégèse Conceptuelle et Rigueur Académique : La résolution de ce problème nécessite une compréhension profonde de la structure de $\mathbb{K}$ -espace vectoriel. Il ne suffit pas d'appliquer aveuglément les formules. Soit E un espace vectoriel sur le corps commutatif $\mathbb{K}$. Considérons un endomorphisme $u \in \mathcal{L}(E)$. La matrice représentative $M = \text{Mat}_{\mathcal{B}}(u)$ dans une base $\mathcal{B}$ encode toute l'information géométrique de u . En particulier, le

théorème du rang, $\dim(E) = \dim(\ker(u)) + \text{rg}(u)$, nous assure que toute perte de dimension dans l'image est rigoureusement compensée par la dimension du noyau. La démonstration repose sur l'extraction d'une base de $\ker(u)$, complétée en une base de E , dont les images par u forment alors une base de $\text{Im}(u)$. Chaque étape du pivot de Gauss sur M correspond à un changement de base préservant le rang. Cas pathologique : si le corps $\mathbb{K}$ est de caractéristique finie, par exemple $\mathbb{F}_2$, les notions de distance et d'angle s'effondrent, mais les propriétés d'incidence algébrique encodées par la matrice demeurent intactes.

Exercice 6 : Représentation matricielle d'une application linéaire et changement de base

Difficulté : ★★☆☆

Énoncé

Soit $E = \mathbb{R}^3$ l'espace vectoriel réel muni de sa base canonique $\mathcal{B} = (e_1, e_2, e_3)$, où $e_1 = (1, 0, 0)$, $e_2 = (0, 1, 0)$, et $e_3 = (0, 0, 1)$.

On considère l'application linéaire $f : E \rightarrow E$ définie pour tout vecteur $v = (x, y, z) \in E$ par l'expression analytique suivante :

$$f(v) = (x + 2y - z, y + z, x - y + 2z)$$

On introduit une nouvelle base de E , notée $\mathcal{B}' = (u_1, u_2, u_3)$, dont les vecteurs sont donnés par leurs coordonnées dans la base canonique $\mathcal{B}$:

$$u_1 = (1, 1, 0)_{\mathcal{B}}, \quad u_2 = (0, 1, 1)_{\mathcal{B}}, \quad u_3 = (1, 0, 1)_{\mathcal{B}}$$

1. Déterminer la matrice $A \in \mathcal{M}_{3,3}(\mathbb{R})$ de l'application linéaire f dans la base canonique $\mathcal{B}$.
2. Déterminer la matrice de passage $P \in \mathcal{M}_{3,3}(\mathbb{R})$ de la base $\mathcal{B}$ à la base $\mathcal{B}'$. Justifier rigoureusement l'inversibilité de la matrice P .
3. Calculer la matrice inverse $P^{-1} \in \mathcal{M}_{3,3}(\mathbb{R})$.
4. Déterminer la matrice $B \in \mathcal{M}_{3,3}(\mathbb{R})$ de l'application linéaire f dans la base $\mathcal{B}'$.

Correction Détaillée

Question 1 : Détermination de la matrice A de f dans la base canonique $\mathcal{B}$

La matrice A de l'application linéaire f dans la base canonique $\mathcal{B} = (e_1, e_2, e_3)$ est obtenue en exprimant les images des vecteurs de la base $\mathcal{B}$ par f comme colonnes de A . Les vecteurs de la base canonique sont $e_1 = (1, 0, 0)$, $e_2 = (0, 1, 0)$, et $e_3 = (0, 0, 1)$.

Calculons les images de ces vecteurs par f : Pour $e_1 = (1, 0, 0)$: $f(e_1) = f(1, 0, 0) = (1 + 2(0) - 0, 0 + 0, 1 - 0 + 2(0)) = (1, 0, 1)$. Ce vecteur est la première colonne de A .

Pour $e_2 = (0, 1, 0)$: $f(e_2) = f(0, 1, 0) = (0 + 2(1) - 0, 1 + 0, 0 - 1 + 2(0)) = (2, 1, -1)$. Ce vecteur est la deuxième colonne de A .

Pour $e_3 = (0, 0, 1)$: $f(e_3) = f(0, 0, 1) = (0 + 2(0) - 1, 0 + 1, 0 - 0 + 2(1)) = (-1, 1, 2)$. Ce vecteur est la troisième colonne de A .

Ainsi, la matrice A est :

$$A = \begin{pmatrix} 1 & 2 & -1 \\ 0 & 1 & 1 \\ 1 & -1 & 2 \end{pmatrix}$$

Question 2 : Détermination de la matrice de passage P et justification de son inversibilité

La matrice de passage P de la base $\mathcal{B}$ à la base $\mathcal{B}'$ est formée en plaçant les coordonnées des vecteurs de la base $\mathcal{B}'$ exprimées dans la base $\mathcal{B}$ en colonnes. Les vecteurs de la base $\mathcal{B}'$ sont $u_1 = (1, 1, 0)$, $u_2 = (0, 1, 1)$, et $u_3 = (1, 0, 1)$.

Donc, la matrice P est :

$$P = \begin{pmatrix} 1 & 0 & 1 \\ 1 & 1 & 0 \\ 0 & 1 & 1 \end{pmatrix}$$

Pour justifier l'inversibilité de P , nous devons calculer son déterminant. Une matrice carrée est inversible si et seulement si son déterminant est non nul. Calculons $\det(P)$ en utilisant le développement par rapport à la première ligne :

$$\det(P) = 1 \cdot \det \begin{pmatrix} 1 & 0 \\ 1 & 1 \end{pmatrix} - 0 \cdot \det \begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix} + 1 \cdot \det \begin{pmatrix} 1 & 1 \\ 0 & 1 \end{pmatrix}$$

Calculons les déterminants des sous-matrices 2×2 : $\det \begin{pmatrix} 1 & 0 \\ 1 & 1 \end{pmatrix} = (1 \cdot 1) - (0 \cdot 1) = 1 - 0 = 1$. $\det \begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix} = (1 \cdot 1) - (0 \cdot 0) = 1 - 0 = 1$. $\det \begin{pmatrix} 1 & 1 \\ 0 & 1 \end{pmatrix} = (1 \cdot 1) - (1 \cdot 0) = 1 - 0 = 1$.

Substituons ces valeurs dans l'expression du déterminant de P :

$$\det(P) = 1 \cdot (1) - 0 \cdot (1) + 1 \cdot (1)$$

$$\det(P) = 1 - 0 + 1$$

$$\det(P) = 2$$

Puisque $\det(P) = 2 \neq 0$, la matrice P est inversible. Cela confirme également que $\mathcal{B}'$ est bien une base de $\mathbb{R}^3$.

Question 3 : Calcul de la matrice inverse P^{-1}

Nous allons utiliser la méthode de la comatrice pour calculer P^{-1} . La formule est $P^{-1} = \frac{1}{\det(P)}(\text{com}(P))^T$, où $\text{com}(P)$ est la matrice des cofacteurs de P . Nous avons déjà calculé $\det(P) = 2$.

Calculons les cofacteurs $C_{ij} = (-1)^{i+j} M_{ij}$, où M_{ij} est le mineur de l'élément P_{ij} .

$$P = \begin{pmatrix} 1 & 0 & 1 \\ 1 & 1 & 0 \\ 0 & 1 & 1 \end{pmatrix}$$

Première ligne : $C_{11} = (-1)^{1+1} \det \begin{pmatrix} 1 & 0 \\ 1 & 1 \end{pmatrix} = +1 \cdot ((1 \cdot 1) - (0 \cdot 1)) = 1 \cdot (1 - 0) = 1$. $C_{12} = (-1)^{1+2} \det \begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix} = -1 \cdot ((1 \cdot 1) - (0 \cdot 0)) = -1 \cdot (1 - 0) = -1$. $C_{13} = (-1)^{1+3} \det \begin{pmatrix} 1 & 1 \\ 0 & 1 \end{pmatrix} = +1 \cdot ((1 \cdot 1) - (1 \cdot 0)) = 1 \cdot (1 - 0) = 1$.

Deuxième ligne : $C_{21} = (-1)^{2+1} \det \begin{pmatrix} 0 & 1 \\ 1 & 1 \end{pmatrix} = -1 \cdot ((0 \cdot 1) - (1 \cdot 1)) = -1 \cdot (0 - 1) = -1 \cdot (-1) = 1$. $C_{22} = (-1)^{2+2} \det \begin{pmatrix} 1 & 1 \\ 0 & 1 \end{pmatrix} = +1 \cdot ((1 \cdot 1) - (1 \cdot 0)) = 1 \cdot (1 - 0) = 1$. $C_{23} = (-1)^{2+3} \det \begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix} = -1 \cdot ((1 \cdot 1) - (0 \cdot 0)) = -1 \cdot (1 - 0) = -1$.

Troisième ligne : $C_{31} = (-1)^{3+1} \det \begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix} = +1 \cdot ((0 \cdot 0) - (1 \cdot 1)) = 1 \cdot (0 - 1) = -1$. $C_{32} = (-1)^{3+2} \det \begin{pmatrix} 1 & 1 \\ 1 & 0 \end{pmatrix} = -1 \cdot ((1 \cdot 0) - (1 \cdot 1)) = -1 \cdot (0 - 1) = -1 \cdot (-1) = 1$. $C_{33} = (-1)^{3+3} \det \begin{pmatrix} 1 & 0 \\ 1 & 1 \end{pmatrix} = +1 \cdot ((1 \cdot 1) - (0 \cdot 1)) = 1 \cdot (1 - 0) = 1$.

La matrice des cofacteurs $\text{com}(P)$ est :

$$\text{com}(P) = \begin{pmatrix} 1 & -1 & 1 \\ 1 & 1 & -1 \\ -1 & 1 & 1 \end{pmatrix}$$

La transposée de la matrice des cofacteurs, $(\text{com}(P))^T$, est appelée la matrice adjointe de P :

$$(\text{com}(P))^T = \begin{pmatrix} 1 & 1 & -1 \\ -1 & 1 & 1 \\ 1 & -1 & 1 \end{pmatrix}$$

Enfin, la matrice inverse P^{-1} est :

$$P^{-1} = \frac{1}{\det(P)} (\text{com}(P))^T = \frac{1}{2} \begin{pmatrix} 1 & 1 & -1 \\ -1 & 1 & 1 \\ 1 & -1 & 1 \end{pmatrix}$$

$$P^{-1} = \begin{pmatrix} 1/2 & 1/2 & -1/2 \\ -1/2 & 1/2 & 1/2 \\ 1/2 & -1/2 & 1/2 \end{pmatrix}$$

Question 4 : Détermination de la matrice B de f dans la base $\mathcal{B}'$

La matrice B de l'application linéaire f dans la base $\mathcal{B}'$ est donnée par la formule de changement de base :

$$B = P^{-1}AP. \text{ Nous avons } A = \begin{pmatrix} 1 & 2 & -1 \\ 0 & 1 & 1 \\ 1 & -1 & 2 \end{pmatrix}, P = \begin{pmatrix} 1 & 0 & 1 \\ 1 & 1 & 0 \\ 0 & 1 & 1 \end{pmatrix}, \text{ et } P^{-1} = \frac{1}{2} \begin{pmatrix} 1 & 1 & -1 \\ -1 & 1 & 1 \\ 1 & -1 & 1 \end{pmatrix}.$$

Commençons par calculer le produit AP :

$$AP = \begin{pmatrix} 1 & 2 & -1 \\ 0 & 1 & 1 \\ 1 & -1 & 2 \end{pmatrix} \begin{pmatrix} 1 & 0 & 1 \\ 1 & 1 & 0 \\ 0 & 1 & 1 \end{pmatrix}$$

Calcul de chaque élément de AP : $(AP)_{11} = (1)(1) + (2)(1) + (-1)(0) = 1 + 2 + 0 = 3$. $(AP)_{12} = (1)(0) + (2)(1) + (-1)(1) = 0 + 2 - 1 = 1$. $(AP)_{13} = (1)(1) + (2)(0) + (-1)(1) = 1 + 0 - 1 = 0$.

$(AP)_{21} = (0)(1) + (1)(1) + (1)(0) = 0 + 1 + 0 = 1$. $(AP)_{22} = (0)(0) + (1)(1) + (1)(1) = 0 + 1 + 1 = 2$. $(AP)_{23} = (0)(1) + (1)(0) + (1)(1) = 0 + 0 + 1 = 1$.

$(AP)_{31} = (1)(1) + (-1)(1) + (2)(0) = 1 - 1 + 0 = 0$. $(AP)_{32} = (1)(0) + (-1)(1) + (2)(1) = 0 - 1 + 2 = 1$. $(AP)_{33} = (1)(1) + (-1)(0) + (2)(1) = 1 + 0 + 2 = 3$.

Donc, la matrice AP est :

$$AP = \begin{pmatrix} 3 & 1 & 0 \\ 1 & 2 & 1 \\ 0 & 1 & 3 \end{pmatrix}$$

Maintenant, calculons $B = P^{-1}(AP)$:

$$B = \frac{1}{2} \begin{pmatrix} 1 & 1 & -1 \\ -1 & 1 & 1 \\ 1 & -1 & 1 \end{pmatrix} \begin{pmatrix} 3 & 1 & 0 \\ 1 & 2 & 1 \\ 0 & 1 & 3 \end{pmatrix}$$

Calcul de chaque élément de $2B$: $(2B)_{11} = (1)(3) + (1)(1) + (-1)(0) = 3 + 1 + 0 = 4$. $(2B)_{12} = (1)(1) + (1)(2) + (-1)(1) = 1 + 2 - 1 = 2$. $(2B)_{13} = (1)(0) + (1)(1) + (-1)(3) = 0 + 1 - 3 = -2$.

$(2B)_{21} = (-1)(3) + (1)(1) + (1)(0) = -3 + 1 + 0 = -2$. $(2B)_{22} = (-1)(1) + (1)(2) + (1)(1) = -1 + 2 + 1 = 2$. $(2B)_{23} = (-1)(0) + (1)(1) + (1)(3) = 0 + 1 + 3 = 4$.

$(2B)_{31} = (1)(3) + (-1)(1) + (1)(0) = 3 - 1 + 0 = 2$. $(2B)_{32} = (1)(1) + (-1)(2) + (1)(1) = 1 - 2 + 1 = 0$. $(2B)_{33} = (1)(0) + (-1)(1) + (1)(3) = 0 - 1 + 3 = 2$.

Donc, la matrice $2B$ est :

$$2B = \begin{pmatrix} 4 & 2 & -2 \\ -2 & 2 & 4 \\ 2 & 0 & 2 \end{pmatrix}$$

Enfin, en divisant par 2, nous obtenons la matrice B :

$$B = \begin{pmatrix} 2 & 1 & -1 \\ -1 & 1 & 2 \\ 1 & 0 & 1 \end{pmatrix}$$

Exégèse Conceptuelle et Rigueur Académique : La résolution de ce problème nécessite une compréhension profonde de la structure de $\mathbb{K}$ -espace vectoriel. Il ne suffit pas d'appliquer aveuglément les formules. Soit E un espace vectoriel sur le corps commutatif $\mathbb{K}$. Considérons un endomorphisme $u \in \mathcal{L}(E)$. La matrice représentative $M = \text{Mat}_{\mathcal{B}}(u)$ dans une base $\mathcal{B}$ encode toute l'information géométrique de u . En particulier, le théorème du rang, $\dim(E) = \dim(\ker(u)) + \text{rg}(u)$, nous assure que toute perte de dimension dans l'image est rigoureusement compensée par la dimension du noyau. La démonstration repose sur l'extraction d'une base de $\ker(u)$, complétée en une base de E , dont les images par u forment alors une base de $\text{Im}(u)$. Chaque étape du pivot de Gauss sur M correspond à un changement de base préservant le rang. Cas pathologique : si le corps $\mathbb{K}$ est de caractéristique finie, par exemple $\mathbb{F}_2$, les notions de distance et d'angle s'effondrent, mais les propriétés d'incidence algébrique encodées par la matrice demeurent intactes.

Exercice 7 : Étude d'une application linéaire sur l'espace des matrices carrées d'ordre 2

Difficulté : ★★☆☆

Énoncé

Soit $E = \mathcal{M}_2(\mathbb{R})$ l'espace vectoriel des matrices carrées d'ordre 2 à coefficients réels. On munit E de la base canonique ordonnée $\mathcal{B} = (E_{11}, E_{12}, E_{21}, E_{22})$, où E_{ij} désigne la matrice élémentaire dont le coefficient à la i -ème ligne et j -ème colonne est 1 et tous les autres sont nuls. Soit $A \in \mathcal{M}_2(\mathbb{R})$ la matrice définie par $A = \begin{pmatrix} 1 & 1 \\ 0 & 1 \end{pmatrix}$. On considère l'application $\Phi_A : E \rightarrow E$ définie pour toute matrice $M \in E$ par $\Phi_A(M) = AM - MA$.

1. Démontrer que Φ_A est une application linéaire.
2. Déterminer la matrice $M_{\mathcal{B}}(\Phi_A)$ représentant l'application linéaire Φ_A dans la base $\mathcal{B}$.
3. Déterminer le noyau $\text{Ker}(\Phi_A)$ et l'image $\text{Im}(\Phi_A)$ de Φ_A . En déduire si Φ_A est injective, surjective, ou bijective.

Correction Détaillée

Question 1 : Démontrer que Φ_A est une application linéaire.

Pour démontrer que Φ_A est une application linéaire, nous devons vérifier deux propriétés : 1. Additivité : $\forall M_1, M_2 \in E, \Phi_A(M_1 + M_2) = \Phi_A(M_1) + \Phi_A(M_2)$. 2. Homogénéité : $\forall \lambda \in \mathbb{R}, \forall M \in E, \Phi_A(\lambda M) = \lambda \Phi_A(M)$.

Soient $M_1, M_2 \in E$ et $\lambda \in \mathbb{R}$.

Vérification de l'additivité : $\Phi_A(M_1 + M_2) = A(M_1 + M_2) - (M_1 + M_2)A$ En utilisant la distributivité de la multiplication matricielle par rapport à l'addition : $A(M_1 + M_2) = AM_1 + AM_2$ $(M_1 + M_2)A = M_1A + M_2A$ Donc, $\Phi_A(M_1 + M_2) = (AM_1 + AM_2) - (M_1A + M_2A)$ $\Phi_A(M_1 + M_2) = AM_1 + AM_2 - M_1A - M_2A$ En réarrangeant les termes : $\Phi_A(M_1 + M_2) = (AM_1 - M_1A) + (AM_2 - M_2A)$ Par définition de Φ_A : $\Phi_A(M_1 + M_2) = \Phi_A(M_1) + \Phi_A(M_2)$ L'additivité est vérifiée.

Vérification de l'homogénéité : $\Phi_A(\lambda M) = A(\lambda M) - (\lambda M)A$ En utilisant la propriété de scalarité de la multiplication matricielle : $A(\lambda M) = \lambda(AM)$ $(\lambda M)A = \lambda(MA)$ Donc, $\Phi_A(\lambda M) = \lambda(AM) - \lambda(MA)$ En factorisant le scalaire λ : $\Phi_A(\lambda M) = \lambda(AM - MA)$ Par définition de Φ_A : $\Phi_A(\lambda M) = \lambda \Phi_A(M)$ L'homogénéité est vérifiée.

Puisque Φ_A satisfait les deux propriétés d'additivité et d'homogénéité, nous pouvons conclure que Φ_A est une application linéaire.

Question 2 : Déterminer la matrice $M_{\mathcal{B}}(\Phi_A)$ représentant l'application linéaire Φ_A dans la base $\mathcal{B}$.

La base $\mathcal{B}$ de $E = \mathcal{M}_2(\mathbb{R})$ est donnée par : $E_{11} = \begin{pmatrix} 1 & 0 \\ 0 & 0 \end{pmatrix}$, $E_{12} = \begin{pmatrix} 0 & 1 \\ 0 & 0 \end{pmatrix}$, $E_{21} = \begin{pmatrix} 0 & 0 \\ 1 & 0 \end{pmatrix}$, $E_{22} = \begin{pmatrix} 0 & 0 \\ 0 & 1 \end{pmatrix}$.

La matrice A est $A = \begin{pmatrix} 1 & 1 \\ 0 & 1 \end{pmatrix}$.

Pour construire la matrice $M_{\mathcal{B}}(\Phi_A)$, nous devons calculer $\Phi_A(E_{ij})$ pour chaque matrice de base et exprimer le résultat comme une combinaison linéaire des matrices de base $E_{11}, E_{12}, E_{21}, E_{22}$. Les coefficients de ces combinaisons linéaires formeront les colonnes de $M_{\mathcal{B}}(\Phi_A)$.

Calcul de $\Phi_A(E_{11})$: $AE_{11} = \begin{pmatrix} 1 & 1 \\ 0 & 1 \end{pmatrix} \begin{pmatrix} 1 & 0 \\ 0 & 0 \end{pmatrix} = \begin{pmatrix} 1 \cdot 1 + 1 \cdot 0 & 1 \cdot 0 + 1 \cdot 0 \\ 0 \cdot 1 + 1 \cdot 0 & 0 \cdot 0 + 1 \cdot 0 \end{pmatrix} = \begin{pmatrix} 1 & 0 \\ 0 & 0 \end{pmatrix}$ $E_{11}A = \begin{pmatrix} 1 & 0 \\ 0 & 0 \end{pmatrix} \begin{pmatrix} 1 & 1 \\ 0 & 1 \end{pmatrix} = \begin{pmatrix} 1 \cdot 1 + 0 \cdot 0 & 1 \cdot 1 + 0 \cdot 1 \\ 0 \cdot 1 + 0 \cdot 0 & 0 \cdot 1 + 0 \cdot 1 \end{pmatrix} = \begin{pmatrix} 1 & 1 \\ 0 & 0 \end{pmatrix}$ $\Phi_A(E_{11}) = AE_{11} - E_{11}A = \begin{pmatrix} 1 & 0 \\ 0 & 0 \end{pmatrix} - \begin{pmatrix} 1 & 1 \\ 0 & 0 \end{pmatrix} = \begin{pmatrix} 1-1 & 0-1 \\ 0-0 & 0-0 \end{pmatrix} = \begin{pmatrix} 0 & -1 \\ 0 & 0 \end{pmatrix}$ En termes de la base $\mathcal{B}$: $\Phi_A(E_{11}) = 0 \cdot E_{11} - 1 \cdot E_{12} + 0 \cdot E_{21} + 0 \cdot E_{22}$. La

première colonne de $M_{\mathcal{B}}(\Phi_A)$ est $\begin{pmatrix} 0 \\ -1 \\ 0 \\ 0 \end{pmatrix}$.

Calcul de $\Phi_A(E_{12})$: $AE_{12} = \begin{pmatrix} 1 & 1 \\ 0 & 1 \end{pmatrix} \begin{pmatrix} 0 & 1 \\ 0 & 0 \end{pmatrix} = \begin{pmatrix} 1 \cdot 0 + 1 \cdot 0 & 1 \cdot 1 + 1 \cdot 0 \\ 0 \cdot 0 + 1 \cdot 0 & 0 \cdot 1 + 1 \cdot 0 \end{pmatrix} = \begin{pmatrix} 0 & 1 \\ 0 & 0 \end{pmatrix}$ $E_{12}A = \begin{pmatrix} 0 & 1 \\ 0 & 0 \end{pmatrix} \begin{pmatrix} 1 & 1 \\ 0 & 1 \end{pmatrix} = \begin{pmatrix} 0 \cdot 1 + 1 \cdot 0 & 0 \cdot 1 + 1 \cdot 1 \\ 0 \cdot 1 + 0 \cdot 0 & 0 \cdot 1 + 0 \cdot 1 \end{pmatrix} = \begin{pmatrix} 0 & 1 \\ 0 & 0 \end{pmatrix}$ $\Phi_A(E_{12}) = AE_{12} - E_{12}A = \begin{pmatrix} 0 & 1 \\ 0 & 0 \end{pmatrix} - \begin{pmatrix} 0 & 1 \\ 0 & 0 \end{pmatrix} = \begin{pmatrix} 0 & 0 \\ 0 & 0 \end{pmatrix}$ En termes de la base $\mathcal{B}$: $\Phi_A(E_{12}) = 0 \cdot E_{11} + 0 \cdot E_{12} + 0 \cdot E_{21} + 0 \cdot E_{22}$. La deuxième colonne de $M_{\mathcal{B}}(\Phi_A)$ est $\begin{pmatrix} 0 \\ 0 \\ 0 \\ 0 \end{pmatrix}$.

Calcul de $\Phi_A(E_{21})$: $AE_{21} = \begin{pmatrix} 1 & 1 \\ 0 & 1 \end{pmatrix} \begin{pmatrix} 0 & 0 \\ 1 & 0 \end{pmatrix} = \begin{pmatrix} 1 \cdot 0 + 1 \cdot 1 & 1 \cdot 0 + 1 \cdot 0 \\ 0 \cdot 0 + 1 \cdot 1 & 0 \cdot 0 + 1 \cdot 0 \end{pmatrix} = \begin{pmatrix} 1 & 0 \\ 1 & 0 \end{pmatrix}$ $E_{21}A = \begin{pmatrix} 0 & 0 \\ 1 & 0 \end{pmatrix} \begin{pmatrix} 1 & 1 \\ 0 & 1 \end{pmatrix} = \begin{pmatrix} 0 \cdot 1 + 0 \cdot 0 & 0 \cdot 1 + 0 \cdot 1 \\ 1 \cdot 1 + 0 \cdot 0 & 1 \cdot 1 + 0 \cdot 1 \end{pmatrix} = \begin{pmatrix} 0 & 0 \\ 1 & 1 \end{pmatrix}$ $\Phi_A(E_{21}) = AE_{21} - E_{21}A = \begin{pmatrix} 1 & 0 \\ 1 & 0 \end{pmatrix} - \begin{pmatrix} 0 & 0 \\ 1 & 1 \end{pmatrix} = \begin{pmatrix} 1-0 & 0-0 \\ 1-1 & 0-1 \end{pmatrix} = \begin{pmatrix} 1 & 0 \\ 0 & -1 \end{pmatrix}$ En termes de la base $\mathcal{B}$: $\Phi_A(E_{21}) = 1 \cdot E_{11} + 0 \cdot E_{12} + 0 \cdot E_{21} - 1 \cdot E_{22}$. La troisième colonne de $M_{\mathcal{B}}(\Phi_A)$ est $\begin{pmatrix} 1 \\ 0 \\ 0 \\ -1 \end{pmatrix}$.

Calcul de $\Phi_A(E_{22})$: $AE_{22} = \begin{pmatrix} 1 & 1 \\ 0 & 1 \end{pmatrix} \begin{pmatrix} 0 & 0 \\ 0 & 1 \end{pmatrix} = \begin{pmatrix} 1 \cdot 0 + 1 \cdot 0 & 1 \cdot 0 + 1 \cdot 1 \\ 0 \cdot 0 + 1 \cdot 0 & 0 \cdot 0 + 1 \cdot 1 \end{pmatrix} = \begin{pmatrix} 0 & 1 \\ 0 & 1 \end{pmatrix}$ $E_{22}A = \begin{pmatrix} 0 & 0 \\ 0 & 1 \end{pmatrix} \begin{pmatrix} 1 & 1 \\ 0 & 1 \end{pmatrix} = \begin{pmatrix} 0 \cdot 1 + 0 \cdot 0 & 0 \cdot 1 + 0 \cdot 1 \\ 0 \cdot 1 + 1 \cdot 0 & 0 \cdot 1 + 1 \cdot 1 \end{pmatrix} = \begin{pmatrix} 0 & 0 \\ 0 & 1 \end{pmatrix}$ $\Phi_A(E_{22}) = AE_{22} - E_{22}A = \begin{pmatrix} 0 & 1 \\ 0 & 1 \end{pmatrix} - \begin{pmatrix} 0 & 0 \\ 0 & 1 \end{pmatrix} = \begin{pmatrix} 0-0 & 1-0 \\ 0-0 & 1-1 \end{pmatrix} = \begin{pmatrix} 0 & 1 \\ 0 & 0 \end{pmatrix}$ En termes de la base $\mathcal{B}$: $\Phi_A(E_{22}) = 0 \cdot E_{11} + 1 \cdot E_{12} + 0 \cdot E_{21} + 0 \cdot E_{22}$. La quatrième colonne de $M_{\mathcal{B}}(\Phi_A)$ est $\begin{pmatrix} 0 \\ 1 \\ 0 \\ 0 \end{pmatrix}$.

En regroupant ces colonnes, la matrice $M_{\mathcal{B}}(\Phi_A)$ est : $M_{\mathcal{B}}(\Phi_A) = \begin{pmatrix} 0 & 0 & 1 & 0 \\ -1 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & -1 & 0 \end{pmatrix}$.

Question 3 : Déterminer le noyau $\text{Ker}(\Phi_A)$ et l'image $\text{Im}(\Phi_A)$ de Φ_A . En déduire si Φ_A est injective, surjective, ou bijective.

Détermination du noyau $\text{Ker}(\Phi_A)$: Le noyau de Φ_A est l'ensemble des matrices $M \in E$ telles que $\Phi_A(M) = 0$. C'est-à-dire, $AM - MA = 0$, ou $AM = MA$. Soit $M = \begin{pmatrix} a & b \\ c & d \end{pmatrix}$ une matrice générique de E .

Calculons AM : $AM = \begin{pmatrix} 1 & 1 \\ 0 & 1 \end{pmatrix} \begin{pmatrix} a & b \\ c & d \end{pmatrix} = \begin{pmatrix} 1 \cdot a + 1 \cdot c & 1 \cdot b + 1 \cdot d \\ 0 \cdot a + 1 \cdot c & 0 \cdot b + 1 \cdot d \end{pmatrix} = \begin{pmatrix} a+c & b+d \\ c & d \end{pmatrix}$. Calculons MA : $MA = \begin{pmatrix} a & b \\ c & d \end{pmatrix} \begin{pmatrix} 1 & 1 \\ 0 & 1 \end{pmatrix} = \begin{pmatrix} a \cdot 1 + b \cdot 0 & a \cdot 1 + b \cdot 1 \\ c \cdot 1 + d \cdot 0 & c \cdot 1 + d \cdot 1 \end{pmatrix} = \begin{pmatrix} a & a+b \\ c & c+d \end{pmatrix}$.

Pour que $AM = MA$, nous devons avoir l'égalité de leurs coefficients : 1. $a + c = a \implies c = 0$. 2. $b + d = a + b \implies d = a$. 3. $c = c$ (cette équation est triviale et cohérente avec $c = 0$). 4. $d = c + d \implies c = 0$ (cette équation est également cohérente avec $c = 0$).

Ainsi, une matrice $M = \begin{pmatrix} a & b \\ c & d \end{pmatrix}$ est dans $\text{Ker}(\Phi_A)$ si et seulement si $c = 0$ et $d = a$. Donc, les matrices du noyau sont de la forme $M = \begin{pmatrix} a & b \\ 0 & a \end{pmatrix}$. Nous pouvons écrire ces matrices comme une combinaison linéaire :

$M = a \begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix} + b \begin{pmatrix} 0 & 1 \\ 0 & 0 \end{pmatrix} = aI_2 + bE_{12}$. Le noyau $\text{Ker}(\Phi_A)$ est l'ensemble des matrices qui commutent avec

A . Une base de $\text{Ker}(\Phi_A)$ est $\left(\begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix}, \begin{pmatrix} 0 & 1 \\ 0 & 0 \end{pmatrix} \right)$. La dimension du noyau est $\dim(\text{Ker}(\Phi_A)) = 2$.

Déduction sur l'injectivité : Puisque $\dim(\text{Ker}(\Phi_A)) = 2 \neq 0$, l'application linéaire Φ_A n'est pas injective.

Détermination de l'image $\text{Im}(\Phi_A)$: L'image de Φ_A est l'ensemble des vecteurs (matrices) de E qui peuvent être atteints par Φ_A . L'image est engendrée par les colonnes de la matrice $M_{\mathcal{B}}(\Phi_A)$. Les colonnes

de $M_{\mathcal{B}}(\Phi_A)$ sont : $C_1 = \begin{pmatrix} 0 \\ -1 \\ 0 \\ 0 \end{pmatrix}$, $C_2 = \begin{pmatrix} 0 \\ 0 \\ 0 \\ 0 \end{pmatrix}$, $C_3 = \begin{pmatrix} 1 \\ 0 \\ 0 \\ -1 \end{pmatrix}$, $C_4 = \begin{pmatrix} 0 \\ 1 \\ 0 \\ 0 \end{pmatrix}$. En termes de matrices de base :

$\Phi_A(E_{11}) = -E_{12}$ $\Phi_A(E_{12}) = 0$ $\Phi_A(E_{21}) = E_{11} - E_{22}$ $\Phi_A(E_{22}) = E_{12}$

L'image $\text{Im}(\Phi_A)$ est l'espace vectoriel engendré par ces matrices : $\text{Im}(\Phi_A) = \text{Vect}(-E_{12}, 0, E_{11} - E_{22}, E_{12})$.

Nous pouvons simplifier cette expression : $\text{Im}(\Phi_A) = \text{Vect}(-E_{12}, E_{11} - E_{22}, E_{12})$. Puisque $-E_{12}$ et E_{12} sont colinéaires, nous pouvons réduire l'ensemble des générateurs à : $\text{Im}(\Phi_A) = \text{Vect}(E_{12}, E_{11} - E_{22})$. Ces

deux matrices, $E_{12} = \begin{pmatrix} 0 & 1 \\ 0 & 0 \end{pmatrix}$ et $E_{11} - E_{22} = \begin{pmatrix} 1 & 0 \\ 0 & -1 \end{pmatrix}$, sont linéairement indépendantes. En effet, si

$\alpha E_{12} + \beta(E_{11} - E_{22}) = 0$, alors $\begin{pmatrix} \beta & \alpha \\ 0 & -\beta \end{pmatrix} = \begin{pmatrix} 0 & 0 \\ 0 & 0 \end{pmatrix}$, ce qui implique $\alpha = 0$ et $\beta = 0$. Donc, une base de

$\text{Im}(\Phi_A)$ est $\left(\begin{pmatrix} 0 & 1 \\ 0 & 0 \end{pmatrix}, \begin{pmatrix} 1 & 0 \\ 0 & -1 \end{pmatrix} \right)$. La dimension de l'image est $\dim(\text{Im}(\Phi_A)) = 2$.

Vérification par le théorème du rang : La dimension de l'espace de départ E est $\dim(E) = 4$. Le théorème du rang stipule que $\dim(E) = \dim(\text{Ker}(\Phi_A)) + \dim(\text{Im}(\Phi_A))$. Nous avons $4 = 2 + 2$, ce qui est cohérent avec nos calculs.

Déduction sur la surjectivité et la bijectivité : Puisque $\dim(\text{Im}(\Phi_A)) = 2 \neq \dim(E) = 4$, l'application linéaire Φ_A n'est pas surjective. Puisqu'elle n'est ni injective ni surjective, Φ_A n'est pas bijective.

Résumé des conclusions : * Le noyau de Φ_A est $\text{Ker}(\Phi_A) = \left\{ \begin{pmatrix} a & b \\ 0 & a \end{pmatrix} \mid a, b \in \mathbb{R} \right\}$, de dimension 2. * L'image

de Φ_A est $\text{Im}(\Phi_A) = \left\{ \begin{pmatrix} x & y \\ 0 & -x \end{pmatrix} \mid x, y \in \mathbb{R} \right\}$, de dimension 2. * Φ_A n'est pas injective car $\text{Ker}(\Phi_A) \neq \{0\}$. * Φ_A n'est pas surjective car $\text{Im}(\Phi_A) \neq E$. * Φ_A n'est pas bijective.

Exégèse Conceptuelle et Rigueur Académique : La résolution de ce problème nécessite une compréhension profonde de la structure de $\mathbb{K}$ -espace vectoriel. Il ne suffit pas d'appliquer aveuglément les formules. Soit E un espace vectoriel sur le corps commutatif $\mathbb{K}$. Considérons un endomorphisme $u \in \mathcal{L}(E)$. La matrice représentative $M = \text{Mat}_{\mathcal{B}}(u)$ dans une base $\mathcal{B}$ encode toute l'information géométrique de u . En particulier, le théorème du rang, $\dim(E) = \dim(\ker(u)) + \text{rg}(u)$, nous assure que toute perte de dimension dans l'image est rigoureusement compensée par la dimension du noyau. La démonstration repose sur l'extraction d'une base de $\ker(u)$, complétée en une base de E , dont les images par u forment alors une base de $\text{Im}(u)$. Chaque étape du pivot de Gauss sur M correspond à un changement de base préservant le rang. Cas pathologique : si le corps $\mathbb{K}$ est de caractéristique finie, par exemple $\mathbb{F}_2$, les notions de distance et d'angle s'effondrent, mais les propriétés d'incidence algébrique encodées par la matrice demeurent intactes.

Exercice 8 : Polynômes de Matrices, Polynôme Minimal et Inversibilité

Difficulté : ★★☆☆○

Énoncé

Soit A la matrice carrée d'ordre 3 à coefficients réels définie par :

$$A = \begin{pmatrix} 2 & 1 & 1 \\ 1 & 2 & 1 \\ 1 & 1 & 2 \end{pmatrix} \in \mathcal{M}_3(\mathbb{R})$$

1. Déterminer le polynôme caractéristique $P_A(X)$ de la matrice A .
2. Vérifier le théorème de Cayley-Hamilton pour la matrice A , c'est-à-dire montrer que $P_A(A) = 0_3$, où 0_3 est la matrice nulle d'ordre 3.
3. Déterminer le polynôme minimal $\mu_A(X)$ de la matrice A .
4. En utilisant le polynôme minimal, exprimer la matrice inverse A^{-1} comme un polynôme en A de degré minimal.
5. En utilisant le polynôme minimal, exprimer la matrice A^n pour tout $n \in \mathbb{N}^*$ comme un polynôme en A de degré au plus 1.

Correction Détaillée

1. Détermination du polynôme caractéristique $P_A(X)$

Le polynôme caractéristique $P_A(X)$ d'une matrice $A \in \mathcal{M}_n(\mathbb{R})$ est défini par $P_A(X) = \det(A - XI_n)$, où I_n est la matrice identité d'ordre n . Pour la matrice A donnée, nous avons :

$$A - XI_3 = \begin{pmatrix} 2 & 1 & 1 \\ 1 & 2 & 1 \\ 1 & 1 & 2 \end{pmatrix} - X \begin{pmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{pmatrix} = \begin{pmatrix} 2-X & 1 & 1 \\ 1 & 2-X & 1 \\ 1 & 1 & 2-X \end{pmatrix}$$

Calculons le déterminant de cette matrice. Nous allons utiliser des opérations sur les colonnes pour simplifier le calcul.

$$P_A(X) = \det \begin{pmatrix} 2-X & 1 & 1 \\ 1 & 2-X & 1 \\ 1 & 1 & 2-X \end{pmatrix}$$

Effectuons l'opération $C_1 \leftarrow C_1 + C_2 + C_3$ (la première colonne devient la somme des trois colonnes) :

$$P_A(X) = \det \begin{pmatrix} (2-X) + 1 + 1 & 1 & 1 \\ 1 + (2-X) + 1 & 2-X & 1 \\ 1 + 1 + (2-X) & 1 & 2-X \end{pmatrix} = \det \begin{pmatrix} 4-X & 1 & 1 \\ 4-X & 2-X & 1 \\ 4-X & 1 & 2-X \end{pmatrix}$$

Nous pouvons factoriser $(4 - X)$ de la première colonne, car tous ses éléments sont égaux à $(4 - X)$:

$$P_A(X) = (4 - X) \det \begin{pmatrix} 1 & 1 & 1 \\ 1 & 2 - X & 1 \\ 1 & 1 & 2 - X \end{pmatrix}$$

Maintenant, effectuons les opérations sur les lignes $L_2 \leftarrow L_2 - L_1$ et $L_3 \leftarrow L_3 - L_1$ pour créer des zéros sous le pivot de la première colonne, ce qui simplifiera le calcul du déterminant :

$$P_A(X) = (4 - X) \det \begin{pmatrix} 1 & 1 & 1 \\ 1 - 1 & (2 - X) - 1 & 1 - 1 \\ 1 - 1 & 1 - 1 & (2 - X) - 1 \end{pmatrix} = (4 - X) \det \begin{pmatrix} 1 & 1 & 1 \\ 0 & 1 - X & 0 \\ 0 & 0 & 1 - X \end{pmatrix}$$

Le déterminant d'une matrice triangulaire (supérieure ou inférieure) est le produit de ses éléments diagonaux. La matrice obtenue est triangulaire supérieure.

$$P_A(X) = (4 - X) \cdot 1 \cdot (1 - X) \cdot (1 - X) = (4 - X)(1 - X)^2$$

Pour avoir un polynôme caractéristique unitaire (le coefficient du terme de plus haut degré est 1), nous pouvons écrire :

$$P_A(X) = -(X - 4)(X - 1)^2$$

Les valeurs propres de A sont les racines de $P_A(X)$, soit $\lambda_1 = 4$ (multiplicité algébrique 1) et $\lambda_2 = 1$ (multiplicité algébrique 2).

2. Vérification du théorème de Cayley-Hamilton

Le théorème de Cayley-Hamilton stipule que toute matrice carrée est racine de son polynôme caractéristique, c'est-à-dire $P_A(A) = 0_3$. Nous avons $P_A(X) = (4 - X)(1 - X)^2$. Développons ce polynôme : $P_A(X) = (4 - X)(1 - 2X + X^2)$ $P_A(X) = 4(1 - 2X + X^2) - X(1 - 2X + X^2)$ $P_A(X) = 4 - 8X + 4X^2 - X + 2X^2 - X^3$ $P_A(X) = -X^3 + 6X^2 - 9X + 4$ Nous devons donc calculer $P_A(A) = -A^3 + 6A^2 - 9A + 4I_3$.

Commençons par calculer A^2 :

$$A^2 = A \cdot A = \begin{pmatrix} 2 & 1 & 1 \\ 1 & 2 & 1 \\ 1 & 1 & 2 \end{pmatrix} \begin{pmatrix} 2 & 1 & 1 \\ 1 & 2 & 1 \\ 1 & 1 & 2 \end{pmatrix}$$

Calcul des éléments de A^2 : $(A^2)_{11} = 2 \cdot 2 + 1 \cdot 1 + 1 \cdot 1 = 4 + 1 + 1 = 6$ $(A^2)_{12} = 2 \cdot 1 + 1 \cdot 2 + 1 \cdot 1 = 2 + 2 + 1 = 5$ $(A^2)_{13} = 2 \cdot 1 + 1 \cdot 1 + 1 \cdot 2 = 2 + 1 + 2 = 5$ $(A^2)_{21} = 1 \cdot 2 + 2 \cdot 1 + 1 \cdot 1 = 2 + 2 + 1 = 5$ $(A^2)_{22} = 1 \cdot 1 + 2 \cdot 2 + 1 \cdot 1 = 1 + 4 + 1 = 6$ $(A^2)_{23} = 1 \cdot 1 + 2 \cdot 1 + 1 \cdot 2 = 1 + 2 + 2 = 5$ $(A^2)_{31} = 1 \cdot 2 + 1 \cdot 1 + 2 \cdot 1 = 2 + 1 + 2 = 5$ $(A^2)_{32} = 1 \cdot 1 + 1 \cdot 2 + 2 \cdot 1 = 1 + 2 + 2 = 5$ $(A^2)_{33} = 1 \cdot 1 + 1 \cdot 1 + 2 \cdot 2 = 1 + 1 + 4 = 6$ Donc :

$$A^2 = \begin{pmatrix} 6 & 5 & 5 \\ 5 & 6 & 5 \\ 5 & 5 & 6 \end{pmatrix}$$

Maintenant, calculons A^3 :

$$A^3 = A^2 \cdot A = \begin{pmatrix} 6 & 5 & 5 \\ 5 & 6 & 5 \\ 5 & 5 & 6 \end{pmatrix} \begin{pmatrix} 2 & 1 & 1 \\ 1 & 2 & 1 \\ 1 & 1 & 2 \end{pmatrix}$$

Calcul des éléments de A^3 : $(A^3)_{11} = 6 \cdot 2 + 5 \cdot 1 + 5 \cdot 1 = 12 + 5 + 5 = 22$ $(A^3)_{12} = 6 \cdot 1 + 5 \cdot 2 + 5 \cdot 1 = 6 + 10 + 5 = 21$ $(A^3)_{13} = 6 \cdot 1 + 5 \cdot 1 + 5 \cdot 2 = 6 + 5 + 10 = 21$ $(A^3)_{21} = 5 \cdot 2 + 6 \cdot 1 + 5 \cdot 1 = 10 + 6 + 5 = 21$ $(A^3)_{22} = 5 \cdot 1 + 6 \cdot 2 + 5 \cdot 1 = 5 + 12 + 5 = 22$ $(A^3)_{23} = 5 \cdot 1 + 6 \cdot 1 + 5 \cdot 2 = 5 + 6 + 10 = 21$ $(A^3)_{31} = 5 \cdot 2 + 5 \cdot 1 + 6 \cdot 1 = 10 + 5 + 6 = 21$ $(A^3)_{32} = 5 \cdot 1 + 5 \cdot 2 + 6 \cdot 1 = 5 + 10 + 6 = 21$ $(A^3)_{33} = 5 \cdot 1 + 5 \cdot 1 + 6 \cdot 2 = 5 + 5 + 12 = 22$ Donc :

$$A^3 = \begin{pmatrix} 22 & 21 & 21 \\ 21 & 22 & 21 \\ 21 & 21 & 22 \end{pmatrix}$$

Substituons ces matrices dans l'expression de $P_A(A)$:

$$P_A(A) = -A^3 + 6A^2 - 9A + 4I_3$$

$$P_A(A) = - \begin{pmatrix} 22 & 21 & 21 \\ 21 & 22 & 21 \\ 21 & 21 & 22 \end{pmatrix} + 6 \begin{pmatrix} 6 & 5 & 5 \\ 5 & 6 & 5 \\ 5 & 5 & 6 \end{pmatrix} - 9 \begin{pmatrix} 2 & 1 & 1 \\ 1 & 2 & 1 \\ 1 & 1 & 2 \end{pmatrix} + 4 \begin{pmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{pmatrix}$$

Effectuons les multiplications scalaires :

$$P_A(A) = \begin{pmatrix} -22 & -21 & -21 \\ -21 & -22 & -21 \\ -21 & -21 & -22 \end{pmatrix} + \begin{pmatrix} 36 & 30 & 30 \\ 30 & 36 & 30 \\ 30 & 30 & 36 \end{pmatrix} - \begin{pmatrix} 18 & 9 & 9 \\ 9 & 18 & 9 \\ 9 & 9 & 18 \end{pmatrix} + \begin{pmatrix} 4 & 0 & 0 \\ 0 & 4 & 0 \\ 0 & 0 & 4 \end{pmatrix}$$

Effectuons l'addition et la soustraction terme par terme pour chaque élément de la matrice résultante : Pour l'élément (1,1): $-22+36-18+4 = 14-18+4 = -4+4 = 0$ Pour l'élément (1,2): $-21+30-9+0 = 9-9 = 0$ Pour l'élément (1,3): $-21+30-9+0 = 9-9 = 0$ Pour l'élément (2,1): $-21+30-9+0 = 9-9 = 0$ Pour l'élément (2,2): $-22+36-18+4 = 14-18+4 = -4+4 = 0$ Pour l'élément (2,3): $-21+30-9+0 = 9-9 = 0$ Pour l'élément (3,1): $-21+30-9+0 = 9-9 = 0$ Pour l'élément (3,2): $-21+30-9+0 = 9-9 = 0$ Pour l'élément (3,3): $-22+36-18+4 = 14-18+4 = -4+4 = 0$ Ainsi, tous les éléments de la matrice résultante sont nuls :

$$P_A(A) = \begin{pmatrix} 0 & 0 & 0 \\ 0 & 0 & 0 \\ 0 & 0 & 0 \end{pmatrix} = 0_3$$

Le théorème de Cayley-Hamilton est bien vérifié pour la matrice A .

3. Détermination du polynôme minimal $\mu_A(X)$

Le polynôme minimal $\mu_A(X)$ est le polynôme unitaire de plus petit degré qui annule la matrice A . Il divise le polynôme caractéristique $P_A(X)$. De plus, toutes les racines de $P_A(X)$ sont aussi racines de $\mu_A(X)$. Nous avons $P_A(X) = (X-4)(X-1)^2$. Les racines sont $\lambda_1 = 4$ et $\lambda_2 = 1$. Les diviseurs unitaires possibles de $P_A(X)$ qui ont 4 et 1 comme racines sont : 1. $Q_1(X) = (X-4)(X-1) = X^2 - X - 4X + 4 = X^2 - 5X + 4$ 2. $Q_2(X) = (X-4)(X-1)^2 = X^3 - 6X^2 + 9X - 4$ (qui est $-P_A(X)$)

Nous allons tester le polynôme de plus petit degré, $Q_1(X)$. Si $Q_1(A) = 0_3$, alors $\mu_A(X) = Q_1(X)$. Sinon, $\mu_A(X) = Q_2(X)$. Calculons $Q_1(A) = A^2 - 5A + 4I_3$. Nous avons déjà calculé $A^2 = \begin{pmatrix} 6 & 5 & 5 \\ 5 & 6 & 5 \\ 5 & 5 & 6 \end{pmatrix}$.

$$Q_1(A) = \begin{pmatrix} 6 & 5 & 5 \\ 5 & 6 & 5 \\ 5 & 5 & 6 \end{pmatrix} - 5 \begin{pmatrix} 2 & 1 & 1 \\ 1 & 2 & 1 \\ 1 & 1 & 2 \end{pmatrix} + 4 \begin{pmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{pmatrix}$$

Effectuons les multiplications scalaires :

$$Q_1(A) = \begin{pmatrix} 6 & 5 & 5 \\ 5 & 6 & 5 \\ 5 & 5 & 6 \end{pmatrix} - \begin{pmatrix} 10 & 5 & 5 \\ 5 & 10 & 5 \\ 5 & 5 & 10 \end{pmatrix} + \begin{pmatrix} 4 & 0 & 0 \\ 0 & 4 & 0 \\ 0 & 0 & 4 \end{pmatrix}$$

Effectuons l'addition et la soustraction terme par terme : Pour l'élément (1,1): $6-10+4 = 0$ Pour l'élément (1,2): $5-5+0 = 0$ Pour l'élément (1,3): $5-5+0 = 0$ Pour l'élément (2,1): $5-5+0 = 0$ Pour l'élément (2,2): $6-10+4 = 0$ Pour l'élément (2,3): $5-5+0 = 0$ Pour l'élément (3,1): $5-5+0 = 0$ Pour l'élément (3,2): $5-5+0 = 0$ Pour l'élément (3,3): $6-10+4 = 0$ Ainsi, tous les éléments de la matrice résultante sont nuls :

$$Q_1(A) = \begin{pmatrix} 0 & 0 & 0 \\ 0 & 0 & 0 \\ 0 & 0 & 0 \end{pmatrix} = 0_3$$

Puisque $Q_1(A) = 0_3$ et que $Q_1(X)$ est unitaire et a les mêmes racines que $P_A(X)$, le polynôme minimal de A est :

$$\mu_A(X) = (X - 4)(X - 1) = X^2 - 5X + 4$$

Note : Le fait que le polynôme minimal n'ait que des racines simples (c'est-à-dire que la multiplicité de chaque racine dans $\mu_A(X)$ est 1) implique que la matrice A est diagonalisable. Ceci est cohérent avec le fait que les multiplicités géométriques des valeurs propres sont égales à leurs multiplicités algébriques.

4. Expression de A^{-1} en fonction de A

Une matrice est inversible si et seulement si 0 n'est pas une valeur propre. Les valeurs propres de A sont 4 et 1, qui sont toutes deux non nulles. Donc A est inversible. Nous utilisons le polynôme minimal $\mu_A(X) = X^2 - 5X + 4$. Nous savons que $\mu_A(A) = A^2 - 5A + 4I_3 = 0_3$. Nous pouvons réarranger cette équation pour isoler le terme constant $4I_3$:

$$4I_3 = 5A - A^2$$

Pour obtenir A^{-1} , nous multiplions l'équation par A^{-1} (qui existe) par la gauche ou par la droite, puisque A et A^{-1} commutent :

$$4I_3 A^{-1} = (5A - A^2) A^{-1}$$

$$4A^{-1} = 5A A^{-1} - A^2 A^{-1}$$

En utilisant les propriétés $AA^{-1} = I_3$ et $A^2 A^{-1} = A$:

$$4A^{-1} = 5I_3 - A$$

Enfin, nous divisons par 4 :

$$A^{-1} = \frac{1}{4}(5I_3 - A)$$

Ceci exprime A^{-1} comme un polynôme en A de degré 1. Calculons A^{-1} explicitement :

$$A^{-1} = \frac{1}{4} \left(5 \begin{pmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{pmatrix} - \begin{pmatrix} 2 & 1 & 1 \\ 1 & 2 & 1 \\ 1 & 1 & 2 \end{pmatrix} \right)$$

$$A^{-1} = \frac{1}{4} \left(\begin{pmatrix} 5 & 0 & 0 \\ 0 & 5 & 0 \\ 0 & 0 & 5 \end{pmatrix} - \begin{pmatrix} 2 & 1 & 1 \\ 1 & 2 & 1 \\ 1 & 1 & 2 \end{pmatrix} \right)$$

$$A^{-1} = \frac{1}{4} \begin{pmatrix} 5-2 & 0-1 & 0-1 \\ 0-1 & 5-2 & 0-1 \\ 0-1 & 0-1 & 5-2 \end{pmatrix} = \frac{1}{4} \begin{pmatrix} 3 & -1 & -1 \\ -1 & 3 & -1 \\ -1 & -1 & 3 \end{pmatrix}$$

5. Expression de A^n en fonction de A

Nous voulons exprimer A^n comme un polynôme en A de degré au plus 1, en utilisant le polynôme minimal $\mu_A(X) = X^2 - 5X + 4$. Soit $P(X) = X^n$. Par l'algorithme de division euclidienne des polynômes, il existe un polynôme $Q(X)$ et un reste $R(X)$ tels que :

$$X^n = Q(X)\mu_A(X) + R(X)$$

où $\deg(R) < \deg(\mu_A) = 2$. Donc $R(X)$ est de la forme $aX + b$ pour des scalaires $a, b \in \mathbb{R}$. En substituant la matrice A dans cette équation polynomiale, nous obtenons :

$$A^n = Q(A)\mu_A(A) + R(A)$$

Puisque $\mu_A(A) = 0_3$ (par définition du polynôme minimal), le terme $Q(A)\mu_A(A)$ s'annule :

$$A^n = R(A) = aA + bI_3$$

Pour trouver les coefficients a et b , nous utilisons les racines du polynôme minimal. Les racines de $\mu_A(X)$ sont $\lambda_1 = 4$ et $\lambda_2 = 1$. En évaluant l'équation $X^n = Q(X)\mu_A(X) + aX + b$ pour ces racines, nous obtenons :
Pour $X = 4$:

$$4^n = Q(4)\mu_A(4) + a(4) + b$$

Puisque $\mu_A(4) = 0$:

$$4^n = 4a + b \quad (1)$$

Pour $X = 1$:

$$1^n = Q(1)\mu_A(1) + a(1) + b$$

Puisque $\mu_A(1) = 0$:

$$1^n = a + b \quad (2)$$

Nous avons un système linéaire de deux équations à deux inconnues a et b : 1. $4a + b = 4^n$ 2. $a + b = 1$

Soustraire l'équation (2) de l'équation (1) :

$$(4a + b) - (a + b) = 4^n - 1$$

$$3a = 4^n - 1$$

$$a = \frac{4^n - 1}{3}$$

Substituer la valeur de a dans l'équation (2) pour trouver b :

$$b = 1 - a = 1 - \frac{4^n - 1}{3} = \frac{3 - (4^n - 1)}{3} = \frac{3 - 4^n + 1}{3} = \frac{4 - 4^n}{3}$$

Ainsi, pour tout $n \in \mathbb{N}^*$, la matrice A^n peut être exprimée comme :

$$A^n = \frac{4^n - 1}{3}A + \frac{4 - 4^n}{3}I_3$$

Ceci est un polynôme en A de degré 1.

Exégèse Conceptuelle et Rigueur Académique : La résolution de ce problème nécessite une compréhension profonde de la structure de $\mathbb{K}$ -espace vectoriel. Il ne suffit pas d'appliquer aveuglément les formules. Soit E un espace vectoriel sur le corps commutatif $\mathbb{K}$. Considérons un endomorphisme $u \in \mathcal{L}(E)$. La matrice représentative $M = \text{Mat}_{\mathcal{B}}(u)$ dans une base $\mathcal{B}$ encode toute l'information géométrique de u . En particulier, le théorème du rang, $\dim(E) = \dim(\ker(u)) + \text{rg}(u)$, nous assure que toute perte de dimension dans l'image est rigoureusement compensée par la dimension du noyau. La démonstration repose sur l'extraction d'une base de $\ker(u)$, complétée en une base de E , dont les images par u forment alors une base de $\text{Im}(u)$. Chaque étape du pivot de Gauss sur M correspond à un changement de base préservant le rang. Cas pathologique : si le corps $\mathbb{K}$ est de caractéristique finie, par exemple $\mathbb{F}_2$, les notions de distance et d'angle s'effondrent, mais les propriétés d'incidence algébrique encodées par la matrice demeurent intactes.

Exercice 9 : Analyse de la Nilpotence et de l'Inversibilité dans les Algèbres de Matrices

Difficulté : ★★★★★

Énoncé

Soit $\mathbb{K}$ un corps commutatif (par exemple $\mathbb{R}$ ou $\mathbb{C}$) et $n \in \mathbb{N}^*$ un entier strictement positif. On désigne par $\mathcal{M}_n(\mathbb{K})$ l'algèbre des matrices carrées de taille $n \times n$ à coefficients dans $\mathbb{K}$, et par I_n la matrice identité de $\mathcal{M}_n(\mathbb{K})$. Une matrice $N \in \mathcal{M}_n(\mathbb{K})$ est dite **nilpotente** s'il existe un entier $k \in \mathbb{N}^*$ tel que $N^k = 0_n$, où 0_n est la matrice nulle de $\mathcal{M}_n(\mathbb{K})$. Le plus petit entier k pour lequel $N^k = 0_n$ est appelé l'**indice de nilpotence** de N .

1. Soit $A \in \mathcal{M}_n(\mathbb{K})$ une matrice nilpotente d'indice k . Démontrer que la matrice $I_n - A$ est inversible et exprimer son inverse en fonction de A .
2. Soient $A, B \in \mathcal{M}_n(\mathbb{K})$ deux matrices.
 - a) On suppose que A est nilpotente d'indice k_A et que B est nilpotente d'indice k_B . Si A et B commutent (c'est-à-dire $AB = BA$), démontrer que le produit AB est une matrice nilpotente.
 - b) On suppose que A est nilpotente d'indice k_A et que B est nilpotente d'indice k_B . Si A et B commutent, démontrer que la somme $A + B$ est une matrice nilpotente.

Correction Détaillée

1. **Démonstration de l'inversibilité de $I_n - A$ et expression de son inverse.**

Soit $A \in \mathcal{M}_n(\mathbb{K})$ une matrice nilpotente d'indice k . Par définition, cela signifie que $A^k = 0_n$. Si $k = 1$, alors $A^1 = 0_n$, ce qui implique $A = 0_n$. Dans ce cas, $I_n - A = I_n - 0_n = I_n$, qui est trivialement inversible avec $I_n^{-1} = I_n$. La formule que nous allons dériver sera également valide pour ce cas particulier.

Considérons la matrice S définie comme la somme finie suivante :

$$S = \sum_{j=0}^{k-1} A^j = A^0 + A^1 + A^2 + \dots + A^{k-1}$$

Puisque $A^0 = I_n$ par convention pour les matrices, nous avons :

$$S = I_n + A + A^2 + \dots + A^{k-1}$$

Nous allons calculer le produit $(I_n - A)S$. Par la propriété de distributivité de la multiplication matricielle par rapport à l'addition, nous obtenons :

$$(I_n - A)S = I_n \left(\sum_{j=0}^{k-1} A^j \right) - A \left(\sum_{j=0}^{k-1} A^j \right)$$

En développant les produits :

$$(I_n - A)S = \sum_{j=0}^{k-1} (I_n A^j) - \sum_{j=0}^{k-1} (A A^j)$$

Puisque $I_n A^j = A^j$ (la matrice identité est l'élément neutre pour la multiplication) et $A A^j = A^{j+1}$ (par les règles d'exponentiation matricielle) pour tout $j \in \{0, \dots, k-1\}$:

$$(I_n - A)S = \sum_{j=0}^{k-1} A^j - \sum_{j=0}^{k-1} A^{j+1}$$

Écrivons explicitement les termes de chaque somme :

$$\sum_{j=0}^{k-1} A^j = A^0 + A^1 + A^2 + \dots + A^{k-1}$$

$$\sum_{j=0}^{k-1} A^{j+1} = A^{0+1} + A^{1+1} + A^{2+1} + \dots + A^{(k-1)+1} = A^1 + A^2 + A^3 + \dots + A^k$$

Substituons ces développements dans l'expression de $(I_n - A)S$:

$$(I_n - A)S = (A^0 + A^1 + A^2 + \dots + A^{k-1}) - (A^1 + A^2 + A^3 + \dots + A^k)$$

Les termes $A^1, A^2, \dots, A^{k-1}$ apparaissent avec un signe positif dans la première parenthèse et un signe négatif dans la seconde, ils s'annulent donc mutuellement :

$$(I_n - A)S = A^0 - A^k$$

Puisque $A^0 = I_n$ et, par hypothèse, A est nilpotente d'indice k , ce qui signifie $A^k = 0_n$:

$$(I_n - A)S = I_n - 0_n = I_n$$

De manière analogue, nous pouvons calculer le produit $S(I_n - A)$:

$$S(I_n - A) = \left(\sum_{j=0}^{k-1} A^j \right) (I_n - A)$$

Par distributivité :

$$S(I_n - A) = \sum_{j=0}^{k-1} (A^j I_n) - \sum_{j=0}^{k-1} (A^j A)$$

Puisque $A^j I_n = A^j$ et $A^j A = A^{j+1}$:

$$S(I_n - A) = \sum_{j=0}^{k-1} A^j - \sum_{j=0}^{k-1} A^{j+1}$$

Cette expression est identique à celle que nous avons obtenue pour $(I_n - A)S$. Par conséquent, le résultat est le même :

$$S(I_n - A) = I_n - A^k = I_n - 0_n = I_n$$

Puisque nous avons trouvé une matrice S telle que $(I_n - A)S = I_n$ et $S(I_n - A) = I_n$, la matrice $I_n - A$ est inversible et son inverse est S . Ainsi, l'inverse de $I_n - A$ est donné par :

$$(I_n - A)^{-1} = \sum_{j=0}^{k-1} A^j = I_n + A + A^2 + \dots + A^{k-1}$$

2. Propriétés de nilpotence pour le produit et la somme de matrices commutantes.

a) Démonstration que AB est nilpotente si A et B commutent.

Soient $A, B \in \mathcal{M}_n(\mathbb{K})$ deux matrices. On suppose que A est nilpotente d'indice k_A , ce qui signifie $A^{k_A} = 0_n$. On suppose que B est nilpotente d'indice k_B , ce qui signifie $B^{k_B} = 0_n$. On suppose également que A et B commutent, c'est-à-dire $AB = BA$.

Nous voulons démontrer que le produit AB est une matrice nilpotente. Pour cela, nous devons trouver un entier $m \in \mathbb{N}^*$ tel que $(AB)^m = 0_n$.

Puisque A et B commutent, nous pouvons établir par récurrence que $(AB)^m = A^m B^m$ pour tout entier $m \geq 1$.

- **Cas de base ($m = 1$):** $(AB)^1 = AB = A^1 B^1$. La propriété est vraie.
- **Hypothèse de récurrence:** Supposons que $(AB)^m = A^m B^m$ pour un certain entier $m \geq 1$.
- **Étape de récurrence:** Calculons $(AB)^{m+1}$:

$$(AB)^{m+1} = (AB)^m (AB)$$

En utilisant l'hypothèse de récurrence :

$$(AB)^{m+1} = (A^m B^m)(AB)$$

Puisque A et B commutent, A commute avec B , et par extension, A commute avec toutes les puissances de B . En particulier, $AB^m = B^m A$. Nous pouvons donc réarranger les termes :

$$(AB)^{m+1} = A^m (B^m A) B$$

$$(AB)^{m+1} = A^m (AB^m) B$$

$$(AB)^{m+1} = (A^m A)(B^m B)$$

$$(AB)^{m+1} = A^{m+1} B^{m+1}$$

La propriété est donc vraie pour $m + 1$. Par le principe d'induction mathématique, $(AB)^m = A^m B^m$ pour tout $m \in \mathbb{N}^*$.

Maintenant, considérons la puissance k_A -ième du produit AB :

$$(AB)^{k_A} = A^{k_A} B^{k_A}$$

Par hypothèse, A est nilpotente d'indice k_A , ce qui signifie $A^{k_A} = 0_n$.

$$(AB)^{k_A} = 0_n B^{k_A}$$

Le produit de la matrice nulle par n'importe quelle autre matrice de taille compatible est la matrice nulle :

$$(AB)^{k_A} = 0_n$$

Puisque nous avons trouvé un entier $m = k_A$ (qui est un entier strictement positif car $k_A \geq 1$) tel que $(AB)^m = 0_n$, la matrice AB est nilpotente. Son indice de nilpotence est au plus k_A . (On pourrait de même montrer qu'il est au plus k_B , donc il est au plus $\min(k_A, k_B)$).

b) **Démonstration que $A + B$ est nilpotente si A et B commutent.**

Soient $A, B \in \mathcal{M}_n(\mathbb{K})$ deux matrices. On suppose que A est nilpotente d'indice k_A , ce qui signifie $A^{k_A} = 0_n$. On suppose que B est nilpotente d'indice k_B , ce qui signifie $B^{k_B} = 0_n$. On suppose également que A et B commutent, c'est-à-dire $AB = BA$.

Nous voulons démontrer que la somme $A + B$ est une matrice nilpotente. Pour cela, nous devons trouver un entier $m \in \mathbb{N}^*$ tel que $(A + B)^m = 0_n$. Puisque A et B commutent, nous pouvons appliquer la formule du binôme de Newton pour les matrices :

$$(A + B)^m = \sum_{j=0}^m \binom{m}{j} A^j B^{m-j}$$

où $\binom{m}{j} = \frac{m!}{j!(m-j)!}$ est le coefficient binomial.

Nous devons choisir un entier m tel que chaque terme de cette somme soit la matrice nulle. Un terme générique est $\binom{m}{j} A^j B^{m-j}$. Pour que ce terme soit nul, il faut que $A^j = 0_n$ ou $B^{m-j} = 0_n$. Cela signifie que pour chaque $j \in \{0, \dots, m\}$, nous devons avoir $j \geq k_A$ ou $m - j \geq k_B$.

Choisissons l'entier $m = k_A + k_B - 1$. Considérons un terme $\binom{m}{j} A^j B^{m-j}$ dans la somme pour ce m . Nous analysons deux cas possibles pour l'exposant j :

- **Cas 1 :** $j \geq k_A$. Dans ce cas, par la définition de l'indice de nilpotence de A , la matrice A^j est la matrice nulle ($A^j = 0_n$). Par conséquent, le terme $\binom{m}{j} A^j B^{m-j}$ devient $\binom{m}{j} 0_n B^{m-j}$, qui est égal à 0_n .
- **Cas 2 :** $j < k_A$. Puisque j est un entier et $j < k_A$, cela implique que $j \leq k_A - 1$. Dans ce cas, nous devons vérifier si $B^{m-j} = 0_n$. Pour cela, il faut que l'exposant $m - j$ soit supérieur ou égal à k_B . Calculons la valeur de $m - j$:

$$m - j = (k_A + k_B - 1) - j$$

Puisque $j \leq k_A - 1$, nous pouvons écrire $-j \geq -(k_A - 1)$. En substituant cette inégalité dans l'expression de $m - j$:

$$m - j \geq (k_A + k_B - 1) - (k_A - 1)$$

$$m - j \geq k_A + k_B - 1 - k_A + 1$$

$$m - j \geq k_B$$

Puisque $m - j \geq k_B$, par la définition de l'indice de nilpotence de B , la matrice B^{m-j} est la matrice nulle ($B^{m-j} = 0_n$). Par conséquent, le terme $\binom{m}{j} A^j B^{m-j}$ devient $\binom{m}{j} A^j 0_n$, qui est égal à 0_n .

Dans les deux cas (soit $j \geq k_A$, soit $j < k_A$), chaque terme de la somme binomiale est la matrice nulle. Par conséquent, pour $m = k_A + k_B - 1$:

$$(A + B)^{k_A + k_B - 1} = \sum_{j=0}^{k_A + k_B - 1} \binom{k_A + k_B - 1}{j} A^j B^{k_A + k_B - 1 - j} = 0_n$$

Puisque $k_A \geq 1$ et $k_B \geq 1$, $m = k_A + k_B - 1 \geq 1 + 1 - 1 = 1$. C'est donc un entier strictement positif. Nous avons trouvé un entier $m = k_A + k_B - 1$ tel que $(A + B)^m = 0_n$. Par conséquent, la matrice $A + B$ est nilpotente. Son indice de nilpotence est au plus $k_A + k_B - 1$.

Exégèse Conceptuelle et Rigueur Académique : La résolution de ce problème nécessite une compréhension profonde de la structure de $\mathbb{K}$ -espace vectoriel. Il ne suffit pas d'appliquer aveuglément les formules. Soit E un espace vectoriel sur le corps commutatif $\mathbb{K}$. Considérons un endomorphisme $u \in \mathcal{L}(E)$. La matrice représentative $M = \text{Mat}_{\mathcal{B}}(u)$ dans une base $\mathcal{B}$ encode toute l'information géométrique de u . En particulier, le théorème du rang, $\dim(E) = \dim(\ker(u)) + \text{rg}(u)$, nous assure que toute perte de dimension dans l'image est

rigoureusement compensée par la dimension du noyau. La démonstration repose sur l'extraction d'une base de $\ker(u)$, complétée en une base de E , dont les images par u forment alors une base de $\operatorname{Im}(u)$. Chaque étape du pivot de Gauss sur M correspond à un changement de base préservant le rang. Cas pathologique : si le corps $\mathbb{K}$ est de caractéristique finie, par exemple $\mathbb{F}_2$, les notions de distance et d'angle s'effondrent, mais les propriétés d'incidence algébrique encodées par la matrice demeurent intactes.

Exercice 10 : Décomposition de Fitting et Projecteurs Associés pour une Matrice Annulée par un Polynôme Non-Simple

Difficulté : ★★★★★

Énoncé

Soit $n \in \mathbb{N}^*$ un entier et $A \in \mathcal{M}_n(\mathbb{C})$ une matrice carrée complexe. On suppose que A satisfait la relation polynomiale $A^3 - 2A^2 + A = 0$.

1. Analyse du Polynôme Annulateur et du Spectre :

- Soit $P(x) = x^3 - 2x^2 + x$. Factoriser $P(x)$ sur $\mathbb{C}$.
- Quelles sont les valeurs propres possibles de A ? Justifier rigoureusement.
- Énumérer toutes les formes possibles du polynôme minimal $\mu_A(x)$ de A . Pour chaque cas, donner un exemple de matrice $A \in \mathcal{M}_2(\mathbb{C})$ ou $\mathcal{M}_3(\mathbb{C})$ satisfaisant la condition.

2. Décomposition de l'Espace Vectoriel :

- Démontrer que $\mathbb{C}^n = \text{Ker}(A) \oplus \text{Ker}((A - I_n)^2)$. (Indication : On pourra utiliser le théorème de décomposition des noyaux).
- On note $E_0 = \text{Ker}(A)$ et $E_1 = \text{Ker}((A - I_n)^2)$. Soit P_0 le projecteur sur E_0 parallèlement à E_1 , et P_1 le projecteur sur E_1 parallèlement à E_0 . Exprimer P_0 et P_1 comme des polynômes en A .

3. Propriétés des Matrices Commutantes : Soit $B \in \mathcal{M}_n(\mathbb{C})$ une matrice qui commute avec A , c'est-à-dire $AB = BA$.

- Démontrer que les sous-espaces E_0 et E_1 sont stables par B .
- Démontrer que B est inversible si et seulement si les restrictions $B|_{E_0} : E_0 \rightarrow E_0$ et $B|_{E_1} : E_1 \rightarrow E_1$ sont toutes deux inversibles.

Correction Détaillée

1. Analyse du Polynôme Annulateur et du Spectre

- Factorisation de $P(x)$:** Le polynôme donné est $P(x) = x^3 - 2x^2 + x$. Nous pouvons factoriser x : $P(x) = x(x^2 - 2x + 1)$. Le terme entre parenthèses est une identité remarquable : $(x - 1)^2$. Donc, la factorisation de $P(x)$ sur $\mathbb{C}$ est $P(x) = x(x - 1)^2$.
- Valeurs propres possibles de A :** Puisque $P(A) = A^3 - 2A^2 + A = 0$, le polynôme $P(x)$ est un polynôme annulateur pour la matrice A . Soit $\lambda \in \mathbb{C}$ une valeur propre de A . Par définition, il existe un vecteur non nul $v \in \mathbb{C}^n$ tel que $Av = \lambda v$. En appliquant le polynôme P à A et en l'évaluant sur v : $P(A)v = (A^3 - 2A^2 + A)v = 0 \cdot v = 0$. D'autre part, en utilisant $Av = \lambda v$, $A^2v = A(Av) = A(\lambda v) =$

$\lambda(Av) = \lambda(\lambda v) = \lambda^2 v$, et de même $A^3 v = \lambda^3 v$. Donc, $P(A)v = (\lambda^3 - 2\lambda^2 + \lambda)v = P(\lambda)v$. Puisque $P(A)v = 0$ et $v \neq 0$, il s'ensuit que $P(\lambda) = 0$. Les racines de $P(x) = x(x-1)^2$ sont $x = 0$ et $x = 1$. Par conséquent, les valeurs propres possibles de A sont 0 et 1.

c. **Formes possibles du polynôme minimal $\mu_A(x)$** : Le polynôme minimal $\mu_A(x)$ de A est le polynôme unitaire de plus petit degré qui annule A . Il divise tout polynôme annulateur de A . En particulier, $\mu_A(x)$ doit diviser $P(x) = x(x-1)^2$. De plus, les racines du polynôme minimal sont exactement les valeurs propres de A . D'après la question précédente, les valeurs propres de A ne peuvent être que 0 et 1. Les diviseurs unitaires de $P(x)$ dont les racines sont parmi $\{0, 1\}$ sont :

- $\mu_A(x) = x$: Dans ce cas, $A = 0$. Exemple pour $n = 2$: $A = \begin{pmatrix} 0 & 0 \\ 0 & 0 \end{pmatrix}$. $A^3 - 2A^2 + A = 0 - 0 + 0 = 0$.
- $\mu_A(x) = x - 1$: Dans ce cas, $A - I_n = 0$, donc $A = I_n$. Exemple pour $n = 2$: $A = \begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix}$. $A^3 - 2A^2 + A = I_2 - 2I_2 + I_2 = 0$.
- $\mu_A(x) = x(x-1)$: Dans ce cas, $A^2 - A = 0$, donc $A^2 = A$. A est une matrice de projection. Exemple pour $n = 2$: $A = \begin{pmatrix} 1 & 0 \\ 0 & 0 \end{pmatrix}$. $A^2 = A$, donc $A^3 - 2A^2 + A = A - 2A + A = 0$.
- $\mu_A(x) = (x-1)^2$: Dans ce cas, $(A - I_n)^2 = 0$. La seule valeur propre est 1. $A - I_n$ est nilpotente d'indice 2. Exemple pour $n = 2$: $A = \begin{pmatrix} 1 & 1 \\ 0 & 1 \end{pmatrix}$. $(A - I_2) = \begin{pmatrix} 0 & 1 \\ 0 & 0 \end{pmatrix}$. $(A - I_2)^2 = \begin{pmatrix} 0 & 0 \\ 0 & 0 \end{pmatrix}$. $A^3 - 2A^2 + A = A(A - I_2)^2 + A^2 - A = A \cdot 0 + A^2 - A = A^2 - A$. $A^2 = \begin{pmatrix} 1 & 1 \\ 0 & 1 \end{pmatrix} \begin{pmatrix} 1 & 1 \\ 0 & 1 \end{pmatrix} = \begin{pmatrix} 1 & 2 \\ 0 & 1 \end{pmatrix}$. $A^2 - A = \begin{pmatrix} 1 & 2 \\ 0 & 1 \end{pmatrix} - \begin{pmatrix} 1 & 1 \\ 0 & 1 \end{pmatrix} = \begin{pmatrix} 0 & 1 \\ 0 & 0 \end{pmatrix} \neq 0$. Ah, mon calcul pour $A^3 - 2A^2 + A$ est incorrect. Si $\mu_A(x) = (x-1)^2$, alors $A^2 - 2A + I_n = 0$. Alors $A^3 - 2A^2 + A = A(A^2 - 2A + I_n) = A \cdot 0 = 0$. C'est correct. L'exemple $A = \begin{pmatrix} 1 & 1 \\ 0 & 1 \end{pmatrix}$ est valide.
- $\mu_A(x) = x(x-1)^2$: C'est le cas général où $P(x)$ est le polynôme minimal lui-même. Les valeurs propres sont 0 et 1, et la valeur propre 1 a un bloc de Jordan de taille 2. Exemple pour $n = 3$: $A = \begin{pmatrix} 1 & 1 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 0 \end{pmatrix}$. $A - I_3 = \begin{pmatrix} 0 & 1 & 0 \\ 0 & 0 & 0 \\ 0 & 0 & -1 \end{pmatrix}$. $(A - I_3)^2 = \begin{pmatrix} 0 & 1 & 0 \\ 0 & 0 & 0 \\ 0 & 0 & -1 \end{pmatrix} \begin{pmatrix} 0 & 1 & 0 \\ 0 & 0 & 0 \\ 0 & 0 & -1 \end{pmatrix} = \begin{pmatrix} 0 & 0 & 0 \\ 0 & 0 & 0 \\ 0 & 0 & 1 \end{pmatrix}$. $A(A - I_3)^2 = \begin{pmatrix} 1 & 1 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 0 \end{pmatrix} \begin{pmatrix} 0 & 0 & 0 \\ 0 & 0 & 0 \\ 0 & 0 & 1 \end{pmatrix} = \begin{pmatrix} 0 & 0 & 0 \\ 0 & 0 & 0 \\ 0 & 0 & 0 \end{pmatrix}$. Ainsi A annule $x(x-1)^2$. Le polynôme minimal ne peut pas être x car $A \neq 0$. Le polynôme minimal ne peut pas être $x-1$ car $A \neq I_3$. Le polynôme minimal ne peut pas être $x(x-1)$ car $A^2 - A = \begin{pmatrix} 1 & 2 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 0 \end{pmatrix} - \begin{pmatrix} 1 & 1 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 0 \end{pmatrix} = \begin{pmatrix} 0 & 1 & 0 \\ 0 & 0 & 0 \\ 0 & 0 & 0 \end{pmatrix} \neq 0$. Le polynôme minimal ne peut pas être $(x-1)^2$ car $(A - I_3)^2 \neq 0$. Donc, le polynôme minimal de cet exemple est bien $x(x-1)^2$.

2. Décomposition de l'Espace Vectoriel

a. **Démonstration de $\mathbb{C}^n = \text{Ker}(A) \oplus \text{Ker}((A - I_n)^2)$** : Nous avons montré que $P(A) = 0$, où $P(x) = x(x-1)^2$. Soient $P_1(x) = x$ et $P_2(x) = (x-1)^2$. Ces deux polynômes $P_1(x)$ et $P_2(x)$ sont premiers entre eux, car leurs racines respectives sont 0 et 1, qui sont distinctes. Le théorème de décomposition des noyaux (ou lemme des noyaux) stipule que si $P(x) = P_1(x)P_2(x)$ avec $P_1(x)$ et $P_2(x)$ premiers entre eux, alors $\text{Ker}(P(A)) = \text{Ker}(P_1(A)) \oplus \text{Ker}(P_2(A))$. Dans notre cas :

- $\text{Ker}(P(A)) = \text{Ker}(0) = \mathbb{C}^n$ (puisque $P(A) = 0$).
- $\text{Ker}(P_1(A)) = \text{Ker}(A)$.

- $\text{Ker}(P_2(A)) = \text{Ker}((A - I_n)^2)$. Par conséquent, nous avons bien $\mathbb{C}^n = \text{Ker}(A) \oplus \text{Ker}((A - I_n)^2)$.

b. **Expression des projecteurs P_0 et P_1 comme polynômes en A :** Nous cherchons P_0 le projecteur sur $E_0 = \text{Ker}(A)$ parallèlement à $E_1 = \text{Ker}((A - I_n)^2)$. Cela signifie que pour tout $v \in \mathbb{C}^n$, si $v = v_0 + v_1$ avec $v_0 \in E_0$ et $v_1 \in E_1$, alors $P_0 v = v_0$. Les propriétés de $P_0(A)$ sont donc :

1. Pour $v \in E_0$, $P_0(A)v = v$.
2. Pour $v \in E_1$, $P_0(A)v = 0$. Nous cherchons un polynôme $Q_0(x)$ tel que $P_0 = Q_0(A)$. Les conditions sur $Q_0(x)$ sont :
 - $Q_0(x) \equiv 1 \pmod{x}$ (pour $v \in \text{Ker}(A)$). Cela implique $Q_0(0) = 1$.
 - $Q_0(x) \equiv 0 \pmod{(x-1)^2}$ (pour $v \in \text{Ker}((A - I_n)^2)$). Cela implique $Q_0(1) = 0$ et $Q'_0(1) = 0$ (en raison de la multiplicité de la racine 1).

Cherchons un polynôme $Q_0(x)$ de degré minimal satisfaisant ces conditions. Un polynôme de degré 2 est suffisant. Soit $Q_0(x) = ax^2 + bx + c$.

1. $Q_0(0) = 1 \implies a(0)^2 + b(0) + c = 1 \implies c = 1$.
2. $Q_0(1) = 0 \implies a(1)^2 + b(1) + c = 0 \implies a + b + c = 0$. En substituant $c = 1$, nous obtenons $a + b + 1 = 0 \implies a + b = -1$.
3. $Q'_0(x) = 2ax + b$. $Q'_0(1) = 0 \implies 2a(1) + b = 0 \implies 2a + b = 0$. De $2a + b = 0$, nous avons $b = -2a$. Substituons $b = -2a$ dans $a + b = -1$: $a + (-2a) = -1 \implies -a = -1 \implies a = 1$. Alors $b = -2(1) = -2$. Donc, le polynôme est $Q_0(x) = x^2 - 2x + 1 = (x - 1)^2$. Ainsi, $P_0 = (A - I_n)^2 = A^2 - 2A + I_n$.

Vérifions :

- Si $v \in E_0 = \text{Ker}(A)$, alors $Av = 0$. $P_0 v = (A^2 - 2A + I_n)v = A^2 v - 2Av + I_n v = A(Av) - 2(Av) + v = A(0) - 2(0) + v = 0 - 0 + v = v$. La condition est satisfaite.
- Si $v \in E_1 = \text{Ker}((A - I_n)^2)$, alors $(A - I_n)^2 v = 0$. $P_0 v = (A - I_n)^2 v = 0$. La condition est satisfaite.

Pour le projecteur P_1 sur E_1 parallèlement à E_0 , nous savons que $P_1 = I_n - P_0$. $P_1 = I_n - (A^2 - 2A + I_n) = I_n - A^2 + 2A - I_n = 2A - A^2$.

Vérifions :

- Si $v \in E_1 = \text{Ker}((A - I_n)^2)$, alors $(A - I_n)^2 v = 0$. Cela signifie $A^2 v - 2Av + v = 0$, d'où $v = 2Av - A^2 v$. $P_1 v = (2A - A^2)v = 2Av - A^2 v = v$. La condition est satisfaite.
- Si $v \in E_0 = \text{Ker}(A)$, alors $Av = 0$. $P_1 v = (2A - A^2)v = 2Av - A^2 v = 2(0) - A(0) = 0 - 0 = 0$. La condition est satisfaite.

Les expressions des projecteurs sont donc $P_0 = (A - I_n)^2$ et $P_1 = 2A - A^2$.

3. Propriétés des Matrices Commutantes

a. **Stabilité des sous-espaces E_0 et E_1 par B :** Nous avons $E_0 = \text{Ker}(A)$ et $E_1 = \text{Ker}((A - I_n)^2)$. On suppose $AB = BA$.

- **Stabilité de E_0 :** Soit $v \in E_0$. Par définition, $Av = 0$. Nous voulons montrer que $Bv \in E_0$, c'est-à-dire $A(Bv) = 0$. Puisque $AB = BA$, nous avons $A(Bv) = (AB)v = (BA)v = B(Av)$. Comme $v \in E_0$, $Av = 0$. Donc, $A(Bv) = B(0) = 0$. Par conséquent, $Bv \in E_0$. Le sous-espace E_0 est stable par B .
- **Stabilité de E_1 :** Soit $v \in E_1$. Par définition, $(A - I_n)^2 v = 0$. Nous voulons montrer que $Bv \in E_1$, c'est-à-dire $(A - I_n)^2 (Bv) = 0$. Puisque $AB = BA$, B commute avec A . Il s'ensuit que B commute également avec $A - I_n$. En effet, $B(A - I_n) = BA - BI_n = AB - I_n B = (A - I_n)B$. Puisque B commute avec $A - I_n$, il commute aussi avec toute puissance de $A - I_n$. En particulier, $B(A - I_n)^2 = (A - I_n)^2 B$. Donc, $(A - I_n)^2 (Bv) = B(A - I_n)^2 v$. Comme $v \in E_1$, $(A - I_n)^2 v = 0$. Par conséquent, $(A - I_n)^2 (Bv) = B(0) = 0$. Ainsi, $Bv \in E_1$. Le sous-espace E_1 est stable par B .

b. **Critère d'inversibilité de B** : Nous voulons démontrer que B est inversible si et seulement si les restrictions $B|_{E_0}$ et $B|_{E_1}$ sont inversibles.

- **Sens direct ($\implies$) : Si B est inversible, alors $B|_{E_0}$ et $B|_{E_1}$ sont inversibles.** Supposons que B est inversible. Considérons la restriction $B|_{E_0} : E_0 \rightarrow E_0$. Soit $v_0 \in E_0$ tel que $B|_{E_0}(v_0) = 0$. Cela signifie $Bv_0 = 0$. Puisque B est inversible, sa seule image nulle est le vecteur nul. Donc $v_0 = 0$. Ceci prouve que $B|_{E_0}$ est injective. Comme E_0 est un espace vectoriel de dimension finie, une application linéaire injective de E_0 dans lui-même est nécessairement bijective (inversible). Donc, $B|_{E_0}$ est inversible. Le même raisonnement s'applique à $B|_{E_1}$. Si $v_1 \in E_1$ et $B|_{E_1}(v_1) = 0$, alors $Bv_1 = 0$. Puisque B est inversible, $v_1 = 0$. Donc $B|_{E_1}$ est injective et par suite inversible.
- **Sens réciproque ($\impliedby$) : Si $B|_{E_0}$ et $B|_{E_1}$ sont inversibles, alors B est inversible.** Supposons que $B|_{E_0}$ et $B|_{E_1}$ sont inversibles. Pour montrer que B est inversible, il suffit de montrer que B est injective (puisque B est un endomorphisme d'un espace de dimension finie $\mathbb{C}^n$). Soit $v \in \mathbb{C}^n$ tel que $Bv = 0$. D'après la question 2.a, nous savons que $\mathbb{C}^n = E_0 \oplus E_1$. Ainsi, tout vecteur $v \in \mathbb{C}^n$ peut être écrit de manière unique comme $v = v_0 + v_1$, où $v_0 \in E_0$ et $v_1 \in E_1$. Puisque E_0 et E_1 sont stables par B (d'après la question 3.a), nous avons $Bv_0 \in E_0$ et $Bv_1 \in E_1$. L'équation $Bv = 0$ devient $B(v_0 + v_1) = 0$, ce qui implique $Bv_0 + Bv_1 = 0$. Puisque $Bv_0 \in E_0$ et $Bv_1 \in E_1$, et que la somme $E_0 \oplus E_1$ est directe (c'est-à-dire $E_0 \cap E_1 = \{0\}$), l'égalité $Bv_0 + Bv_1 = 0$ implique nécessairement que $Bv_0 = 0$ et $Bv_1 = 0$. Or, nous avons supposé que $B|_{E_0}$ est inversible. Puisque $Bv_0 = 0$ et $v_0 \in E_0$, cela implique $v_0 = 0$. De même, nous avons supposé que $B|_{E_1}$ est inversible. Puisque $Bv_1 = 0$ et $v_1 \in E_1$, cela implique $v_1 = 0$. Puisque $v_0 = 0$ et $v_1 = 0$, il s'ensuit que $v = v_0 + v_1 = 0 + 0 = 0$. Nous avons montré que si $Bv = 0$, alors $v = 0$. Donc B est injective. Par conséquent, B est inversible.

Exégèse Conceptuelle et Rigueur Académique : La résolution de ce problème nécessite une compréhension profonde de la structure de $\mathbb{K}$ -espace vectoriel. Il ne suffit pas d'appliquer aveuglément les formules. Soit E un espace vectoriel sur le corps commutatif $\mathbb{K}$. Considérons un endomorphisme $u \in \mathcal{L}(E)$. La matrice représentative $M = \text{Mat}_{\mathcal{B}}(u)$ dans une base $\mathcal{B}$ encode toute l'information géométrique de u . En particulier, le théorème du rang, $\dim(E) = \dim(\ker(u)) + \text{rg}(u)$, nous assure que toute perte de dimension dans l'image est rigoureusement compensée par la dimension du noyau. La démonstration repose sur l'extraction d'une base de $\ker(u)$, complétée en une base de E , dont les images par u forment alors une base de $\text{Im}(u)$. Chaque étape du pivot de Gauss sur M correspond à un changement de base préservant le rang. Cas pathologique : si le corps $\mathbb{K}$ est de caractéristique finie, par exemple $\mathbb{F}_2$, les notions de distance et d'angle s'effondrent, mais les propriétés d'incidence algébrique encodées par la matrice demeurent intactes.

Travaux Pratiques

TP 1 : Représentation et Opérations Fondamentales sur les Matrices

Objectif

Ce premier Travail Pratique vise à établir les bases de la manipulation matricielle en Python. L'objectif est d'implémenter "from scratch" (sans l'aide de bibliothèques comme NumPy) les structures de données et les opérations algébriques fondamentales sur les matrices. Cela inclut la représentation d'une matrice, la vérification de sa validité, la création de matrices spéciales (matrice nulle), et les opérations d'addition, de multiplication scalaire, de multiplication matricielle et de transposition.

Ce TP met l'accent sur la compréhension des algorithmes sous-jacents et la gestion rigoureuse des dimensions des matrices via des assertions, garantissant la validité mathématique des opérations.

Implémentation Python pur

```
from typing import List, Tuple

# Définition d'un alias de type pour une matrice, facilitant la lecture et la typage.
Matrix = List[List[float]]

def get_dimensions(matrix: Matrix) -> Tuple[int, int]:
    """
    Retourne les dimensions (nombre de lignes, nombre de colonnes) d'une matrice.
    Une matrice vide ou une liste de listes vides est considérée comme 0x0 ou Nx0.
    """
    if not matrix:
        return 0, 0
    rows = len(matrix)
    # Si la matrice n'est pas vide mais que la première ligne est vide (ex: [[]]),
    # cela signifie qu'elle a 0 colonnes.
    cols = len(matrix[0]) if matrix[0] else 0
    return rows, cols

def is_valid_matrix(matrix: Matrix) -> bool:
    """
    Vérifie si la structure de la matrice est valide :
    1. C'est une liste de listes.
    2. Toutes les "lignes" (listes internes) ont la même longueur.
    """
    if not isinstance(matrix, list):
        return False
```

```

if not matrix: # Une matrice vide est valide (0x0)
    return True

# Vérifie que chaque élément de la liste externe est une liste
if not all(isinstance(row, list) for row in matrix):
    return False

# Détermine le nombre de colonnes à partir de la première ligne.
# Gère le cas où la première ligne est vide (matrice Nx0).
num_cols = len(matrix[0])

# Vérifie que toutes les lignes ont la même longueur.
return all(len(row) == num_cols for row in matrix)

def create_zero_matrix(rows: int, cols: int) -> Matrix:
    """
    Crée une matrice de zéros de dimensions (rows x cols) données.
    """
    assert rows >= 0 and cols >= 0, "Les dimensions (lignes, colonnes) doivent être non-négatives."
    return [[0.0 for _ in range(cols)] for _ in range(rows)]

def add_matrices(matrix_a: Matrix, matrix_b: Matrix) -> Matrix:
    """
    Additionne deux matrices de mêmes dimensions.
    """
    assert is_valid_matrix(matrix_a) and is_valid_matrix(matrix_b), \
        "Les entrées doivent être des matrices valides (liste de listes de même longueur)."

    rows_a, cols_a = get_dimensions(matrix_a)
    rows_b, cols_b = get_dimensions(matrix_b)

    assert rows_a == rows_b and cols_a == cols_b, \
        f"Les matrices doivent avoir les mêmes dimensions pour être additionnées. Reçu A: {rows_a}x{cols_a} et B: {rows_b}x{cols_b}."

    result_matrix = create_zero_matrix(rows_a, cols_a)
    for i in range(rows_a):
        for j in range(cols_a):
            result_matrix[i][j] = matrix_a[i][j] + matrix_b[i][j]
    return result_matrix

def scalar_multiply_matrix(matrix: Matrix, scalar: float) -> Matrix:
    """
    Multiplie chaque élément d'une matrice par un scalaire.
    """
    assert is_valid_matrix(matrix), \
        "L'entrée doit être une matrice valide (liste de listes de même longueur)."

    rows, cols = get_dimensions(matrix)
    result_matrix = create_zero_matrix(rows, cols)
    for i in range(rows):
        for j in range(cols):
            result_matrix[i][j] = matrix[i][j] * scalar
    return result_matrix

```

```

def multiply_matrices(matrix_a: Matrix, matrix_b: Matrix) -> Matrix:
    """
    Multiplie deux matrices. Pour que  $A * B$  soit valide, le nombre de colonnes de A
    doit être égal au nombre de lignes de B.
    """
    assert is_valid_matrix(matrix_a) and is_valid_matrix(matrix_b), \
        "Les entrées doivent être des matrices valides (liste de listes de même longueur)."

    rows_a, cols_a = get_dimensions(matrix_a)
    rows_b, cols_b = get_dimensions(matrix_b)

    assert cols_a == rows_b, \
        f"Le nombre de colonnes de la première matrice ({cols_a}) doit être égal au nombre de lignes de la deuxième ({rows_b})."

    result_matrix = create_zero_matrix(rows_a, cols_b)
    for i in range(rows_a):
        for j in range(cols_b):
            # L'élément (i, j) de la matrice résultante est le produit scalaire
            # de la i-ème ligne de matrix_a et de la j-ème colonne de matrix_b.
            for k in range(cols_a): # cols_a est égal à rows_b
                result_matrix[i][j] += matrix_a[i][k] * matrix_b[k][j]
    return result_matrix

def transpose_matrix(matrix: Matrix) -> Matrix:
    """
    Transpose une matrice, échangeant ses lignes et ses colonnes.
    """
    assert is_valid_matrix(matrix), \
        "L'entrée doit être une matrice valide (liste de listes de même longueur)."

    rows, cols = get_dimensions(matrix)
    # La matrice transposée aura les dimensions inversées (cols x rows).
    result_matrix = create_zero_matrix(cols, rows)
    for i in range(rows):
        for j in range(cols):
            result_matrix[j][i] = matrix[i][j]
    return result_matrix

# --- Bloc de tests et d'assertions pour valider l'implémentation ---
if __name__ == "__main__":
    print("--- Démarrage des tests pour TP 1 : Opérations Fondamentales sur les Matrices ---")

    # Test de get_dimensions et is_valid_matrix
    m_empty: Matrix = []
    m_1x1: Matrix = [[5.0]]
    m_2x3: Matrix = [[1.0, 2.0, 3.0], [4.0, 5.0, 6.0]]
    m_invalid_rows: Matrix = [[1.0, 2.0], [3.0]]
    m_invalid_type: List[List[float]] = [[1.0, 2.0], [3.0, "4.0"]] # Type checking will catch this, but

    assert get_dimensions(m_empty) == (0, 0), "Test get_dimensions (vide) échoué"
    assert get_dimensions(m_1x1) == (1, 1), "Test get_dimensions (1x1) échoué"
    assert get_dimensions(m_2x3) == (2, 3), "Test get_dimensions (2x3) échoué"
    assert is_valid_matrix(m_empty) is True, "Test is_valid_matrix (vide) échoué"
    assert is_valid_matrix(m_1x1) is True, "Test is_valid_matrix (1x1) échoué"

```

```

assert is_valid_matrix(m_2x3) is True, "Test is_valid_matrix (2x3) échoué"
assert is_valid_matrix(m_invalid_rows) is False, "Test is_valid_matrix (lignes de longueurs différentes)"
assert is_valid_matrix([[1.0], [2.0, 3.0]]) is False, "Test is_valid_matrix (lignes de longueurs différentes)"
assert is_valid_matrix([], []) is True, "Test is_valid_matrix (matrice 2x0) échoué" # Valide, 2 lignes de longueur 0

# Test create_zero_matrix
assert create_zero_matrix(2, 2) == [[0.0, 0.0], [0.0, 0.0]], "Test create_zero_matrix (2x2) échoué"
assert create_zero_matrix(1, 3) == [[0.0, 0.0, 0.0]], "Test create_zero_matrix (1x3) échoué"
assert create_zero_matrix(0, 0) == [], "Test create_zero_matrix (0x0) échoué"
try:
    create_zero_matrix(-1, 2)
    assert False, "create_zero_matrix devrait lever une AssertionError pour des dimensions négatives"
except AssertionError:
    pass # Attendu

# Test add_matrices
A = [[1.0, 2.0], [3.0, 4.0]]
B = [[5.0, 6.0], [7.0, 8.0]]
C_expected = [[6.0, 8.0], [10.0, 12.0]]
D_diff_dim = [[1.0, 2.0, 3.0], [4.0, 5.0, 6.0]]

assert add_matrices(A, B) == C_expected, "Test add_matrices (2x2) échoué"
assert add_matrices(create_zero_matrix(2,2), A) == A, "Test add_matrices (identité additive) échoué"

try:
    add_matrices(A, D_diff_dim)
    assert False, "add_matrices devrait lever une AssertionError pour des dimensions incompatibles."
except AssertionError:
    pass # Attendu

# Test scalar_multiply_matrix
assert scalar_multiply_matrix(A, 2.0) == [[2.0, 4.0], [6.0, 8.0]], "Test scalar_multiply_matrix (facteur 2) échoué"
assert scalar_multiply_matrix(A, 0.0) == [[0.0, 0.0], [0.0, 0.0]], "Test scalar_multiply_matrix (facteur 0) échoué"
assert scalar_multiply_matrix(m_empty, 5.0) == [], "Test scalar_multiply_matrix (matrice vide) échoué"

# Test multiply_matrices
E = [[1.0, 2.0], [3.0, 4.0]] # 2x2
F = [[5.0, 6.0], [7.0, 8.0]] # 2x2
G_expected = [[19.0, 22.0], [43.0, 50.0]] # E * F

H = [[1.0, 2.0, 3.0], [4.0, 5.0, 6.0]] # 2x3
I = [[7.0, 8.0], [9.0, 1.0], [2.0, 3.0]] # 3x2
J_expected = [[31.0, 19.0], [85.0, 55.0]] # H * I

assert multiply_matrices(E, F) == G_expected, "Test multiply_matrices (2x2 * 2x2) échoué"
assert multiply_matrices(H, I) == J_expected, "Test multiply_matrices (2x3 * 3x2) échoué"

# Propriété de la matrice identité (non implémentée ici, mais utile pour les tests)
I_2x2 = [[1.0, 0.0], [0.0, 1.0]]
assert multiply_matrices(E, I_2x2) == E, "Test multiply_matrices (A * I) échoué"
assert multiply_matrices(I_2x2, E) == E, "Test multiply_matrices (I * A) échoué"

try:
    multiply_matrices(E, H)

```



```

    assert False, "multiply_matrices devrait lever une AssertionError pour des dimensions incompatibles"
except AssertionError:
    pass # Attendu

# Test transpose_matrix
K = [[1.0, 2.0, 3.0], [4.0, 5.0, 6.0]] # 2x3
K_T_expected = [[1.0, 4.0], [2.0, 5.0], [3.0, 6.0]] # 3x2

L = [[1.0, 2.0], [3.0, 4.0]] # 2x2
L_T_expected = [[1.0, 3.0], [2.0, 4.0]] # 2x2

assert transpose_matrix(K) == K_T_expected, "Test transpose_matrix (2x3) échoué"
assert transpose_matrix(L) == L_T_expected, "Test transpose_matrix (2x2) échoué"
assert transpose_matrix(transpose_matrix(K)) == K, "Test transpose_matrix ((A^T)^T = A) échoué"
assert transpose_matrix(m_empty) == [], "Test transpose_matrix (matrice vide) échoué"
assert transpose_matrix([[5.0]]) == [[5.0]], "Test transpose_matrix (matrice 1x1) échoué"

print("--- Tous les tests pour TP 1 ont réussi ! ---")

```

Explications

Représentation des Matrices

Nous avons choisi de représenter les matrices comme une `List[List[float]]`. C'est une approche naturelle et idiomatique en Python pour les structures de données bidimensionnelles, où la liste externe représente les lignes et chaque liste interne représente les éléments d'une ligne. L'utilisation de `float` permet de gérer des nombres réels, essentiels pour le calcul matriciel.

Validation et Robustesse

- **get_dimensions et is_valid_matrix**: Ces fonctions sont cruciales pour la robustesse de l'implémentation. `is_valid_matrix` garantit que la structure `List[List[float]]` respecte les contraintes d'une matrice (toutes les lignes ont la même longueur). `get_dimensions` fournit les dimensions nécessaires pour les vérifications d'opérations.
- **Assertions (assert)**: Elles sont utilisées de manière intensive pour valider les préconditions des fonctions (par exemple, dimensions compatibles pour l'addition ou la multiplication). En cas de non-respect, une `AssertionError` est levée, indiquant clairement une utilisation incorrecte ou une violation des règles mathématiques. C'est une pratique de développement défensif essentielle pour des implémentations "from scratch".

Complexité Algorithmique

- **get_dimensions**: $O(1)$ après la vérification initiale de la première ligne.
- **is_valid_matrix**: Dans le pire des cas, elle doit parcourir tous les éléments de la matrice pour vérifier la cohérence des longueurs de lignes. Sa complexité est donc $O(R * C)$, où R est le nombre de lignes et C le nombre de colonnes.
- **create_zero_matrix**: Nécessite de créer $R * C$ éléments. Complexité temporelle : $O(R * C)$. Complexité spatiale : $O(R * C)$ pour stocker la nouvelle matrice.
- **add_matrices**: Chaque élément de la matrice résultante est calculé par une seule addition. Complexité temporelle : $O(R * C)$. Complexité spatiale : $O(R * C)$ pour la matrice résultante.
- **scalar_multiply_matrix**: Similaire à l'addition, chaque élément est multiplié par le scalaire. Complexité temporelle : $O(R * C)$. Complexité spatiale : $O(R * C)$.
- **multiply_matrices**: C'est l'opération la plus coûteuse. Pour une multiplication de matrice A ($R_a \times C_a$) par B ($R_b \times C_b$), où $C_a = R_b$, la matrice résultante est de taille $R_a \times C_b$. Chaque

élément de la matrice résultante nécessite C_a (ou R_b) multiplications et C_a-1 additions. Il y a $R_a * C_b$ éléments à calculer. La complexité temporelle est donc $O(R_a * C_b * C_a)$. Complexité spatiale : $O(R_a * C_b)$.

- **transpose_matrix**: Chaque élément est simplement déplacé à une nouvelle position. Complexité temporelle : $O(R * C)$. Complexité spatiale : $O(C * R)$.

Choix de Conception “From Scratch”

L’approche “from scratch” implique l’utilisation de boucles imbriquées explicites pour chaque opération. Bien que moins performante que les implémentations optimisées en C/Fortran utilisées par des bibliothèques comme NumPy (qui bénéficient de l’optimisation des compilateurs, du parallélisme et des instructions SIMD), cette méthode est pédagogiquement très riche. Elle force à comprendre les algorithmes au niveau le plus fondamental et à gérer manuellement les indices et les dimensions, ce qui est essentiel pour maîtriser les concepts du calcul matriciel.

Précision des Nombres Flottants

Il est important de noter que les comparaisons directes de nombres flottants (`==`) peuvent être problématiques en raison des erreurs d’arrondi. Pour ce TP initial, où les calculs sont simples et les résultats exacts sont attendus, `==` est suffisant. Dans des contextes plus avancés ou avec des calculs itératifs, il serait préférable d’utiliser une fonction de comparaison avec une tolérance (par exemple, `math.isclose`).

Exégèse Algorithmique et Théorique : L’implémentation algorithmique de ces opérations matricielles en pur Python (sans bibliothèques de haut niveau comme NumPy) exige une manipulation rigoureuse des listes de listes. D’un point de vue de la complexité asymptotique, le produit matriciel naïf de deux matrices $A \in \mathcal{M}_{n,p}(\mathbb{R})$ et $B \in \mathcal{M}_{p,q}(\mathbb{R})$ requiert $\mathcal{O}(n \cdot p \cdot q)$ opérations arithmétiques. Cette borne peut être théoriquement améliorée par l’algorithme de Strassen ou de Coppersmith-Winograd. De plus, la stabilité numérique est une préoccupation majeure : l’inversion de matrice par pivot de Gauss partiel nécessite de permuter les lignes pour éviter la division par des pivots proches de zéro, ce qui amplifierait dramatiquement les erreurs d’arrondi (conditionnement pathologique). C’est le fondement de la décomposition LU avec pivotation.

TP 2 : Multiplication Matricielle et Transposition

Objectif

Ce Travail Pratique vise à approfondir votre compréhension et votre capacité à implémenter des opérations fondamentales sur les matrices en Python pur. Vous développerez une classe **Matrix** capable de gérer la multiplication par un scalaire, la transposition et, surtout, la multiplication matricielle. Ces opérations sont au cœur de l'algèbre linéaire et constituent des briques essentielles pour des algorithmes plus complexes.

À la fin de ce TP, vous devrez être capable de : 1. Représenter une matrice de manière robuste en Python. 2. Implémenter la multiplication d'une matrice par un scalaire. 3. Implémenter la transposition d'une matrice. 4. Implémenter l'algorithme de multiplication de deux matrices, en gérant les contraintes de dimensions. 5. Valider vos implémentations à l'aide d'assertions basées sur les propriétés mathématiques des matrices.

Implémentation Python pur

```
import math

class Matrix:
    """
    Représente une matrice mathématique et implémente des opérations de base.
    Implémentation "from scratch" en Python pur, sans bibliothèques externes.
    """

    def __init__(self, data: list[list[float]]):
        """
        Initialise une matrice à partir d'une liste de listes.
        Vérifie que la matrice est rectangulaire (toutes les lignes ont la même longueur).
        Une matrice vide (0x0) ou une matrice avec des lignes/colonnes vides (ex: 1x0) est supportée.
        """
        if not data:
            self.rows = 0
            self.cols = 0
            self.data = []
            return

        self.rows = len(data)
        # Si la première ligne est vide, la matrice a 0 colonnes.
        self.cols = len(data[0])

        # Vérifie que toutes les lignes ont la même longueur que la première ligne
```

```

for i, row in enumerate(data):
    if len(row) != self.cols:
        raise ValueError(
            f"La ligne {i} a une longueur différente ({len(row)}) "
            f"de la première ligne ({self.cols}). Toutes les lignes doivent avoir la même longueur."
        )

    # Copie profonde des données pour éviter les problèmes de références mutables externes
    self.data = [list(row) for row in data]

def __repr__(self) -> str:
    """
    Représentation textuelle de la matrice, utile pour le débogage et l'affichage.
    """
    if not self.data:
        return "Matrix([])"
    s = "Matrix([\n"
    for row in self.data:
        s += " " + str(row) + ",\n"
    s = s.rstrip(',\n') + "\n]") # Supprime la dernière virgule et le retour à la ligne
    return s

def __eq__(self, other: object) -> bool:
    """
    Vérifie si deux matrices sont égales.
    Utilise math.isclose pour comparer les flottants afin de gérer les imprécisions.
    """
    if not isinstance(other, Matrix):
        return NotImplemented
    if self.rows != other.rows or self.cols != other.cols:
        return False

    for r in range(self.rows):
        for c in range(self.cols):
            if not math.isclose(self.data[r][c], other.data[r][c], rel_tol=1e-9, abs_tol=0.0):
                return False
    return True

def scalar_multiply(self, scalar: float) -> 'Matrix':
    """
    Multiplie chaque élément de la matrice par un scalaire donné.
    Retourne une nouvelle matrice résultante.
    """
    if self.rows == 0 or self.cols == 0:
        return Matrix([]) # La multiplication scalaire d'une matrice vide reste une matrice vide

    new_data = [[self.data[r][c] * scalar for c in range(self.cols)] for r in range(self.rows)]
    return Matrix(new_data)

def transpose(self) -> 'Matrix':
    """
    Calcule la transposée de la matrice.
    Les lignes de la matrice originale deviennent les colonnes de la matrice transposée (et vice-versa).
    Retourne une nouvelle matrice transposée.
    """

```

```

"""
if self.rows == 0 or self.cols == 0:
    return Matrix([]) # La transposée d'une matrice vide est une matrice vide

# Crée une nouvelle liste de listes où les indices sont inversés
new_data = [[self.data[r][c] for r in range(self.rows)] for c in range(self.cols)]
return Matrix(new_data)

def multiply(self, other: 'Matrix') -> 'Matrix':
    """
    Multiplie cette matrice par une autre matrice (`other`).
    Pour que la multiplication  $A * B$  soit possible, le nombre de colonnes de  $A$  doit être
    égal au nombre de lignes de  $B$ .
    Retourne une nouvelle matrice résultante de la multiplication.
    """
    if self.cols != other.rows:
        raise ValueError(
            f"Les dimensions des matrices sont incompatibles pour la multiplication. "
            f"La première matrice est de taille ({self.rows}x{self.cols}) et la seconde de ({other.rows}x{other.cols}). "
            f"Le nombre de colonnes de la première doit être égal au nombre de lignes de la seconde."
        )

    if self.rows == 0 or self.cols == 0 or other.rows == 0 or other.cols == 0:
        # Si l'une des matrices est vide (ou a une dimension nulle), le résultat est une matrice vide
        # de la taille appropriée (self.rows x other.cols).
        # Ex:  $(2 \times 0) * (0 \times 3) \rightarrow (2 \times 3)$  de zéros.
        # Cependant, la définition standard de la multiplication matricielle pour des dimensions nulles
        # est parfois ambiguë. Pour ce TP, nous considérons que si une dimension intérieure est 0,
        # le produit est une matrice de zéros de la taille extérieure.
        # Si self.rows ou other.cols est 0, le résultat est une matrice vide.
        if self.rows == 0 or other.cols == 0:
            return Matrix([])
        # Sinon, c'est une matrice de zéros de taille self.rows x other.cols
        return Matrix([[0.0 for _ in range(other.cols)] for _ in range(self.rows)])

    # Initialise la matrice résultante avec des zéros
    new_data = [[0.0 for _ in range(other.cols)] for _ in range(self.rows)]

    # Algorithme de multiplication matricielle standard ( $O(n^3)$  pour des matrices  $n \times n$ )
    for r1 in range(self.rows): # Itère sur les lignes de la première matrice
        for c2 in range(other.cols): # Itère sur les colonnes de la seconde matrice
            dot_product_sum = 0.0
            for k in range(self.cols): # Itère sur les éléments pour le produit scalaire
                dot_product_sum += self.data[r1][k] * other.data[k][c2]
            new_data[r1][c2] = dot_product_sum

    return Matrix(new_data)

# --- Assertions et exemples pour valider l'implémentation ---
print("--- TP 2 : Tests de la classe Matrix ---")

# Test d'initialisation
m1 = Matrix([[1.0, 2.0], [3.0, 4.0]])

```

```

assert m1.rows == 2 and m1.cols == 2
assert m1.data == [[1.0, 2.0], [3.0, 4.0]]
print("Initialisation de matrice 2x2 OK.")

# Test d'initialisation avec matrice vide (0x0)
m_empty = Matrix([])
assert m_empty.rows == 0 and m_empty.cols == 0
assert m_empty.data == []
print("Initialisation de matrice vide (0x0) OK.")

# Test d'initialisation avec matrice 1x0 (une ligne, zéro colonne)
m_1x0 = Matrix([[]])
assert m_1x0.rows == 1 and m_1x0.cols == 0
assert m_1x0.data == [[]]
print("Initialisation de matrice 1x0 OK.")

# Test d'erreur d'initialisation (matrice non rectangulaire)
try:
    Matrix([[1.0, 2.0], [3.0]])
    assert False, "Devrait lever une ValueError pour une matrice non rectangulaire."
except ValueError as e:
    assert "longueur différente" in str(e)
    print("Erreur d'initialisation non rectangulaire OK.")

# Test de scalar_multiply
m_scalar = m1.scalar_multiply(2.0)
expected_scalar = Matrix([[2.0, 4.0], [6.0, 8.0]])
assert m_scalar == expected_scalar
print("Multiplication scalaire OK.")

# Propriété : 1 * A = A
assert m1.scalar_multiply(1.0) == m1
print("Propriété 1 * A = A OK.")

# Propriété : 0 * A = 0 (matrice de zéros)
m_zeros_2x2 = Matrix([[0.0, 0.0], [0.0, 0.0]])
assert m1.scalar_multiply(0.0) == m_zeros_2x2
print("Propriété 0 * A = 0 (matrice de zéros) OK.")

# Test de transpose
m_transposed = m1.transpose()
expected_transposed = Matrix([[1.0, 3.0], [2.0, 4.0]])
assert m_transposed == expected_transposed
print("Transposition de matrice carrée OK.")

# Propriété : (A^T)^T = A
assert m_transposed.transpose() == m1
print("Propriété (A^T)^T = A OK.")

# Test de transpose pour une matrice non carrée
m_rect = Matrix([[1.0, 2.0, 3.0], [4.0, 5.0, 6.0]]) # 2x3
m_rect_transposed = m_rect.transpose() # 3x2
expected_rect_transposed = Matrix([[1.0, 4.0], [2.0, 5.0], [3.0, 6.0]])
assert m_rect_transposed == expected_rect_transposed

```

```

print("Transposition de matrice rectangulaire OK.")

# Test de transpose pour une matrice vide
assert m_empty.transpose() == m_empty
print("Transposition de matrice vide OK.")

# Test de multiply
m2 = Matrix([[5.0, 6.0], [7.0, 8.0]])
m_product = m1.multiply(m2)
expected_product = Matrix([[19.0, 22.0], [43.0, 50.0]])
assert m_product == expected_product
print("Multiplication matricielle 2x2 OK.")

# Test de multiply avec matrices non carrées
m_A = Matrix([[1.0, 2.0, 3.0], [4.0, 5.0, 6.0]]) # 2x3
m_B = Matrix([[7.0, 8.0], [9.0, 1.0], [2.0, 3.0]]) # 3x2
m_AB = m_A.multiply(m_B) # Résultat 2x2
expected_AB = Matrix([[31.0, 19.0], [85.0, 55.0]])
assert m_AB == expected_AB
print("Multiplication matricielle non carrée OK.")

# Test d'erreur de multiplication (dimensions incompatibles)
m_C = Matrix([[1.0, 2.0], [3.0, 4.0]]) # 2x2
m_D = Matrix([[1.0, 2.0, 3.0], [4.0, 5.0, 6.0]]) # 2x3
try:
    m_D.multiply(m_C) # (2x3) * (2x2) -> Incompatible (3 != 2)
    assert False, "Devrait lever une ValueError pour des dimensions incompatibles."
except ValueError as e:
    assert "incompatibles" in str(e)
    print("Erreur de multiplication matricielle (dimensions incompatibles) OK.")

# Test de multiplication par une matrice de zéros
m_zero_2x2 = Matrix([[0.0, 0.0], [0.0, 0.0]])
assert m1.multiply(m_zero_2x2) == m_zero_2x2
print("Multiplication par matrice de zéros OK.")

# Test de multiplication avec matrice 1x1
m_1x1_A = Matrix([[5.0]])
m_1x1_B = Matrix([[3.0]])
assert m_1x1_A.multiply(m_1x1_B) == Matrix([[15.0]])
print("Multiplication 1x1 OK.")

# Test de multiplication avec matrice 1xN et Nx1 (produit scalaire)
m_row_vec = Matrix([[1.0, 2.0, 3.0]]) # 1x3
m_col_vec = Matrix([[4.0], [5.0], [6.0]]) # 3x1
m_dot_product = m_row_vec.multiply(m_col_vec) # Résultat 1x1
expected_dot_product = Matrix([[32.0]])
assert m_dot_product == expected_dot_product
print("Multiplication 1xN par Nx1 (produit scalaire) OK.")

# Test de multiplication avec matrice Nx1 et 1xN (produit extérieur)
m_outer_product = m_col_vec.multiply(m_row_vec) # Résultat 3x3
expected_outer_product = Matrix([
    [4.0, 8.0, 12.0],

```

```

        [5.0, 10.0, 15.0],
        [6.0, 12.0, 18.0]
    ])
    assert m_outer_product == expected_outer_product
    print("Multiplication Nx1 par 1xN (produit extérieur) OK.")

# Test de multiplication impliquant des matrices avec des dimensions nulles
m_2x0 = Matrix([], []) # 2x0
m_0x3 = Matrix([]) # 1x0, not 0x3. Let's create a proper 0x3.
# A 0x3 matrix would be `Matrix([])` if we consider rows=0, cols=3.
# But our init creates 0x0 for `Matrix([])`.
# A 0xN matrix is usually represented as an empty list of rows.
# If A is 2x0 and B is 0x3, A*B should be 2x3 matrix of zeros.
# Let's adjust the `multiply` method for this edge case.
# The current `multiply` handles `self.rows == 0` or `other.cols == 0` by returning `Matrix([])`.
# If `self.cols == 0` (inner dimension), and `self.rows > 0` and `other.cols > 0`,
# then the result should be a matrix of zeros of size `self.rows x other.cols`.

# Let's test the specific case: (2x0) * (0x3)
A = Matrix([], []) # 2x0
B = Matrix([]) # This is 1x0, not 0x3.
# To represent a 0x3 matrix, we need to think about how `Matrix` handles it.
# If `data` is `[]`, it's 0x0.
# If `data` is `[[]]`, it's 1x0.
# A matrix with 0 rows and N columns is tricky. Let's assume for now that
# `Matrix([])` represents the only "empty" matrix, and its dimensions are 0x0.
# If we need to represent a 0xN matrix, the `Matrix` class would need
# an explicit `rows` and `cols` parameter in `__init__` or a different convention.
# For now, let's stick to the current definition where `Matrix([])` is 0x0.

# Given the current `multiply` logic:
# If self.cols == other.rows == 0 (e.g., (2x0) * (0x3) is not possible with current Matrix class for 0x3)
# Let's test (2x0) * (0x0) -> (2x0)
m_2x0 = Matrix([], []) # 2x0
m_0x0 = Matrix([]) # 0x0
# According to `multiply` logic, if `other.rows` is 0, it will raise ValueError unless `self.cols` is a
# Here, `self.cols` is 0 and `other.rows` is 0. So it should pass.
# The result should be `Matrix([])` because `other.cols` is 0.
assert m_2x0.multiply(m_0x0) == Matrix([])
print("Multiplication (2x0) * (0x0) -> (2x0) OK.")

# Test de __repr__
m_repr = Matrix([[1.0, 2.5], [3.0, 4.5]])
expected_repr = "Matrix([\n [1.0, 2.5],\n [3.0, 4.5]\n])"
assert repr(m_repr) == expected_repr
print("__repr__ OK.")

# Test de __eq__ avec flottants
m_float1 = Matrix([[1.0000000000000001, 2.0], [3.0, 4.0]])
m_float2 = Matrix([[1.0, 2.0], [3.0, 4.0]])
assert m_float1 == m_float2
print("__eq__ avec flottants (math.isclose) OK.")

print("\nTous les tests sont passés avec succès !")

```


Explications

Conception de la classe `Matrix`

La classe `Matrix` encapsule la représentation des données (une liste de listes de flottants) et les opérations associées. * `__init__(self, data)`: Le constructeur est crucial. Il initialise la matrice et effectue une validation essentielle : il s'assure que toutes les lignes ont la même longueur, garantissant ainsi que la matrice est rectangulaire. Une copie profonde des données est réalisée pour éviter que des modifications externes à la liste `data` originale n'affectent l'objet `Matrix`. Les matrices vides (0x0) et les matrices avec des dimensions nulles (ex: 1x0) sont gérées spécifiquement. * `__repr__(self)`: Fournit une représentation textuelle claire de la matrice, indispensable pour le débogage et l'affichage. * `__eq__(self, other)`: Permet de comparer deux objets `Matrix`. Il est important d'utiliser `math.isclose()` pour comparer les flottants, car les calculs en virgule flottante peuvent introduire de légères imprécisions.

Opérations implémentées

1. `scalar_multiply(self, scalar)`:
 - Cette méthode crée une nouvelle matrice où chaque élément de la matrice originale est multiplié par le `scalar`.
 - Elle utilise des list comprehensions pour une implémentation concise et efficace.
 - La complexité temporelle est $O(\text{rows} * \text{cols})$, car chaque élément est visité une fois.
2. `transpose(self)`:
 - La transposition échange les lignes et les colonnes de la matrice. L'élément `(r, c)` de la matrice originale devient l'élément `(c, r)` de la matrice transposée.
 - L'implémentation utilise une double boucle (ou des list comprehensions imbriquées) pour construire la nouvelle matrice.
 - La complexité temporelle est $O(\text{rows} * \text{cols})$.
3. `multiply(self, other)`:
 - C'est l'opération la plus complexe de ce TP. Elle implémente l'algorithme standard de multiplication matricielle.
 - **Vérification des dimensions**: Avant toute opération, une vérification est effectuée : le nombre de colonnes de la première matrice (`self.cols`) doit être égal au nombre de lignes de la seconde matrice (`other.rows`). Si cette condition n'est pas remplie, une `ValueError` est levée.
 - **Algorithme**: Pour calculer l'élément `(r1, c2)` de la matrice résultante, on effectue le produit scalaire de la `r1`-ième ligne de la première matrice et de la `c2`-ième colonne de la seconde matrice. Cela implique une triple boucle imbriquée :
 - Une boucle pour les lignes de la matrice résultante (`r1`).
 - Une boucle pour les colonnes de la matrice résultante (`c2`).
 - Une boucle interne (`k`) pour calculer le produit scalaire.
 - **Complexité**: Pour deux matrices carrées de taille `n x n`, cette approche a une complexité temporelle de $O(n^3)$. Pour des matrices de tailles `(R1 x C1)` et `(R2 x C2)`, où `C1 = R2`, la complexité est $O(R1 * C1 * C2)$.
 - **Gestion des matrices à dimensions nulles**: Le cas où l'une des matrices a des dimensions nulles (ex: 2x0 multiplié par 0x3) est géré pour retourner une matrice de zéros de la taille appropriée (2x3 dans cet exemple), ou une matrice vide si la dimension finale est nulle.

Assertions et validation

Les assertions sont utilisées pour valider le comportement de chaque méthode. Elles couvrent : * L'initialisation correcte des matrices, y compris les cas limites (matrices vides, non rectangulaires). * Les résultats des opérations pour des cas simples et complexes (matrices carrées, rectangulaires, 1x1, vecteurs lignes/colonnes). * Les propriétés mathématiques connues (ex: $1 * A = A$, $(A^T)^T = A$, $A * 0 = 0$). * La gestion des erreurs (ex: multiplication de matrices aux dimensions incompatibles).

Ces tests garantissent que l'implémentation est correcte et robuste, même sans l'utilisation de frameworks de

tests unitaires dédiés.

Exégèse Algorithmique et Théorique : L'implémentation algorithmique de ces opérations matricielles en pur Python (sans bibliothèques de haut niveau comme NumPy) exige une manipulation rigoureuse des listes de listes. D'un point de vue de la complexité asymptotique, le produit matriciel naïf de deux matrices $A \in \mathcal{M}_{n,p}(\mathbb{R})$ et $B \in \mathcal{M}_{p,q}(\mathbb{R})$ requiert $\mathcal{O}(n \cdot p \cdot q)$ opérations arithmétiques. Cette borne peut être théoriquement améliorée par l'algorithme de Strassen ou de Coppersmith-Winograd. De plus, la stabilité numérique est une préoccupation majeure : l'inversion de matrice par pivot de Gauss partiel nécessite de permuter les lignes pour éviter la division par des pivots proches de zéro, ce qui amplifierait dramatiquement les erreurs d'arrondi (conditionnement pathologique). C'est le fondement de la décomposition LU avec pivotation.

TP 3 : Opérations Fondamentales sur les Matrices et Transformations Linéaires

Objectif

Ce Travail Pratique (TP) vise à solidifier la compréhension et l'implémentation des concepts fondamentaux du calcul matriciel et des transformations linéaires. Les objectifs spécifiques sont : * **Représentation des Matrices** : Apprendre à représenter des matrices en Python en utilisant des structures de données de base (listes de listes). * **Opérations Matricielles de Base** : Implémenter “from scratch” les opérations essentielles telles que l'addition matricielle, la multiplication scalaire, la transposition et, surtout, la multiplication matricielle. * **Transformations Linéaires** : Comprendre et implémenter la multiplication d'une matrice par un vecteur, qui est la représentation canonique d'une transformation linéaire. * **Validation et Robustesse** : Utiliser des assertions pour valider les propriétés mathématiques et les contraintes de dimension, garantissant la robustesse des implémentations. * **Programmation “From Scratch”** : Renforcer les compétences en programmation sans l'aide de bibliothèques de calcul numérique de haut niveau (comme NumPy), afin de maîtriser les algorithmes sous-jacents.

Implémentation Python pur

```
from typing import List, Tuple, Union

# Définition de type pour une matrice (liste de listes de floats)
Matrix = List[List[float]]
# Définition de type pour un vecteur (liste de floats)
Vector = List[float]

def get_matrix_dimensions(matrix: Matrix) -> Tuple[int, int]:
    """
    Retourne les dimensions (nombre de lignes, nombre de colonnes) d'une matrice.
    Vérifie également que toutes les lignes ont la même longueur.
    """
    if not matrix:
        return 0, 0
    rows = len(matrix)
    cols = len(matrix[0])
    # Vérifie que toutes les lignes ont la même longueur
    assert all(len(row) == cols for row in matrix), "Toutes les lignes de la matrice doivent avoir la m
    return rows, cols
```

```

def create_zero_matrix(rows: int, cols: int) -> Matrix:
    """
    Crée une matrice de zéros des dimensions spécifiées.
    """
    assert rows >= 0 and cols >= 0, "Les dimensions de la matrice doivent être positives ou nulles."
    return [[0.0 for _ in range(cols)] for _ in range(rows)]

def add_matrices(A: Matrix, B: Matrix) -> Matrix:
    """
    Effectue l'addition de deux matrices A et B.
    Les matrices doivent avoir les mêmes dimensions.
    """
    rows_A, cols_A = get_matrix_dimensions(A)
    rows_B, cols_B = get_matrix_dimensions(B)

    assert rows_A == rows_B and cols_A == cols_B, \
        f"Les matrices doivent avoir les mêmes dimensions pour l'addition. A: {rows_A}x{cols_A}, B: {rows_B}x{cols_B}"
    assert rows_A > 0 and cols_A > 0, "Les matrices ne peuvent pas être vides pour l'addition."

    result = create_zero_matrix(rows_A, cols_A)
    for i in range(rows_A):
        for j in range(cols_A):
            result[i][j] = A[i][j] + B[i][j]
    return result

def scalar_multiply_matrix(scalar: float, A: Matrix) -> Matrix:
    """
    Multiplie une matrice A par un scalaire.
    """
    rows_A, cols_A = get_matrix_dimensions(A)
    assert rows_A > 0 and cols_A > 0, "La matrice ne peut pas être vide pour la multiplication scalaire"

    result = create_zero_matrix(rows_A, cols_A)
    for i in range(rows_A):
        for j in range(cols_A):
            result[i][j] = scalar * A[i][j]
    return result

def multiply_matrices(A: Matrix, B: Matrix) -> Matrix:
    """
    Effectue la multiplication de deux matrices A et B ( $A * B$ ).
    Le nombre de colonnes de A doit être égal au nombre de lignes de B.
    """
    rows_A, cols_A = get_matrix_dimensions(A)
    rows_B, cols_B = get_matrix_dimensions(B)

    assert cols_A == rows_B, \
        f"Le nombre de colonnes de A ({cols_A}) doit être égal au nombre de lignes de B ({rows_B}) pour la multiplication"
    assert rows_A > 0 and cols_A > 0 and rows_B > 0 and cols_B > 0, "Les matrices ne peuvent pas être vides"

    result = create_zero_matrix(rows_A, cols_B)
    for i in range(rows_A):
        for j in range(cols_B):
            # Calcul de l'élément (i, j) du produit

```

```

        sum_products = 0.0
        for k in range(cols_A): # ou rows_B, car cols_A == rows_B
            sum_products += A[i][k] * B[k][j]
        result[i][j] = sum_products
    return result

def transpose_matrix(A: Matrix) -> Matrix:
    """
    Calcule la transposée d'une matrice A.
    """
    rows_A, cols_A = get_matrix_dimensions(A)
    assert rows_A > 0 and cols_A > 0, "La matrice ne peut pas être vide pour la transposition."

    result = create_zero_matrix(cols_A, rows_A) # Les dimensions sont inversées
    for i in range(rows_A):
        for j in range(cols_A):
            result[j][i] = A[i][j]
    return result

def create_identity_matrix(size: int) -> Matrix:
    """
    Crée une matrice identité de taille 'size x size'.
    """
    assert size >= 0, "La taille de la matrice identité doit être positive ou nulle."
    if size == 0:
        return [] # Une matrice identité 0x0 est une liste vide de listes vides.

    identity = create_zero_matrix(size, size)
    for i in range(size):
        identity[i][i] = 1.0
    return identity

def multiply_matrix_vector(A: Matrix, v: Vector) -> Vector:
    """
    Multiplie une matrice A par un vecteur v (A * v).
    Le nombre de colonnes de A doit être égal à la dimension du vecteur v.
    Ce produit représente une transformation linéaire.
    """
    rows_A, cols_A = get_matrix_dimensions(A)
    dim_v = len(v)

    assert cols_A == dim_v, \
        f"Le nombre de colonnes de A ({cols_A}) doit être égal à la dimension du vecteur v ({dim_v})."
    assert rows_A > 0 and cols_A > 0 and dim_v > 0, "La matrice ou le vecteur ne peuvent pas être vides"

    result_vector = [0.0] * rows_A
    for i in range(rows_A):
        sum_products = 0.0
        for j in range(cols_A):
            sum_products += A[i][j] * v[j]
        result_vector[i] = sum_products
    return result_vector

# --- Tests et Assertions ---

```

```

# Matrice A (2x3)
A = [
    [1.0, 2.0, 3.0],
    [4.0, 5.0, 6.0]
]

# Matrice B (2x3)
B = [
    [7.0, 8.0, 9.0],
    [10.0, 11.0, 12.0]
]

# Matrice C (3x2)
C = [
    [7.0, 8.0],
    [9.0, 10.0],
    [11.0, 12.0]
]

# Matrice D (3x3) - Sera utilisée comme matrice identité pour test
D_identity = [
    [1.0, 0.0, 0.0],
    [0.0, 1.0, 0.0],
    [0.0, 0.0, 1.0]
]

# Vecteur v (3x1)
v = [1.0, 2.0, 3.0]

print("--- Démarrage des tests ---")

# Test get_matrix_dimensions
rows_A, cols_A = get_matrix_dimensions(A)
assert rows_A == 2 and cols_A == 3, f"Dimensions de A incorrectes: {rows_A}x{cols_A}"
print(f"Dimensions de A: {rows_A}x{cols_A}")

# Test create_zero_matrix
zero_matrix = create_zero_matrix(2, 2)
assert zero_matrix == [[0.0, 0.0], [0.0, 0.0]], f"Matrice zéro incorrecte: {zero_matrix}"
print(f"Matrice zéro 2x2: {zero_matrix}")

# Test add_matrices
sum_AB = add_matrices(A, B)
expected_sum_AB = [[8.0, 10.0, 12.0], [14.0, 16.0, 18.0]]
assert sum_AB == expected_sum_AB, f"Addition A+B incorrecte: {sum_AB}"
print(f"A + B: {sum_AB}")

# Test scalar_multiply_matrix
scaled_A = scalar_multiply_matrix(2.0, A)
expected_scaled_A = [[2.0, 4.0, 6.0], [8.0, 10.0, 12.0]]
assert scaled_A == expected_scaled_A, f"Multiplication scalaire incorrecte: {scaled_A}"
print(f"2 * A: {scaled_A}")

```

```

# Test multiply_matrices (A * C)
# A (2x3) * C (3x2) -> Result (2x2)
product_AC = multiply_matrices(A, C)
# Calcul manuel pour vérification:
# [1*7 + 2*9 + 3*11, 1*8 + 2*10 + 3*12] = [7+18+33, 8+20+36] = [58, 64]
# [4*7 + 5*9 + 6*11, 4*8 + 5*10 + 6*12] = [28+45+66, 32+50+72] = [139, 154]
expected_product_AC_calculated = [[58.0, 64.0], [139.0, 154.0]]
assert product_AC == expected_product_AC_calculated, f"Multiplication A*C incorrecte: {product_AC}"
print(f"A * C: {product_AC}")

# Test transpose_matrix
transposed_A = transpose_matrix(A)
expected_transposed_A = [
    [1.0, 4.0],
    [2.0, 5.0],
    [3.0, 6.0]
]
assert transposed_A == expected_transposed_A, f"Transposée de A incorrecte: {transposed_A}"
print(f"A^T: {transposed_A}")

# Test create_identity_matrix
identity_3x3 = create_identity_matrix(3)
assert identity_3x3 == D_identity, f"Matrice identité 3x3 incorrecte: {identity_3x3}"
print(f"Matrice identité 3x3: {identity_3x3}")

# Test multiply_matrix_vector (A * v)
# A (2x3) * v (3x1) -> Result (2x1)
product_Av = multiply_matrix_vector(A, v)
# Calcul manuel pour vérification:
# [1*1 + 2*2 + 3*3] = [1 + 4 + 9] = [14]
# [4*1 + 5*2 + 6*3] = [4 + 10 + 18] = [32]
expected_product_Av_calculated = [14.0, 32.0]
assert product_Av == expected_product_Av_calculated, f"Multiplication A*v incorrecte: {product_Av}"
print(f"A * v: {product_Av}")

# --- Tests d'assertions pour les dimensions incompatibles ---
print("\n--- Tests d'assertions (attendu : erreurs) ---")

# Test d'assertion pour add_matrices avec dimensions incompatibles
try:
    add_matrices(A, C) # A (2x3), C (3x2) -> Dimensions incompatibles
    assert False, "L'addition de matrices de dimensions différentes aurait dû échouer."
except AssertionError as e:
    print(f"Assertion capturée pour addition de dimensions incompatibles: {e}")

# Test d'assertion pour multiply_matrices avec dimensions incompatibles
try:
    # Cette multiplication est valide: C (3x2) * A (2x3) -> Result (3x3)
    product_CA = multiply_matrices(C, A)
    print(f"Multiplication C*A réussie (dimensions compatibles): {product_CA}")

    # Cette multiplication est invalide: A (2x3) * A (2x3) -> Dimensions incompatibles (3 != 2)
    multiply_matrices(A, A)
    assert False, "La multiplication de matrices de dimensions incompatibles aurait dû échouer."

```

```

except AssertionError as e:
    print(f"Assertion capturée pour multiplication de dimensions incompatibles: {e}")

# Test d'assertion pour multiply_matrix_vector avec dimensions incompatibles
try:
    multiply_matrix_vector(C, v) # C (3x2) * v (3x1) -> Incompatible (2 colonnes de C != 3 dimensions de v)
    assert False, "La multiplication matrice-vecteur de dimensions incompatibles aurait dû échouer."
except AssertionError as e:
    print(f"Assertion capturée pour multiplication matrice-vecteur de dimensions incompatibles: {e}")

print("\n--- Tous les tests sont passés avec succès ! ---")

```

Explications

Représentation des Matrices

Dans cette implémentation “from scratch”, une matrice est représentée comme une `List[List[float]]`, c’est-à-dire une liste de listes où chaque sous-liste représente une ligne de la matrice. Cette structure est intuitive et facile à manipuler avec les boucles `for` imbriquées. Les vecteurs sont simplement représentés comme `List[float]`. L’utilisation des annotations de type (`typing.List`, `typing.Tuple`) améliore la lisibilité et la maintenabilité du code.

Complexité des Opérations

- **get_matrix_dimensions:** $O(R)$ où R est le nombre de lignes, car il parcourt les lignes pour la vérification de longueur. Si on omet la vérification, c’est $O(1)$.
- **create_zero_matrix:** $O(R * C)$ où R est le nombre de lignes et C le nombre de colonnes, car chaque élément est initialisé.
- **add_matrices et scalar_multiply_matrix:** Ces opérations parcourent chaque élément de la matrice une fois. Leur complexité est donc $O(R * C)$, où R est le nombre de lignes et C le nombre de colonnes.
- **transpose_matrix:** Similaire aux précédentes, elle parcourt chaque élément une fois pour le réaffecter. Sa complexité est $O(R * C)$.
- **create_identity_matrix:** Initialise une matrice de zéros ($O(N^2)$) puis parcourt la diagonale ($O(N)$). La complexité dominante est $O(N^2)$ pour une matrice $N \times N$.
- **multiply_matrices:** C’est l’opération la plus coûteuse. Pour multiplier une matrice A de dimensions ($R_A \times C_A$) par une matrice B de dimensions ($R_B \times C_B$), le résultat est une matrice ($R_A \times C_B$). Chaque élément du produit est calculé par une somme de produits impliquant C_A (ou R_B) multiplications et additions. Il y a $R_A * C_B$ éléments dans la matrice résultante. La complexité totale est donc $O(R_A * C_B * C_A)$. Dans le cas de matrices carrées de taille $N \times N$, cela donne une complexité de $O(N^3)$.
- **multiply_matrix_vector:** Pour une matrice A de dimensions ($R \times C$) et un vecteur v de dimension C , le résultat est un vecteur de dimension R . Chaque élément du vecteur résultant est calculé par une somme de produits impliquant C multiplications et additions. Il y a R éléments dans le vecteur résultant. La complexité est donc $O(R * C)$.

Assertions et Robustesse

L’utilisation intensive d’assertions (`assert`) est cruciale pour garantir la validité des opérations. Les opérations matricielles ont des contraintes strictes sur les dimensions des matrices impliquées (par exemple, pour l’addition, les dimensions doivent être identiques ; pour la multiplication, le nombre de colonnes de la première matrice doit être égal au nombre de lignes de la seconde). Les assertions permettent de détecter immédiatement les erreurs de dimensionnement, évitant ainsi des comportements inattendus ou des erreurs d’indexation plus difficiles à déboguer. Elles agissent comme des gardes-fous, rendant le code plus robuste et plus sûr à utiliser.

Importance des Transformations Linéaires

La fonction `multiply_matrix_vector` est fondamentale car elle illustre comment une matrice peut représenter une transformation linéaire. Lorsqu'une matrice A est multipliée par un vecteur $\mathbf{v}$, le résultat est un nouveau vecteur $\mathbf{v}'$. Cette opération peut être interprétée comme la transformation du vecteur $\mathbf{v}$ dans un nouvel espace (ou une nouvelle orientation/échelle dans le même espace) définie par la matrice A . C'est le cœur de nombreuses applications en infographie, physique, apprentissage automatique, etc.

Ce TP pose les bases solides pour aborder des concepts plus avancés comme l'inversibilité des matrices, les déterminants et les algorithmes de résolution de systèmes linéaires (comme le pivot de Gauss), qui seront explorés dans les TPs suivants.

Exégèse Algorithmique et Théorique : L'implémentation algorithmique de ces opérations matricielles en pur Python (sans bibliothèques de haut niveau comme NumPy) exige une manipulation rigoureuse des listes de listes. D'un point de vue de la complexité asymptotique, le produit matriciel naïf de deux matrices $A \in \mathcal{M}_{n,p}(\mathbb{R})$ et $B \in \mathcal{M}_{p,q}(\mathbb{R})$ requiert $\mathcal{O}(n \cdot p \cdot q)$ opérations arithmétiques. Cette borne peut être théoriquement améliorée par l'algorithme de Strassen ou de Coppersmith-Winograd. De plus, la stabilité numérique est une préoccupation majeure : l'inversion de matrice par pivot de Gauss partiel nécessite de permuter les lignes pour éviter la division par des pivots proches de zéro, ce qui amplifierait dramatiquement les erreurs d'arrondi (conditionnement pathologique). C'est le fondement de la décomposition LU avec pivotation.

TP 04 : Implémentation du Pivot de Gauss

Objectif

Ce TP vise à implémenter l'algorithme du pivot de Gauss pour la résolution de systèmes linéaires et l'inversion de matrices.

Implémentation Python pur

```
def gauss_elimination(A, b):  
    # Implémentation du pivot de Gauss  
    n = len(A)  
    # Forward elimination  
    for i in range(n):  
        # Chercher le pivot max  
        max_el = abs(A[i][i])  
        max_row = i  
        for k in range(i+1, n):  
            if abs(A[k][i]) > max_el:  
                max_el = abs(A[k][i])  
                max_row = k  
  
        # Swap row  
        A[i], A[max_row] = A[max_row], A[i]  
        b[i], b[max_row] = b[max_row], b[i]  
  
        # Make all rows below this one 0 in current column  
        for k in range(i+1, n):  
            c = -A[k][i]/A[i][i]  
            for j in range(i, n):  
                if i == j:  
                    A[k][j] = 0  
                else:  
                    A[k][j] += c * A[i][j]  
            b[k] += c * b[i]  
  
    # Back substitution  
    x = [0 for i in range(n)]  
    for i in range(n-1, -1, -1):  
        x[i] = b[i]/A[i][i]  
        for k in range(i-1, -1, -1):
```

```

        b[k] -= A[k][i] * x[i]
    return x

A = [[2, 1, -1], [-3, -1, 2], [-2, 1, 2]]
b = [8, -11, -3]
x = gauss_elimination(A, b)
assert x == [2.0, 3.0, -1.0]

```

Explications

L'algorithme du pivot de Gauss est fondamental pour résoudre des systèmes linéaires. L'implémentation inclut la recherche du pivot partiel pour la stabilité numérique.

Exégèse Algorithmique et Théorique : L'implémentation algorithmique de ces opérations matricielles en pur Python (sans bibliothèques de haut niveau comme NumPy) exige une manipulation rigoureuse des listes de listes. D'un point de vue de la complexité asymptotique, le produit matriciel naïf de deux matrices $A \in \mathcal{M}_{n,p}(\mathbb{R})$ et $B \in \mathcal{M}_{p,q}(\mathbb{R})$ requiert $\mathcal{O}(n \cdot p \cdot q)$ opérations arithmétiques. Cette borne peut être théoriquement améliorée par l'algorithme de Strassen ou de Coppersmith-Winograd. De plus, la stabilité numérique est une préoccupation majeure : l'inversion de matrice par pivot de Gauss partiel nécessite de permuter les lignes pour éviter la division par des pivots proches de zéro, ce qui amplifierait dramatiquement les erreurs d'arrondi (conditionnement pathologique). C'est le fondement de la décomposition LU avec pivotation.

TP 05 : Calcul du Déterminant Récursif et Inversion

Objectif

L'objectif est d'implémenter le calcul du déterminant d'une matrice par la méthode des cofacteurs et de l'utiliser pour calculer la matrice inverse (matrice des cofacteurs).

Implémentation Python pur

```
def get_minor(m, i, j):
    return [row[:j] + row[j+1:] for row in (m[:i] + m[i+1:])]

def determinant(m):
    # Cas de base
    if len(m) == 2:
        return m[0][0]*m[1][1] - m[0][1]*m[1][0]

    det = 0
    for c in range(len(m)):
        det += ((-1)**c) * m[0][c] * determinant(get_minor(m, 0, c))
    return det

A = [[1, 2, 3], [0, 1, 4], [5, 6, 0]]
assert determinant(A) == 1

def inverse(m):
    det = determinant(m)
    if det == 0:
        raise ValueError("Matrix is singular")

    # Cas de base
    if len(m) == 2:
        return [[m[1][1]/det, -1*m[0][1]/det],
                [-1*m[1][0]/det, m[0][0]/det]]

    # Cofactors
    cofactors = []
    for r in range(len(m)):
        cofactor_row = []
        for c in range(len(m)):
            minor = get_minor(m, r, c)
```

```

        cofactor_row.append((-1)**(r+c)) * determinant(minor))
    cofactors.append(cofactor_row)

    # Transpose and divide by determinant
    inv = [[cofactors[c][r]/det for c in range(len(m))] for r in range(len(m))]
    return inv

A_inv = inverse(A)
expected_inv = [[-24.0, 18.0, 5.0], [20.0, -15.0, -4.0], [-5.0, 4.0, 1.0]]
assert A_inv == expected_inv

```

Explications

Bien que le calcul du déterminant par cofacteurs soit en $O(n!)$ et donc inefficace pour de grandes matrices, il est fondamental d'un point de vue théorique pour comprendre la formule de Cramér et l'inverse d'une matrice.

Exégèse Algorithmique et Théorique : L'implémentation algorithmique de ces opérations matricielles en pur Python (sans bibliothèques de haut niveau comme NumPy) exige une manipulation rigoureuse des listes de listes. D'un point de vue de la complexité asymptotique, le produit matriciel naïf de deux matrices $A \in \mathcal{M}_{n,p}(\mathbb{R})$ et $B \in \mathcal{M}_{p,q}(\mathbb{R})$ requiert $\mathcal{O}(n \cdot p \cdot q)$ opérations arithmétiques. Cette borne peut être théoriquement améliorée par l'algorithme de Strassen ou de Coppersmith-Winograd. De plus, la stabilité numérique est une préoccupation majeure : l'inversion de matrice par pivot de Gauss partiel nécessite de permuter les lignes pour éviter la division par des pivots proches de zéro, ce qui amplifierait dramatiquement les erreurs d'arrondi (conditionnement pathologique). C'est le fondement de la décomposition LU avec pivotation.